



分类号:

密级:

论文编号:

学号: 52190401181

重庆理工大学硕士学位论文

锥桶环境下的智能车辆感知与路径规划控制

研 究 生: 施友宁

指 导 教 师: 刘西教授

董立强高级工程师

学 位 类 型: 专业学位

专业学位类别: 工程硕士 (车辆工程领域)

研 究 方 向: 智能驾驶

培 养 单 位: 车辆工程学院

论文完成时间: 2022 年 3 月 25 日

论文答辩日期: 2022 年 5 月 28 日

Category Number:

Level of Secrecy:

Serial Number :

Student Number: 52190401181

Master's Dissertation of Chongqing University of Technology

Intelligent vehicle perception and path planning control in cone environment

Postgraduate: Shi Youning

Supervisor: Professor Liu Xi

Senior Engineer Dong Liqiang

Degree Category: Professional Degree

Specialty: Master of Engineering

(Vehicle Engineering)

Research Direction: Intelligent vehicle

Training Unit: Institute of vehicle engineering

Thesis Deadline: March 25, 2022

Oral Defense Date: May 28, 2022

摘要

交通锥桶常常被放置在道路中间、危险地区和施工地段等，用以分隔车流，引导交通、以及在放置在场馆与停车场，用以引导车辆驶入。本文针对车辆在锥桶环境下的锥桶感知、场景决策、路径规划与路径跟踪控制进行研究，通过本文所研究的方法使车辆能够在锥桶环境下安全无碰撞的通过。

首先，使用 YOLOv3-Tiny 算法对锥桶进行检测识别，并针对该算法在检测锥桶时存在的问题，通过（1）增加网络输入尺寸；（2）更改损失函数；（3）更改 YOLOv3-Tiny 主干网络；（4）融合浅层特征，构建新的特征金字塔等方向对其进行了改进，然后对改进后的算法进行了实验，改进后相较于改进前对锥桶检测的精确率与召回率分别提升了 8.7 与 25.7 个百分点，检测效果得到了显著的提升。

随后依据锥桶尺寸这一先验信息，实现了三种锥桶在相机坐标系下的定位方法。通过分析锥桶的几何特征，设计了锥桶关键点检测网络，并制作了数据集，通过模型训练得到了锥桶关键点检测模型，然后利用 PnP 算法实现了第一种基于锥桶关键点的 PnP 定位方法；通过将锥桶抽象为圆柱体，将其圆心横截面矩形在图像坐标系上的投影近似为锥桶的检测框，利用 PnP 算法实现了第二种基于检测框的 PnP 定位方法；通过分析相机成像模型，利用针孔相机成像模型实现了第三种基于针孔相机成像模型的锥桶定位方法。最后在实车实验平台上，利用两颗工业摄像头进行了定位实验，验证了其定位精度，可以满足车辆在行驶中的锥桶定位要求。

然后选取了四种锥桶的道路交通应用场景，并进行了简化，设计了相应的决策场景类型、决策算法以及路径规划算法，同时针对混合车道场景与纯锥桶场景下规划得到的路径曲率不连续的问题，设计了多段贝塞尔曲线进行拼接来平滑路径，包括五阶贝塞尔曲线拼接五阶贝塞尔曲线、五阶贝塞尔曲线拼接三阶贝塞尔曲线。

最终依托 2021 i-VISTA 自动驾驶汽车挑战赛-低速无人驾驶挑战赛，提取了比赛场景的参考路径，基于车辆运动学模型，搭建了 Pure Pursuit 路径跟踪控制器，并基于 ROS 系统进行了路径控制算法的仿真，并借助参赛车辆进行了实车路径跟踪实验，对算法的路径跟踪效果进行了验证。

关键词：YOLOv3-Tiny；锥桶检测；场景决策；路径规划；路径跟踪控制

Abstract

Traffic cones are often placed in the middle of roads, hazardous areas, construction sites, etc. to separate traffic and guide traffic, and in venues and parking lots to guide vehicles. In this paper, we study the cone perception, scenario decision, path planning and path tracking control of vehicles in cone bucket environment, so that vehicles can pass safely and collision-free in cone bucket environment by the method studied in this paper.

First, the YOLOv3-Tiny algorithm is used to detect and identify cone buckets, and the algorithm is improved by (1) increasing the network input size, (2) changing the loss function, (3) changing the YOLOv3-Tiny backbone network, (4) fusing shallow features and constructing a new feature pyramid to address the problems of the algorithm in detecting cone buckets, and then the improved algorithm is The accuracy and recall rates of the improved algorithm were improved by 8.7 and 25.7 percentage points, respectively, compared with those before the improvement, and the detection effect was significantly improved.

Then, based on the a priori information of cone bucket size, three methods of positioning cone buckets in the camera coordinate system were implemented. By analyzing the geometric characteristics of the conical bucket, a conical bucket key point detection network is designed and a dataset is produced, a conical bucket key point detection model is obtained through model training, and then the first PnP localization method based on the conical bucket key point is implemented by using the PnP algorithm; by abstracting the conical bucket as a cylinder, the projection of its circular cross-section rectangle on the image coordinate system is approximated as the detection frame of the conical bucket, and the PnP The second PnP positioning method based on the detection frame is implemented by the algorithm; the third cone bucket positioning method based on the imaging model of the pinhole camera is implemented by analyzing the imaging model of the camera. Finally, a positioning experiment was conducted on a real vehicle experimental platform using two industrial cameras to verify its positioning accuracy, which can meet the cone bucket positioning requirements of the vehicle in motion.

Then four cone bucket road traffic application scenarios were selected and simplified, and the corresponding decision scenario types, decision algorithms and path planning algorithms were designed. Meanwhile, for the problem of discontinuous curvature of the

paths planned in mixed lane scenarios and pure cone bucket scenarios, multiple Bessel curves were designed for splicing to smooth the paths, including fifth-order Bessel curves spliced with fifth-order Bessel curves and fifth-order Bessel curves spliced with third-order Bessel curves.

Finally, based on the 2021 i-VISTA Autonomous Vehicle Challenge - Low Speed Unmanned Driving Challenge, the reference path of the competition scenario was extracted, the Pure Pursuit path tracking controller was built based on the vehicle kinematic model, and the path control algorithm was simulated based on the ROS system, and the path tracking effect of the algorithm was verified by conducting a real vehicle path tracking experiment with the participating vehicles.

Key words: YOLOv3-Tiny; Cone Detection; Scenario Decision Making; Path Planning; Path Tracking;

目 录

摘要	I
Abstract	III
1 绪论	1
1.1 课题研究背景及意义	1
1.2 国内外应用及研究现状	2
1.2.1 目标检测与视觉测距研究现状	2
1.2.2 局部路径规划研究现状	3
1.2.3 路径跟踪控制研究现状	4
1.3 研究内容与章节安排	5
2 基于深度学习的交通锥桶检测	7
2.1 卷积神经网络基本概念	7
2.1.1 卷积层	7
2.1.2 激活函数	8
2.1.3 池化层	9
2.1.4 全连接层	10
2.2 基于深度学习的目标检测算法	10
2.2.1 YOLOv3-Tiny 目标检测算法原理	11
2.2.2 锥桶数据集	13
2.2.3 YOLOv3-Tiny 模型改进方法	15
2.2.4 评价指标	22
2.2.5 模型训练	22
2.2.6 实验结果分析	23
2.3 本章小结	25
3 基于先验信息的交通锥桶定位	27
3.1 基于 PNP 算法的锥桶定位方法	27
3.1.1 PNP 算法原理	27
3.1.2 基于锥桶关键点的 PnP 定位方法	29
3.1.3 基于锥桶检测框的 PnP 定位方法	32
3.2 基于针孔相机模型的锥桶定位方法	33

3.2.1 针孔相机成像模型	33
3.2.2 定位方法流程	35
3.3 锥桶定位实验	37
3.3.1 相机标定	38
3.3.2 定位方法试验对比分析	39
3.4 本章小结	40
4 基于场景决策的路径规划	41
4.1 场景工况选取与简化	41
4.2 场景决策算法设计	43
4.3 不同场景下的路径规划算法	44
4.3.1 换道场景下的路径规划	44
4.3.2 纯车道场景下的路径规划	47
4.3.3 混合车道场景下的路径规划	47
4.3.4 纯锥桶场景下的路径规划	48
4.4 基于贝塞尔曲线的路径平滑处理	49
4.4.1 五阶贝塞尔曲线拼接五阶贝塞尔曲线	52
4.4.2 五阶贝塞尔曲线拼接三阶贝塞尔曲线	53
4.5 仿真实验分析	54
4.5.1 场景决策仿真实验	54
4.5.2 基于贝塞尔曲线的路径平滑拼接实验	55
4.5.3 换道场景下的路径规划仿真实验	56
4.5.4 纯车道场景下的路径规划仿真实验	58
4.5.5 混合车道场景下的路径规划仿真实验	59
4.5.6 纯锥桶场景下的路径规划仿真实验	61
4.6 本章小结	62
5 基于 Pure Pursuit 算法的路径跟踪控制	63
5.1 参考路径提取	63
5.2 基于 Pure Pursuit 算法的路径跟踪	64
5.2.1 车辆运动学模型	64
5.2.2 Pure Pursuit 算法	66
5.3 控制器效果分析	67
5.3.1 仿真分析	67
5.3.2 实车分析	69

5.4 本章小结	71
6 总结与展望	73
6.1 总结	73
6.2 展望	74
参考文献	77
个人简历、在学期间发表的学术论文及取得的研究成果	83

1 绪论

1.1 课题研究背景及意义

对于道路中行驶的车辆，道路上的障碍物可大致分其他交通参与者，例如汽车、自行车、行人以及用于道路约束的交通锥桶。其中交通锥桶由于其便利性，经常出现在道路交通场景中，起到标识危险场景、隔离施工路段等分隔车流、引导交通的作用，以及放置在场馆与停车场，用以引导车辆驶入，如图 1.1 所示。交通锥桶由于尺寸较小，一旦驾驶员注意力不集中，极易导致各种交通事故。例如，2013 年北京丰台区发生了一起大型交通事故，事故原因为大货车司机撞上正在交通锥桶隔离带中作业的道路养护人员，后又冲过隔离带，与对向驶来的车辆发生严重碰撞，事故造成 5 人死亡，2 人受伤；2014 年沈阳市大东区沈铁路周边，一辆公交车因未注意到交通锥桶，致使一位在道路中进行正常垃圾清理作业的女环卫工当场死亡；2016 年沪蓉高速，由于道路后方高速车辆未及时注意到交通锥桶导致车辆无法避开正在道路中处理一起三车追尾事故的交警以及事故现场，致使发生了“二次追尾”，造成一人死亡、七人受伤以及现场车辆不同程度受损。

由于交通锥桶具有使用场景灵活多变、价格便宜、实用性强与便于使用得特点，且在交通环境中出现频率高，因此对此类场景进行研究有着重要的现实意义。



图 1.1 真实交通场景中的锥桶

障碍物检测是智能驾驶感知模块的主要功能，它为智能车路径规划模块提供当前局部环境内的可通行区域及障碍物位置。目前，已有众多学者针对车辆、行人与自行车等交通参与者的检测识别进行了研究，并取得了显著成果，而对于交通锥桶的检测识别的研究开展较少，因此具有较高的研究意义。

本文重点研究车辆在交通锥桶环境下，基于视觉的交通锥桶检测及其定位方法以及其路径规划控制方法，并对交通锥桶的感知定位及跟踪控制进行实车验证。通

过本文所研究的交通锥桶检测与定位及其路径规划方法可以使智能车辆在道路受到交通锥桶约束时，能够安全无碰撞通过。

1.2 国内外应用及研究现状

1.2.1 目标检测与障碍物定位研究现状

（1）目标检测研究现状

目标检测是计算机视觉领域内的重要研究内容，以及众多计算机视觉任务的基础^[1]。大量应用于人脸检测、自动驾驶等方向。随着计算机算力的提升以及大数据时代的来到，目标检测算法经历了从传统机器视觉向深度学习算法发展的过程。

传统机器视觉严重依赖人工设置的特征进行目标检测，对于复杂道路环境中的交通锥桶，由于其尺寸较小且周围环境复杂，因此基于传统机器视觉对交通锥桶进行检测难以满足当前智能驾驶的需求。而深度学习算法从大量数据中自行提取目标物特征，然后基于提取的特征进行障碍物检测，裨益于卷积神经网络（Convolution Neural Network, CNN）和候选区域（region proposal）的提出^[2]，深度学习具有效率高、精度高等优点，适宜交通锥桶检测。

针对交通锥桶检测，国内外已有学者分别基于传统机器视觉与深度学习展开了研究。国内—杜晨健^[3]依据交通锥桶的颜色特征和连通域分布实现交通检测，然后依据摄像头与交通锥桶之间的几何关系实现定位。郅磊澎等^[4]基于 OpenCV 库中的图像处理算法对交通锥桶进行检测。卢远志^[5]提出的算法主要包括目标检测和识别两个主要步骤，首先依据其颜色、几何等先验知识进行快速定位和提取感兴趣区域，然后对结果使用不变矩和 BP 神经网络的方法进行分类。李松斌等^[6]基于交通锥桶颜色轮廓对其进行检测，首先通过红色二值图得到疑似区域，继而通过红白颜色二值图得到备选区域，最终进行颜色模式匹配得到交通锥桶的检测结果。史林波^[7]首先基于交通锥桶的颜色特征与几何形态以及连通域分布得到待定区域，然后对检测结果进行处理，筛除其中可信度不高或错误的检测结果。

国外—HUANG Yong 等^[8]提出了一种基于视觉与雷达的传感器融合方法来检测交通锥桶，该算法包括两部分，首先将雷达数据映射至图像上，从中筛选出相应的小图像块，然后基于轮廓特征的方法，用于确定图像块中的障碍物是否为圆锥体。Ankit Dhall 等^[9]使用 yolov2 目标检测算法对交通锥桶进行识别。

（2）障碍物定位研究现状

在智能驾驶领域现有的障碍物定位研究中，依据传感器的不同，主要分为基于雷达传感器的障碍物定位以及基于摄像头传感器的障碍物定位。

基于毫米波雷达和激光雷达传感器的障碍物定位,具有精度高的优点,同时也存在价格高且易受到车辆周围环境的影响;基于摄像头传感器的障碍物定位具有价格便宜且通过多摄像头融合的方式可以达到较高精度的优点,主要分为单目视觉与立体视觉,其中,单目视觉算法简单、计算量小且实时性好^[10];立体视觉精度高,但计算过程复杂,实时性较差^[11]。

国内—吴强等^[12]使用消失点估计的优化算法完成了对摄像机的标定工作,配合使用视觉检测算法完成了对停车场的车辆位置的定位,从而实现了在停车场时对空闲车位的空间位置锁定。王振等^[13]基于障碍物与其接触面边缘的颜色不同这一属性,结合视觉传感器成像原理实现了障碍物定位。韩峰^[14]利用 SURF 算法对所获取的图像进行特征点的提取和匹配,实现对物体的定位。毛先胤等^[15]基于双目视觉系统,使用半全局立体匹配算法获得深度图像,然后使用加权最小二乘滤波对其进行优化,最后通过重投影获得障碍物的位置。王立勇等^[16]基于双目视觉生成的深度图像与交通锥桶的检测结果匹配,得到交通锥桶的位置信息。

国外—Daniel Glasner 等^[17]描述了一种从 2D 图像对刚性 3D 对象进行类别级别检测和视点估计的方法。与许多现有方法相比,他们将 3D 推理与基于外观的投票架构直接集成在一起,投票方法采用了联合检测和观点假设空间的新型参数化,可以有效地收集证据。Shubham Tulsiani 等^[18]提出了一种通过确定视点去解释粗略姿势估计和关键点预测以便于获取更多的细节去描述姿势估计问题。Homayoun Moradi 等^[19]提出了一种利用卷积网络的输出,以便了解对象的依赖性并在一定时间内保留其标签。此外,在另一幅图像中发现相同的对象,并使用提出的比较立体匹配算法计算其视差值。3D 位置可以使用视差值和对极几何形状来确定。

交通锥桶检测从传统的计算机视觉发展到深度学习目标检测,检测的效果得到了提升但对硬件的算力要求较高。但对于复杂场景下的检测效果还有改善提升的空间。对于物体定位,传统的双目视觉定位,在全局中进行特征匹配,计算量大且容易出现误匹配,存在改善提升的空间,单目视觉方法简单,对算力要求较低且具有一定的精度。

1.2.2 局部路径规划研究现状

局部路径规划是指,在智能车辆实时的感知范围内,依据上层系统给出的局部目标点,规划出从当前位置到目标点的无碰撞路径以控制车辆运动^[20]。

国内—韩月起等^[21]结合凸近似原理与模型预测控制原理,并考虑了道路约束、车辆约束、横向加速度以及车距等影响,提出了一种复杂动态场景下的路径规划方法。杜明博^[22]为了使规划得到的路径更符合真实人类的驾驶行为,结合驾驶员视觉行为,提出了一种基于驾驶员视觉行为的 RRT 运动规划算法。李麒等^[23]在路径规划

算法中加入车辆稳定性与交通流的影响,实现规划得到路径满足提升安全性与能量利用率的要求。赵杰等^[24]基于果蝇优化算法,针对该算法搜索时间长、易陷入局部最优的缺点,加入了多种群之间的信息交流,并引入交叉操作。李刚等^[25]采用四边重心算法求解除无人驾驶赛车在赛道场景下的路径点,然后使用三次样条拟合。王炳琪^[26]采用最长枝叶扩展和目标导引策略对 RRT 算法进行了改进,即利用叶节点距起点的树枝长度和该叶节点距目标点的欧式距离构造度量函数进行扩展,并依据车辆转向模型对路径增加了车辆运动学约束。

国外—Sreenatha Anavatti 等^[27]提出了一种基于连续蚁群的多车路径规划算法,包括无人车路径规划和多无人车协同。使用基于概率的随机游走策略在布朗运动和柯西运动之间切换来平衡算法,以修改当前规划方法,自适应航路点修复策略和重新初始化策略提高了算法在寻找可行路径方面的性能。Pedro B.Fernandes 等^[28]提出了一种改进粒子群优化算法。这种算法允许创建具有更多样化种群的粒子群,从而增加了解决方案空间内的搜索范围,还引入了一种称为直线法的技术,可以有效减少车辆导航空间路线优化所需的点数。Mohammed MS Ibrahim 等^[29]使用 Q-learning 算法进行局部路径规划。Aye Aye Maw 等^[30]提出了一种新的随时增量搜索算法,称为改进的随时动态 A* (iADA*)。该算法基于当前流行的随时启发式搜索算法,即 Anytime Dynamic A* (ADA*)。iADA* 算法改进了路径长度的计算并降低了整个搜索过程中路径的计算频率,从而显著加快了搜索速度。当局部变化受到影响时,该算法可以在复杂的动态搜索环境提供有效的解决方案。

由于智能车可看作是一个四轮机器人,因此路径规划算法大多是从机器人领域发展而来,大致可分为基于图搜索的规划方法;基于采样的路径规划方法;基于人工智能的路径规划算法;基于几何模型的路径规划算法;基于遗传进化的路径规划算法。而这些路径规划算法规划出的路径往往有突变,曲率不连续,因此不适用于车辆,需要对规划算法进行改进或对规划好的路径进行平滑处理。

1.2.3 路径跟踪控制研究现状

路径跟踪控制是指,在获知期望路径后,通过算法计算出车辆的速度与前轮转角来控制车辆的横纵向运动,实现路径跟踪。

国内—白国星等^[31]在非线性模型预测控制的基础上,使用三次多项式拟合获得了最佳预测时域和纵向控制率。任玥等^[32]针对车辆的横纵向速度难以精确测量的问题,建立了状态估计系统来降低误差,并使用自适应滑模算法实现转向控制。负海涛等^[33]针对车辆高速避让时控制效果差的问题,利用神经网络在线整定控制器参数以提升控制效果。金立生等^[34]提出了一种考虑驾驶员特性的算法,该算法首先将不同的驾驶员特性分类,然后基于数据得到分类模型,最后将其与模型预测控制相结

合。

国外—Kibeom Lee 等^[35]提出了一种基于模型具有自适应 Q 矩阵的线性二次高斯 (LQG) 控制, 以有效地设计任何给定目标车辆的路径跟踪控制器, 同时有效处理定位和路径产生的噪声和误差问题。Hamid Taghavifar 等提出了一种基于类指数滑动模式的模糊 2 型神经网络方法的新型鲁棒自适应间接控制方法, 用于增强道路自主车辆的路径跟踪性能, 这些车辆受与车辆标称转弯刚度、道路-轮胎附着系数、惯性参数和纵向速度的影响。Lukas Hewing 等^[36]提出了一个模型预测控制 (MPC) 方法, 将名义系统与动力学建模的附加非线性部分集成在一起作为高斯过程 (GP), 描述了一种机会受限的 MPC 问题的原理化方法, 该方法考虑了 GP 模型提供的剩余不确定性以实现谨慎控制。Ardashir Mohammadzadeh 等^[37]提出了一种鲁棒的模糊控制方法, 用于自动驾驶公路车辆 (ARV) 的横向路径跟踪。通过基于非平稳模糊集的新非单模糊系统在线估计 ARV 的动力学。确保了所提方法的渐近稳定性, 并基于 Lyapunov 稳定性定理推导了所提出模糊系统的自适应律。验证了所提出的控制方法的鲁棒性, 适用于车辆系统在结构化和未建模的不确定性以及不同扰动下以不同的前进速度执行双车道变更操纵的情况。

路径跟踪控制的方法主要分为经典控制理论、现代控制理论以及智能控制理论, 近年来基于人工智能的控制策略得到了发展。模型预测控制作为该领域的研究热点具有对模型要求不高、控制效果好等优点, 同时由于算法自身特点, 也具有计算复杂、算力要求高的不足。

1.3 研究内容与章节安排

本文主要研究交通锥桶环境下基于单目视觉的智能车辆感知与规划控制, 主要包含三部分内容: 交通锥桶检测与其定位、智能车辆所处场景决策与局部路径规划、路径跟踪控制。交通锥桶检测与其定位的作用是在车辆行驶过程中检测道路中的交通锥桶并获得其相对于车辆的位置坐标; 场景决策与规划部分, 根据车辆周围环境信息判断车辆当前具体行驶场景, 继而根据所处场景的不同进行不同的局部路径规划; 路径跟踪控制部分根据参考路径对车辆进行控制。

论文的主要研究内容主要分为以下六章:

第一章绪论。对本文的研究背景与意义进行了概述, 并对国内外目标检测、视觉测距、局部路径规划与跟踪控制的研究进行了综述与分析, 最后介绍本文的主要研究内容。

第二章基于深度学习的交通锥桶检测。针对 YOLOv3-Tiny 目标检测算法在检测交通锥桶时存在的问题, 研究针对检测小目标物-交通锥桶的 YOLOv3-Tiny 算法改进

措施，并对改进后的算法进行实验验证。

第三章基于交通锥桶先验信息的锥桶定位。基于交通锥桶先验信息研究交通锥桶相对于车辆的定位，并基于实车实验平台展开其定位实验。

第四章基于场景决策的路径规划。依据交通锥桶在真实道路交通场景中的应用，选取四种交通锥桶环境下的工况，确定本文需要决策的场景类型，并设计决策算法；然后分别设计本文所决策的四种场景下的路径规划算法，进行仿真验证。

第五章基于 Pure Pursuit 算法的路径跟踪控制。基于车辆运动学模型，设计 Pure Pursuit 算法；在 ROS 系统环境下，验证所设计控制器的路径跟踪控制性能，并进行实车实验。

第六章总结与展望。总结本文的研究内容并针对本文的不足进行展望。

2 基于深度学习的交通锥桶检测

交通锥桶起到引导交通、分隔车流和标示危险场景等作用，因此需要被识别。对于智能驾驶车辆，交通锥桶识别有两个难题。1) 在交通锥桶场景下，车辆的速度在城市中一般可达到 30~40km/h。为了确保车辆行驶的安全性，必须在至少 25 米外检测到并定位锥体。同时，交通锥桶的尺寸相对于其他如车辆行人等交通参与者，从摄像机的角度来看，这是一个非常小的物体，特别是在这么长的距离内。这种特殊性使得传感器很难提取特征。2) 实时性成为第二个问题。本文基于单目视觉来实现道路环境中交通锥桶的检测。

2.1 卷积神经网络基本概念

以人类神经元连接结构为灵感的神经网络发展至今，诞生了多种经典的神经网络结构，卷积神经网络（Convolutional Neural Network, CNN）作为其中的一员，在图像处理领域有着广泛的应用。卷积神经网络是一种前馈神经网络，整个网络表现出单一的可区分的分数函数，即从一端的原始图像像素到另一端的类别分数^[38]。卷积神经网络的一般结构如图 2.1 所示。

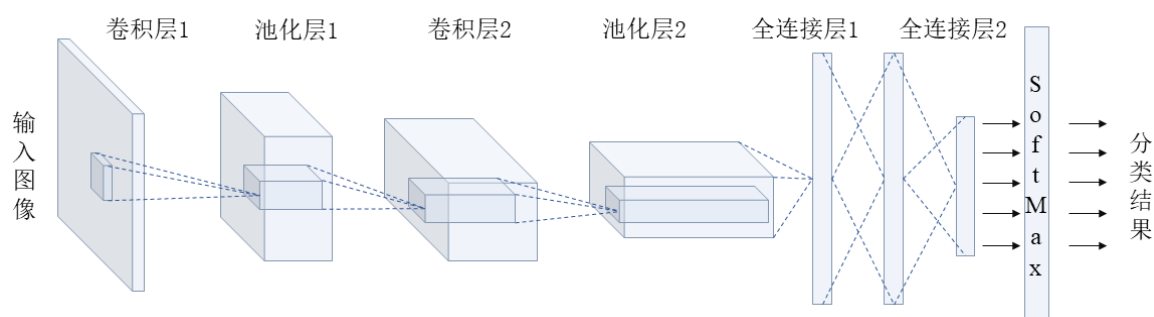


图 2.1 用于图像分类的卷积神经网络

2.1.1 卷积层

对输入图像的特征进行提取是卷积神经网络的主要功能，简单地说，就是对原始图像应用过滤器，将其分割成更小的部分，这个功能由卷积层实现，其原理如图 2.2 所示。卷积层由几个特征映射组成，这些特征映射执行相同的操作，即对来自上一层的图像应用特定的过滤器（内核）。每个特征地图将在不同的位置寻找一个单一的特征，例如在较低的级别可以搜索一条曲线，而在较高的级别可以搜索一只猫或

一张脸。因此，多特征映射将构成多特征识别。

LeCun 在其文章^[39]中给出了卷积的数学表达式。 l 层的输出 $Y^{(l)}$ 将有 $N^{(l)}$ 个特征图， i^{th} 个特征图表示为 $Y_i^{(l)}$ ，由式(2.1)给出。

$$Y_i^{(l)} = \sum_{j=1}^{N^{(l-1)}} K_{i,j}^{(l)} * Y_j^{(l-1)} + B_i^l \quad (2.1)$$

式中： $*$ 为卷积运算符， $K_{i,j}^{(l)}$ 为卷积核，其中 i 为上一层特征图的索引， j 为当前层的特征图的索引， B_i^l 为上一层特征图的偏差。

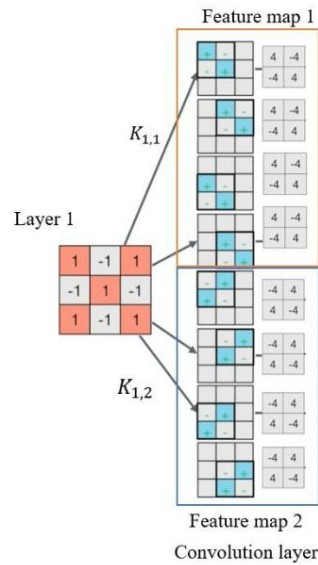


图 2.2 卷积示意图

2.1.2 激活函数

激活函数是卷积神经网络中至关重要的组成部分，如果没有它，那么网络只能进行简单的线性变换即只能拟合线性函数。然而，由于线性函数的导数为常数这个性质致使神经网络进行训练（参数反向传播）时，无法更新误差以调整网络结构中各个层的神经元权重参数以达到最优从而提高最终的预测结果。而通过使用非线性激活函数就能解决这个问题，根据应用场景的不同，非线性激活函数可能会有所不同。图 2.3 显示了最常用的非线性激活函数方法。

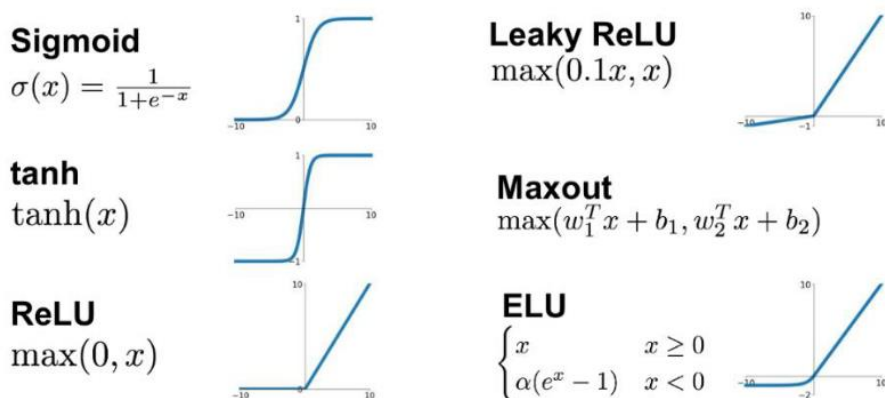


图 2.3 常用的非线性激活函数

2.1.3 池化层

在神经网络设计过程中，若直接对卷积层提取到的大量特征进行分类操作，不但会使网络训练结果出现过拟合现象，且需要大量的算力，导致训练耗时增加。斯坦福大学深度学习教程网站^[40]的一个例子对此进行了说明。一个大小为 96×96 像素的图像，在 8×8 输入上具有 400 个特征，将导致每次卷积的输出大小为 $(96-8+1) \times (96-8+1) = 7921$ ，可使用 CNN 斯坦福课程^[41]中提到的公式(2.2)计算：

$$\text{Output size} = \frac{W - K + 2P}{S} + 1 \quad (2.2)$$

式中， W 为图片像素大小， K 为滤波器的大小， S 为步长， P 为填充的大小，调整图像以使滤波器能够正常工作。

那么最后的图片结果向量为每个样本图像有 $7921 \times 400 = 3168400$ 个特征。

为此，设计了池化层（Pool）来对卷积的输出进行下采样。此外，在水平合并的情况下，平移不变性将添加到特征中。从数学上讲，输出的张量大小为 $m_1^{(l)} m_2^{(l)} m_3^{(l)}$ ：

$$\begin{aligned} m_1^{(l)} &= m_1^{(l-1)} \\ m_2^{(l)} &= \frac{(m_2^{(l-1)} - F^{(l)})}{S^{(l)}} + 1 \\ m_3^{(l)} &= \frac{(m_3^{(l-1)} - F^{(l)})}{S^{(l)}} + 1 \end{aligned} \quad (2.3)$$

式(2.3)中， F 为池化层滤波器的大小， S 为步长。

池化层有两种类型：最大池化、平均池化。如图 2.4 所示。平均池化取滤波器内均值，最大池化取滤波器中最大值。

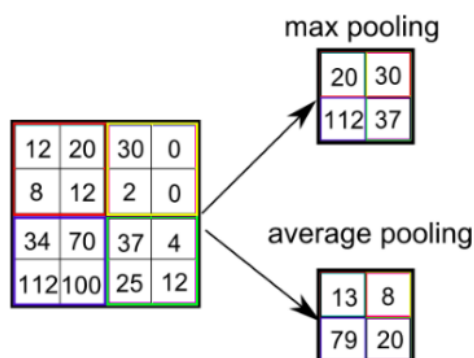


图 2.4 池化层示意图

2.1.4 全连接层

CNN 中全连接层的目标是处理和分类前一层的结果。它们被形成为具有两层或三层的多层感知器。输出将是一个向量，其元素是属于网络训练的每个标签的输入图像的概率。

2.2 基于深度学习的目标检测算法

目标检测发展至今，诞生了很多经典的算法。表 2.1 列出了几种经典算法的特点以及优势。因为 YOLO 系列算法相比于其他几种算法，在提高了算法检测速度的同时，保持了一定的精度，且在工业上已有大规模的落地应用，因此本文选择 YOLO 系列算法进行交通锥桶检测。

表 2.1 典型卷积神经网络的特点与优势

网络名称	特点	优势	文献
SPPNet	替换池化层为为空间金字塔层（SPP），以保留特征空间信息。	可变输入图像大小	[42]
R-CNN	三个步骤：区域建议、特征提取和边界框。	具有良好的精度	[43]
YOLO	将输入图片划分为网格，分配置信度与概率。	检测速度快精度好	[44]
SSD	对输入图像进行单次前馈传递。这将生成具有不同比例和置信度分数的固定大小的边界框集合。	实时目标检测和分类速度高达 46 fps	[45]

2.2.1 YOLOv3-Tiny 算法原理

YOLO 系列算法由 Joseph Redmon 于 2016 年提出，建立在他开发的 Darknet 框架下。该算法的主要目标是以一种简单的方式运行单个卷积神经网络来进行目标检测。

与其他的目标检测算法相比，YOLO 系列算法的主要区别在于，它不使用滑动窗口法来遍历图像，而是通过将图像划分为 $S \times S$ 个网格区域，全局地考虑图像。如果有物体的中心点落在某网格内，则该网格预测 x 、 y 、 w 、 h 以及置信度 C ，对于每个类，都有一个边界框 B 的定义。 x 、 y 为边界框中心在图像坐标系下的坐标， w 、 h 为边界框的宽度和高度。每个对象的最佳边界框从一组具有各种尺寸的预定义锚框中选择并替换。

YOLO 系列算法考虑的最后一件重要的事情是它的损失函数。它可以分为两部分，一部分用于目标分类，另一部分用于目标定位。Lilian Weng^[46]的一篇文章以一种比 YOLO 系列算法论文更直观的方式表达了这一点，如式(2.5)所示：

$$\begin{aligned}\mathcal{L}_{\text{loc}} &= \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B 1_{ij}^{\text{obj}} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 + (\sqrt{w_i} - \sqrt{\hat{w}_i})^2 + (\sqrt{h_i} - \sqrt{\hat{h}_i})^2 \right] \\ \mathcal{L}_{\text{class}} &= \sum_{i=0}^{S^2} \sum_{j=0}^B (1_{ij}^{\text{obj}} + \lambda_{\text{noobj}} (1 - 1_{ij}^{\text{obj}})) (c_i - \hat{c}_i)^2 + \sum_{i=0}^{S^2} \sum_{j=0}^B 1_i^{\text{obj}} (p_i(c) - p_i(c))^2 \\ \mathcal{L} &= \mathcal{L}_{\text{class}} + \mathcal{L}_{\text{loc}}\end{aligned}\quad (2.5)$$

式中， $\mathcal{L}$ ：总损失函数；

$\mathcal{L}_{\text{loc}}$ ：目标定位损失函数；

$\mathcal{L}_{\text{class}}$ ：目标分类损失函数；

1_i^{obj} ：判断当前网格内是否有物体，存在则为 1，表示正在对目标分类；

1_{ij}^{obj} ：表示第 i 个网格第 j 个 Bounding Box 是否负责某个目标的检测；

c_i ：表示第 i 个网格的置信度；

$\hat{c}_i$ ：表示预测的置信度；

c ：类集合中的某一个具体类别；

$p_i(c)$ ：表示当前网格内包含类别 c 的条件概率；

λ_{noobj} ：表示类别分类损失权重，论文中使用的值为 0.5；

λ_{coord} ：表示目标定位损失权重，论文中使用的值为 0.5。

而在 YOLO 系列算法中，应用最为广泛的为 YOLOv3 算法，其网络结构如图 2.5 所示。可以看出 YOLOv3 算法的网络结构复杂，而本文只检测锥桶一个类别，无需复杂网络结构，因此本文选择 YOLOv3-Tiny 进行交通锥桶检测。

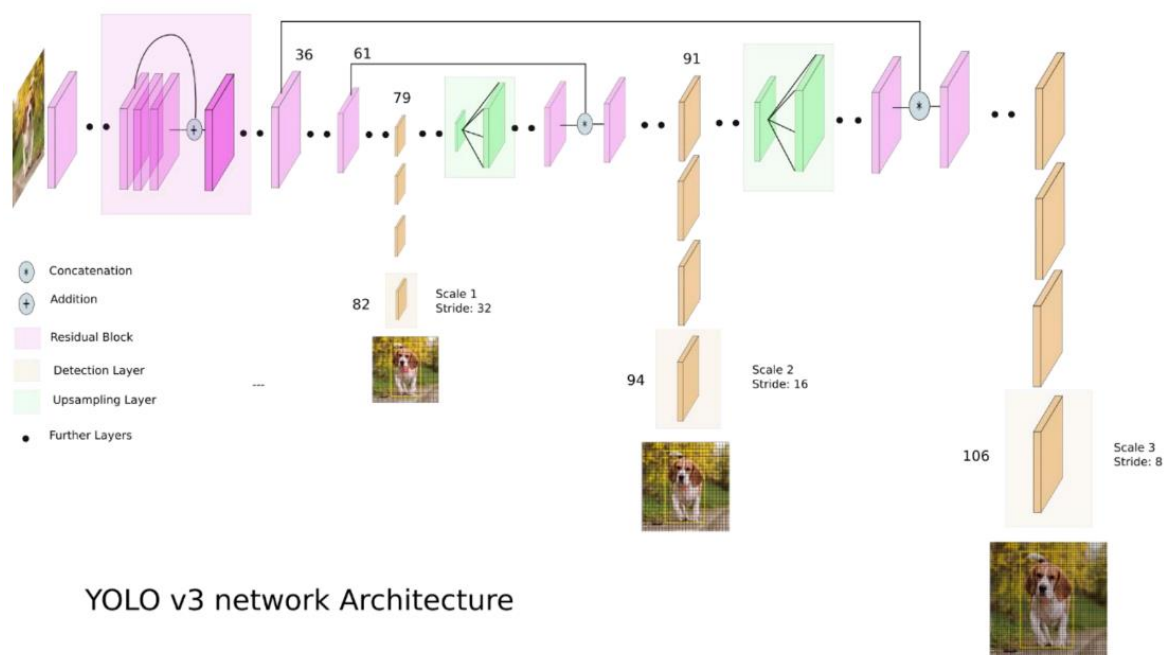


图 2.5 yolov3 网络结构图

YOLOv3-Tiny 算法压缩 YOLOv3 的主干网络为 7 个卷积层, 6 个池化层, 并将特征尺度缩减为两个尺度, 以在降低参数量提高检测速度的同时能够应对大目标物体的检测。其网络架构如图 2.6 所示, 图中 Conv 表示卷积层, Maxpool 表示最大池化层, 虚线内为 YOLOv3-Tiny 的主干网络 (Backbone)。输入图片尺寸为: 416×416 像素的 3 通道 RGB 图像, 最后由两个检测头所对应尺度的特征实现目标检测, 每个检测头输出的张量维度可表示为 $N \times N \times (3 \times (4 + 1 + C))$, 其中 $N \times N$ 表示特征图的尺寸, “3”表示每个特征尺度锚框 (anchor) 的数量, “4”表示边界框的坐标, “1”表示边界框中是否有物体, C 表示类别数量。

若使用 COCO 数据集, 则 YOLOv3-Tiny 网络的两个检测头分别输出的张量维度为: $13 \times 13 \times 3 \times (4 + 1 + 80)$ 和 $26 \times 26 \times 3 \times (4 + 1 + 80)$ 。两个不同尺度的特征构成了特征金字塔结构, 从而实现目标检测。

YOLOv3-Tiny 虽然简化了网络结构并加快了目标检测的速度, 但对小物体的检测能力不强, 并且容易出现漏检的问题。因此针对 YOLOv3-Tiny 在检测锥桶时存在的不足, 本文提出了改进优化 YOLOv3-Tiny 的方法。

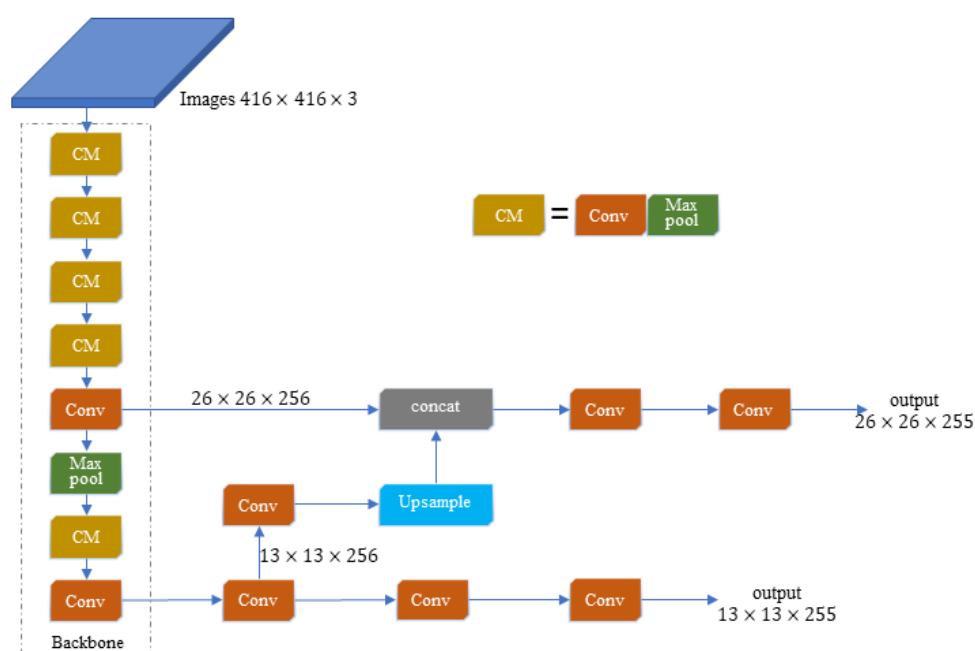


图 2.6 YOLOv3-Tiny 网络结构图

2.2.2 锥桶数据集

本文采用的交通锥桶数据集来源分为 nuScenes 公开数据集^[47]和手动标注两种。

nuScenes 自动驾驶数据集由自动驾驶公司安波福（nuTonomy）在美国波士顿与新加坡各地采集的代表城市环境中复杂驾驶场景的大规模数据集。其数据采集车如图 2.7 所示。



图 2.7 采集车

该数据集由 1000 个城市道路中的场景组成，在 20 秒的场景中包含了丰富的道路交通情景。通过对关键帧进行边界框的手工标注，对其标注了物体的大小、范围、类别以及可见程度等，建立了该数据集。数据集共包括 23 个类，共 140 万张图像，边界框标注文件为 Json 文件，如图 2.8 所示。

Category	nuScenes cuboids	Cuboid ratio	Lidarseg points	Point ratio
animal	787	0.07%	5,385	0.01%
human.pedestrian.adult	208,240	17.86%	2,156,470	2.73%
human.pedestrian.child	2,066	0.18%	9,655	0.01%
human.pedestrian.construction_worker	9,161	0.79%	139,443	0.18%
human.pedestrian.personal_mobility	395	0.03%	8,723	0.01%
human.pedestrian.police_officer	727	0.06%	9,159	0.01%
human.pedestrian.stroller	1,072	0.09%	8,809	0.01%
human.pedestrian.wheelchair	503	0.04%	12,168	0.02%
movable_object.barrier	152,087	13.04%	9,305,106	11.79%
movable_object.debris	3,016	0.26%	66,861	0.08%
movable_object.pushable_pullable	24,605	2.11%	718,641	0.91%
movable_object.trafficcone	97,959	8.40%	736,239	0.93%
static_object.bicycle_rack *	2,713	0.23%	163,126	0.21%
vehicle.bicycle	11,859	1.02%	141,351	0.18%
vehicle.bus.bendy	1,820	0.16%	357,463	0.45%
vehicle.bus.rigid	14,501	1.24%	4,247,297	5.38%
vehicle.car	493,322	42.30%	38,104,219	48.27%
vehicle.construction	14,671	1.26%	1,514,414	1.92%
vehicle.emergency.ambulance	49	0.00%	2,218	0.00%
vehicle.emergency.police	638	0.05%	59,590	0.08%
vehicle.motorcycle	12,617	1.08%	427,391	0.54%
vehicle.trailer	24,860	2.13%	4,907,511	6.22%
vehicle.truck	88,519	7.59%	15,841,384	20.07%
Total	1,166,187	100.00%	78,942,623	100.00%

attribute

JSON 文件

calibrated_sensor

JSON 文件

category

JSON 文件

ego_pose

JSON 文件

filename

JSON 文件

log

JSON 文件

object_ann

JSON 文件

sample

JSON 文件

sample_data

JSON 文件

sample_token_data

JSON 文件

sensor

JSON 文件

surface_ann

JSON 文件

图 2.8 数据集类别占比以及标注文件

但由于本文只针对锥桶进行检测，故通过编写 Python 脚本对 nuScenes 数据集进行过滤，只保留锥桶的图片与其标注信息，得到 22041 张图片，71784 个标注框，并将 Json 格式的标注文件通过 Python 脚本转换为 YOLO 标签格式 txt 文件。数据筛选后的部分数据集及 Python 脚本文件如图 2.9 所示。

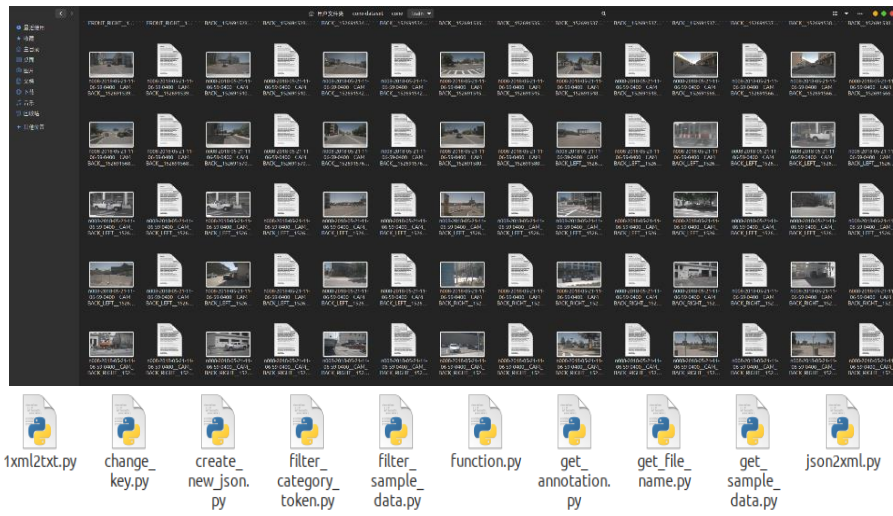


图 2.9 数据过滤后的部分数据集以及 python 脚本文件

手动标注数据集的图片通过网络爬虫与实际采集两种方式获得，然后通过 labellImg 软件对图片进行标注，如图 2.10 所示，共得到 1300 张图片与 3947 个边界框。

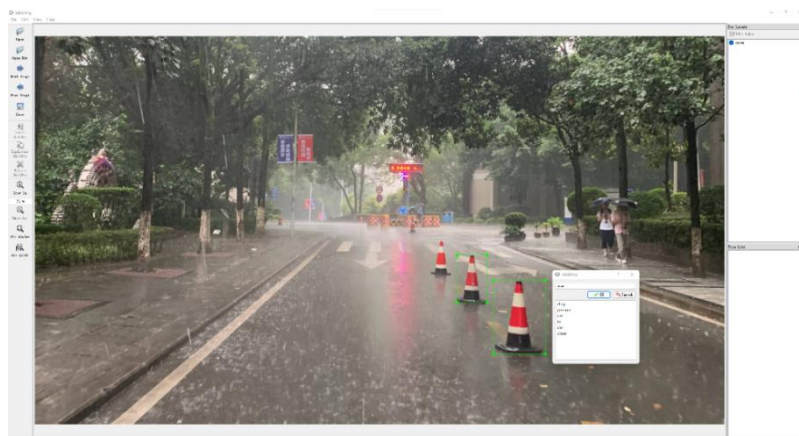


图 2.10 LabelImg 软件标注界面

整合过滤后的 nuScenes 数据集与手动标注的数据集，得到本文训练与测试的锥桶数据集，共 23341 张图片，75731 个边界框，其中图像大小为 1600×900 。训练集与测试集的比例为 7:3。

2.2.3 YOLOv3-Tiny 模型改进方法

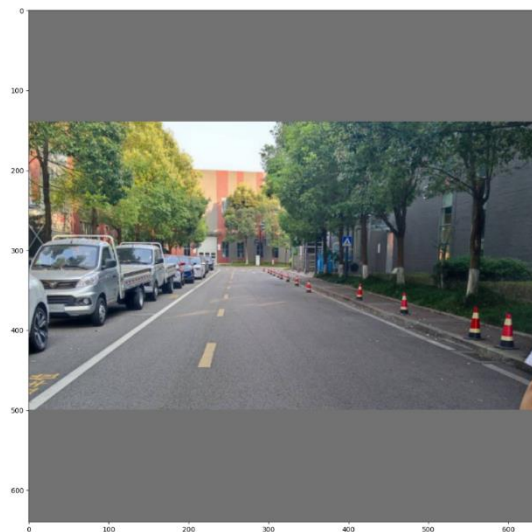
通过分析数据集以及 YOLOv3-Tiny 网络结构，提出了以下四种锥桶检测模型的改进优化方法。

(1) 增加网络输入图片尺寸

YOLOv3-Tiny 的网络输入尺寸为 416×416 个像素，当数据集尺寸与网络输入要求尺寸不一致时，为了使图片尺寸一致且不发生拉伸失真，会对图片进行 `resize` 与 `letterbox` 操作，如图 2.11 所示。而数据集的图片尺寸为 1600×900 个像素，因此会对输入的图片进行上述操作，而锥桶在道路环境中相比于车辆等物体，属于小目标，在图片中所占像素点很少。因此在对图片进行上述操作后，会丢失大量的特征，从而使 YOLOv3-Tiny 网络无法提取到足够的交通锥桶特征。



(a) 原图



(b) 经过 resize 与 letterbox 操作后的图片

图 2.11 数据集原图与经过处理后的图片对比

以通用数据集 COCO^[48]对图像中物体大小的定义为基准，小目标指小于 32×32 个像素，中物体是指 32×32 - 96×96 个像素，大物体是指大于 96×96 个像素。以该基准，分别分析了网络输入尺寸在 416×416 、 640×640 与 800×800 下的标注框的像素点大小分布，如图 2.12 所示。

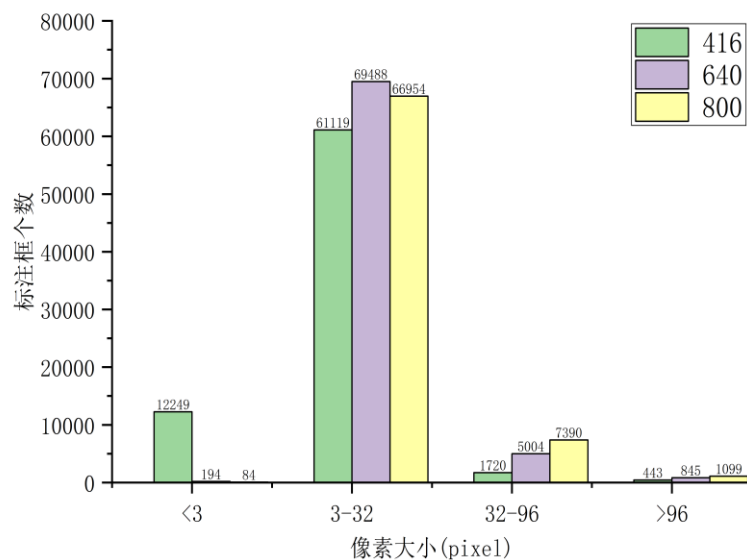


图 2.12 不同输入分辨率下的标注框内的像素点分布

由图可知，在不同输入分辨率下，标注框的 80.9%~91.9%均为小目标物。当网络输入分辨率为 416×416 个像素时，有 12249 个标注框小于 3 个像素，导致网络无法

提取到有效特征，占据总标注框的 16.2%；当网络输入分辨率分别为 640×640 与 800×800 个像素时，小于 3 个像素的标注框分别为 194 与 88 个，占总标注框的比例 0.12%~0.26%，对网络特征提取影响极低，而随着输入分辨率的增加，也会使网络训练速度变慢，进行模型推理时，也会增加计算成本，导致推理速度变慢，因此本文最终确定网络的输入图像分辨率为 640×640 个像素。

当输入尺寸为 416×416 个像素时，步长（stride）为 32，YOLOv3-Tiny 将图片划分为 13×13 个网格。因此当输入分辨率更改为 640×640 个像素时。为保证步长不变，需将图片划分为 20×20 个网格。

（2）损失函数改进

在卷积神经网络的设计过程中，设计损失函数占据着重要环节，因为其作为网络计算预测误差的函数，很大程度上影响着神经网络模型收敛的效果^[49]。因此许多研究者^[50]都在研究如何提升神经网络模型的训练效果、检测效果通过设计更好的损失函数。

YOLOv3-Tiny 算法的损失函数采用误差平方和（SSE）的形式，在分类置信度和类别方面使用了交叉熵（Cross Entropy）^[50, 51]。

由于本文只检测交通锥桶这一类小目标物，因此对损失函数进行了以下改进：

改进 1：由于只有一个检测类别，无须预测类别概率，因此对损失函数进行简化，删除预测类别损失，加速网络收敛。

改进 2：由于网络输入特征图大小为 640×640 个像素，而交通锥桶在图像中所占像素大小一般为 20×10 个像素，如图 2.13 所示。正负样本的比例达到了 1:2048，相差过大，因此引入 Focal loss^[52]来平衡正负样本。



图 2.13 锥桶在图像中所占大小示意图

单阶段目标检测可能由于数据集中正负样本比例不平衡而导致检测效果差，而

为了解决这种问题，一些研究者提出了困难样本挖掘，焦点损失函数（Focal Loss）应运而生。本文中 Focal loss 损失函数是由交叉熵损失函数修改而来^[53]，其表达式如式(2.6)所示：

$$FL(p, y) = -y(1-p)^{\gamma} \lg p - (1-y)p^{\gamma} \lg (1-p) \quad (2.6)$$

式中， p ：表示激活函数 Sigmoid 的结果值；

$\gamma \geq 0$ ：表示聚焦参数；

$(1-p)^{\gamma}$ ：表示调节因子；

Y ：表示标签值。

则改进后的 YOLOv3-Tiny 算法损失函数如下：

$$\begin{aligned} \text{Loss} = & \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B I_{ij}^{\text{obj}} (2 - \widehat{w}_i^j \times \widehat{h}_i^j) [FL(t_{x_i^j}, \widehat{t}_{x_i^j}) + FL(t_{y_i^j}, \widehat{t}_{y_i^j}) \\ & + FL(t_{w_i^j}, \widehat{t}_{w_i^j}) + FL(t_{h_i^j}, \widehat{t}_{h_i^j})] + \sum_{i=0}^{S^2} \sum_{j=0}^B I_{ij}^{\text{obj}} FL(c_i^j, \widehat{c}_i^j) \\ & + \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^B I_{ij}^{\text{noobj}} FL(c_i^j, \widehat{c}_i^j) \end{aligned} \quad (2.7)$$

式(2.7)中， λ_{coord} 、 λ_{noobj} ：坐标损失权重和不含 object 的置信度损失权重； S ：网格划分的横纵向个数； B ：该网格具有的 Bounding Box 个数； I_{ij}^{obj} 、 I_{ij}^{noobj} ：第 i 个网格第 j 个 Bounding Box 是否负责某个对象的检测； $(t_{x_i^j}, t_{y_i^j}, t_{w_i^j}, t_{h_i^j}, c_i^j)$ ：网络输出相对于 Anchor Box 的坐标、宽高偏移量及预测框置信度，与之相对的则表示真实目标框坐标、宽高、置信度和类别概率预测。

（3）更改 YOLOv3-Tiny 的主干网络

主干网络的作用为提取目标物的特征，在很大程度上影响目标检测的效果，众多学者针对在不显著增加计算量的前提下，对 YOLOv3-Tiny 主干网络的优化展开了研究^[54-56]。而 YOLOv3-Tiny 的主干网络为简单的卷积层与最大池化层的堆叠，其中最大池化层在不断的下采样时，会丢失大量的图像特征信息，如图 2.14 所示。不利于网络提取足够的有效特征检测小目标物锥桶。

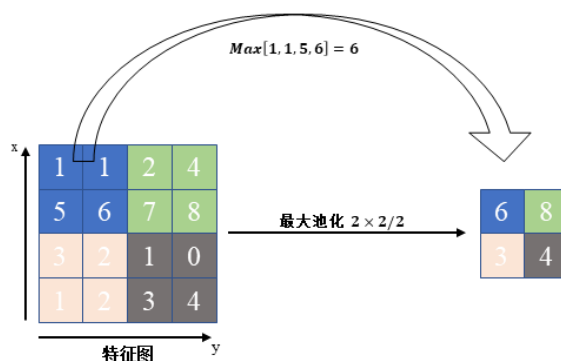


图 2.14 最大池化层示意图

据此对 YOLOv3-Tiny 主干网络做出以下改进：

改进 1：在原有第一个卷积层后，新增加一个卷积核为 3×3 ，步长为 2 的卷积层。并且将原有卷积层的步长修改为 2，从而起到网络下采样的作用^[57]。

改进 2：替换原有主干网络中的最大池化层为 Ghost-Bottleneck^[58]，其网络结构如图 2.15 所示。主干通路由两个 Ghost Module (GM) 串联组成，其中第一个 GM 扩大通道数，第二个 GM 将通道数降低到与输入通道数一致；再使用跳跃通路 (shortcut) 对输入与输出进行 add 操作。Ghost-Bottleneck 通过吸收 MobileNetV2 的优点，对结构中第二个 GM 放弃使用 ReLU 激活函数，而其他层依然使用 ReLU 激活函数和批量归一化 (BN: Batch Normalization)。通过这种结构，可以在加深网络、增强主干网络特征提取能力的同时，保证参数量基本不变。

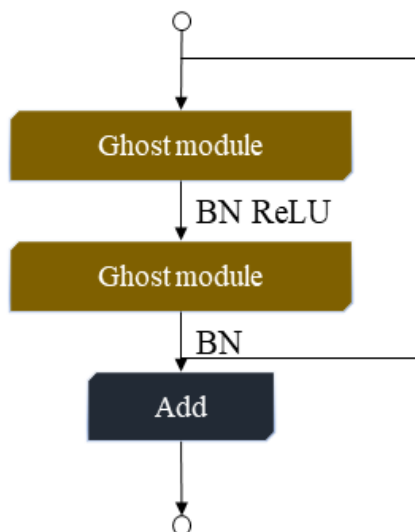


图 2.15 Ghost-Bottleneck 模块结构

图中的 Ghost Module 结构如图 2.16 所示。Ghost Module 分成三步：首先基于卷积得到输入的本征特征图（intrinsic feature maps），然后对本征特征图中每一个通道的特征图进行线性变换得到 Ghost 特征图，最后将本征特征图与 Ghost 特征图 concat 起来，得到输出。通过这种结构，可以在减小参数数量的同时，保证精度。

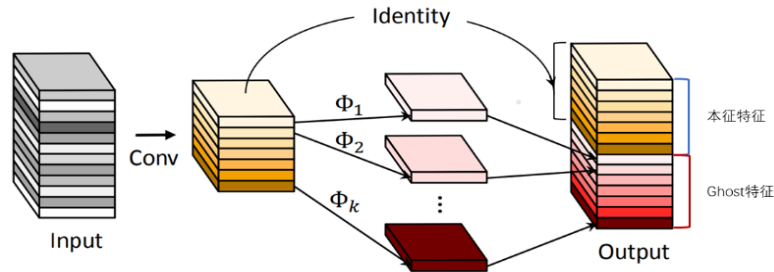


图 2.16 Ghost Conv 模块示意图

则改进后的主干网络如表 2.2 所示：

表 2.2 改进后的主干网络

Type	Filter	Size	Output
Conv	16	3×3/1	416×416×16
Conv	32	3×3/2	208×208×32
Ghost-Bottleneck	32		208×208×32
Conv	64	3×3/2	104×104×64
Ghost-Bottleneck	64		104×104×64
Conv	128	3×3/2	52×52×128
Ghost-Bottleneck	128		52×52×128
Conv	256	3×3/2	26×26×256
Ghost-Bottleneck	256		26×26×256
Conv	512	3×3/2	13×13×512
Ghost-Bottleneck	512		13×13×512
Conv	1024	3×3/1	13×13×1024

（4）融合浅层特征，构建全新的特征金子塔结构

特征金字塔（FPN: Feature Pyramid Networks）。FPN 自上而下地利用语义丰富得网络层的信息构建高分辨率特征层，使得到的特征图同时具有这两个优点。但是由于经过了多次上采样下采样，目标位置已经不准确。所以在上采样得到的网络层和相应的特征层（feature map）之间构建横向连接，以使得检测头可以更好地预测目标的位置。

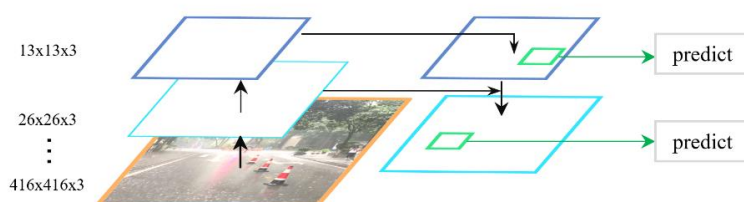
在 YOLOv3-Tiny 网络中, 网络最后输出大小为 13×13 , 26×26 的特征图与上采样得到的对应大小的特征图进行横向拼接, 构成一个特征金字塔, 如图 2.17 所示。由于不同尺度下特征图的感受野 (RF : Receptive Field) 不同, 故不同尺度下特征也不同。感受野是指卷积神经网络每层输出特征图上像素点映射到输出图片上区域大小, 感受野计算方法为:

$$F_l = S_l(F_{l-1} - 1) + K_l \quad (2.8)$$

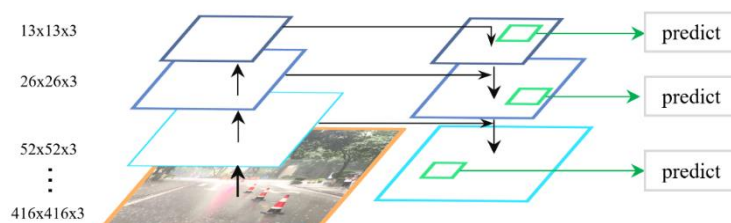
式(2.8)中: F_l 、 S_l 和 K_l 分别为 l 层的感受野、卷积步长与卷积核大小。

由式(2.8)可知: 随着网络的加深, 特征图上的感受野也逐渐增大, 所提取的特征更关注全局信息^[59], 对小目标交通锥桶特征信息丢失严重。因此对 YOLOv3-Tiny 的特征金字塔结构进行改进。

改进方法: 在原有网络的 2 个输出层的基础上, 增加了一层特征图大小为 52×52 的输出层。 52×52 的输出是由上一层尺寸为 26×26 的输出特征图经过上采样后得到尺寸为 52×52 的特征图, 并将其与主干网络对应的尺寸为 52×52 的特征图通过 concat 函数进行拼接得到。改进后的特征金字塔如图所示, 该改进方案可以使网络模型提取到更多网络浅层的特征, 为检测锥桶提供了更多检测依据, 提升检测鲁棒性。



(a) YOLOv3-Tiny 网络的特征金字塔结构



(b) 改进后的特征金字塔结构

图 2.17 改进前后特征金字塔结构对比

综上所述, 本文针对 YOLOv3-Tiny 模型在检测锥桶时存在的不足, 在保持参数量无显著增加的前提下, 提出了四种改进优化方法。改进后的模型命名为 YOLOv3-Tiny-GB3, 其中“GB”代表 Ghost-Bottleneck 网络模块, “3”代表增加特征金字塔

结构为 3 层，则改进后的网络模型结构如图 2.18 所示。

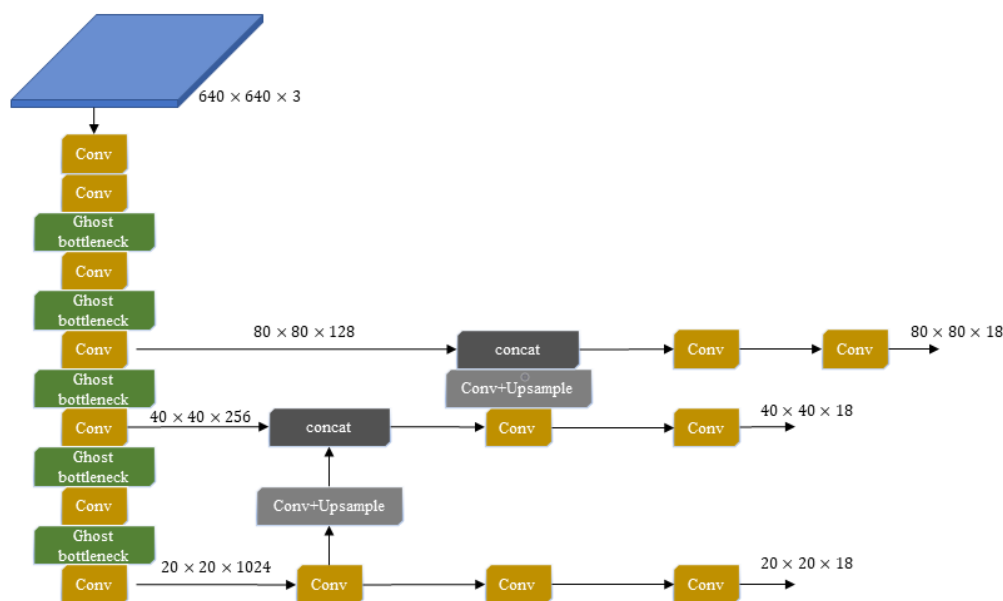


图 2.18 YOLOv3-tiny-GB3 网络结构

2.2.4 评价指标

使用精确率 P 与召回率 R 对模型的检测效果进行评价。使用检测单张图片耗时间来评价算法的检测速度。

P 越大，模型的检测准确度越高，其计算公式如下：

$$Precision = \frac{TP}{TP + FP} \quad (2.9)$$

R 能够准确表达测试集中漏检的数量， R 越大，漏检越少，其计算公式如下：

$$Recall = \frac{TP}{TP + FN} \quad (2.10)$$

式(2.9)、(2.10)中的 TP 、 FP 和 FN ， TP 的含义如表 2.3 所示。

表 2.3 指定类别的混淆矩阵

预测值 真实值	预测值	
	Positive	Negative
Positive	True Positive (TP)	False Negative (FN)
Negative	False Positive (FP)	True Negative (TN)

2.2.5 模型训练

在 Ubuntu 系统下，使用 Pytorch 深度学习框架搭建网络模型，训练硬件环境为

两块英伟达 GTX-2080ti 平台。优化方法采用 Adam (Adaptive Moment Estimation), 模型训练设置迭代 500 轮 (epoch), 批处理大小 (batch-size) 设为 16, 学习率为 0.001, 权重衰减系数为 0.0002, 并在训练过程中, 对训练集的图像进行数据增强, 提升训练效果。

由于 YOLOV3-Tiny 算法初始设置的锚框参数并不适应本文交通锥桶数据集的标注框大小, 并且适宜的锚框也会提升模型的预测结果。故本文使用 K-means 聚类算法提取适宜本文数据集的锚框。

通过 K-means 聚类算法得到适应本文交通锥桶数据集的先验框 (anchor) 如表 2.4 所示。

表 2.4 不同输入尺寸、尺度下的先验框

输入尺寸	Anchor (先验框)	备注
416×416	(30,61)(62,45)(59,119)、(10,13)(16,30)(33,23)	2 个尺度
640×640	(7,22)(9,29)(10,42)、(12,33)(13,46)(15,58)、 (20,70)(27,104)(59,209)	3 个尺度

9 个先验框按宽度从小到大排序分别为: (7,22)、(9,29)、(10,42)、(12,33)、(13,46)、(15,58)、(20,70)、(27,104)、(59,209)。

2.2.6 实验结果分析

(1) 网络参数量对比分析

分别对原始 YOLOv3-Tiny 算法, 主干网络引入 Ghost-Bottleneck 模块的 YOLOv3-Tiny-GB 算法以及融合浅层特征之后的 YOLOv3-Tiny-GB3 算法进行了参数量的对比, 如表 2.5 所示。可以看出在替换主干网络中的最大池化层为 Ghost-Bottleneck 后, 模型整体的参数量基本保持不变。而在融合浅层特征后, 模型整体的参数量相较于原始模型只增加了 $0.86/10^6$, 占比 9.7%, 无显著增加, 参数量基本保持不变。

表 2.5 网络参数量

模型	参数量/ 10^6	备注
YOLOv3-Tiny	8.85	
YOLOv3-Tiny-GB	8.86	改进主干网络
YOLOv3-Tiny-GB3	9.21	融合浅层特征

(2) 检测效果对比分析

分别对原始 YOLOv3-Tiny 模型, 更改输入分辨率 640×640 的 YOLOv3-Tiny 模型, 更改损失函数为 Focal loss 以及删除类别损失后的 YOLOv3-Tiny 模型, 主干网络引入 Ghost-Bottleneck 模块的 YOLOv3-Tiny-GB 算法以及融合浅层特征之后的 YOLOv3-Tiny-GB3 算法进行对比试验训练。结果如表 2.6 所示。

表 2.6 YOLOv3-Tiny 算法与改进算法模型训练结果

模型	P/%	R/%	检测耗时	输入尺寸	备注
YOLOv3-Tiny	77.7	49.6	3ms	416×416	
YOLOv3-Tiny	78.7	53.1	3ms	416×416	改进损失函数
YOLOv3-Tiny	83.9	59.6	5ms	640×640	
YOLOv3-Tiny-GB	85.8	66.9	5ms	640×640	改进主干网络
YOLOv3-Tiny-GB3	86.4	75.3	6ms	640×640	融合浅层特征

由上表可知: 在改进损失函数后, YOLOv3-Tiny 算法在锥桶数据集上目标检测的 P 值与 R 值分别提升了 1 与 3.5 个百分点。在提高输入分辨率到 640×640 后, P 值与 R 值分别较初始模型提升了 6.2 与 14.8 个百分点。因为提高输入尺寸, 图像中包含的信息更多, 更利于检测锥桶这种小目标。在替换主干网络中的最大池化层为 Ghost-Bottleneck 后, P 值与 R 值分别较初始模型提升了 8.1 与 17.3 个百分点, 而检测耗时几乎没有增加。在融合浅层特征之后, P 值与 R 值分别较初始模型提升了 8.7 与 25.7 个百分点, 使得算法对小目标的漏检情况得到了改善。综合来看, 改进后的算法 YOLOv3-Tiny-GB3 相比于 YOLOv3-Tiny 在参数量增加不多的前提下, 提升了模型的检测能力, 并且检测耗时也能满足实车的要求。检测效果对比如图 2.19 所示。

由图 2.19 可以看出: 在三组对照组中, 由于交通锥桶在真实交通环境中的摆放方法以及车载视角的前提下, 图像中存在距离远以及被严重遮挡的交通锥桶, 因此原始 YOLOv3-Tiny 算法无法有效精准的检测出图像中的交通锥桶, 而经过本文改进的算法 YOLOv3-Tiny-GB3 在这种情况下可以精准有效的检测出图像中远距离处以及被遮挡的交通锥桶; 由于交通锥桶的特征较简单, 因此在对照组 A 中本文算法出现了一个误检, 需要后续优化; 在对照组 B 中, 由于输入图片分辨率低, 只有 300×170 个像素, YOLOv3-Tiny 算法出现了严重的漏检, 而本文改进的算法精确的识别到了更多的锥桶。由此可以得出, 经过本文改进方法改进后的 YOLOv3-Tiny-GB3 算法可

以胜任检测远距离交通锥桶以及被遮挡的交通锥桶。

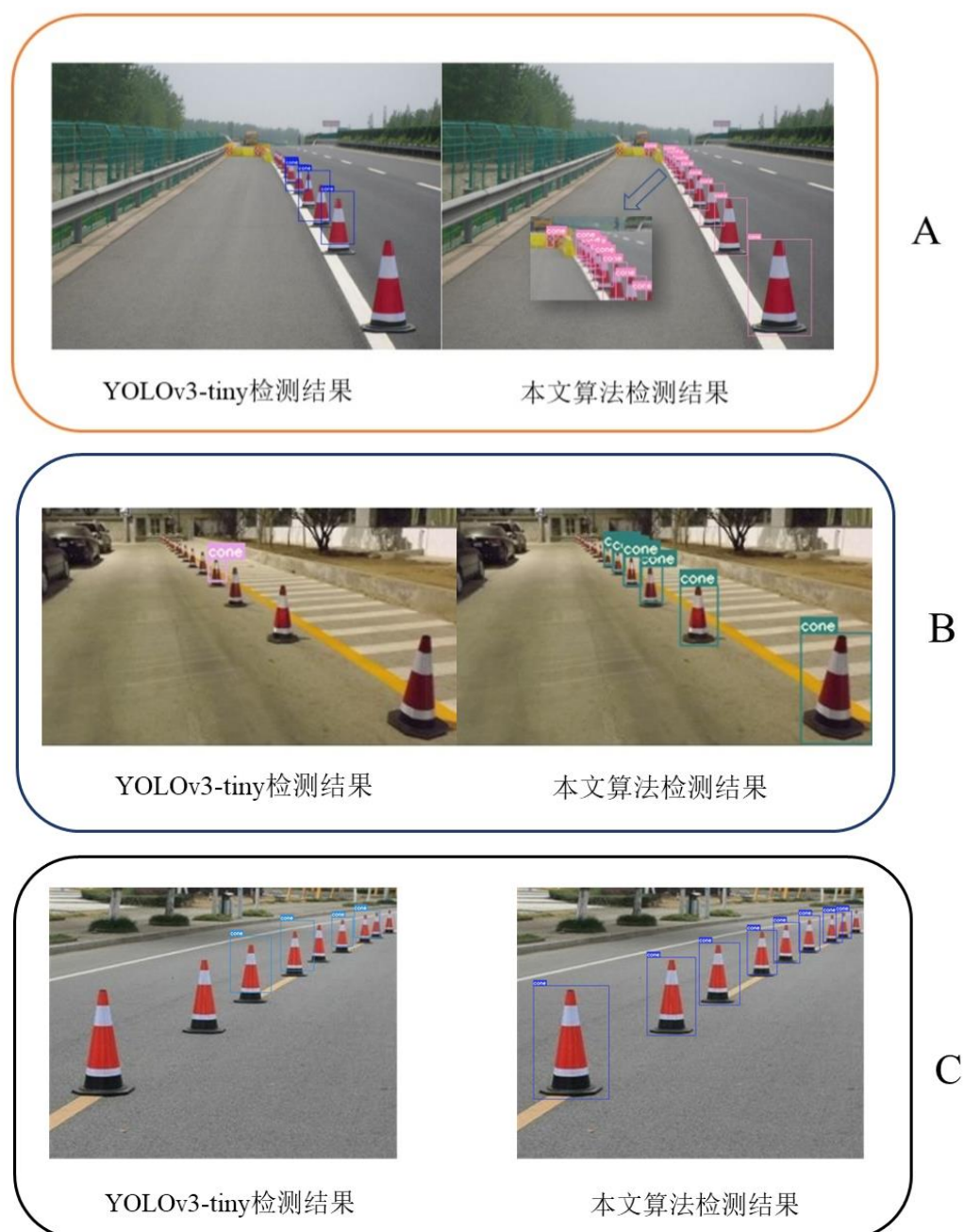


图 2.19 实际交通环境下锥桶的检测效果对比

2.3 本章小结

本章简单介绍了卷积神经网络的基本概念，然后目标检测领域常用的算法进行了对比分析。基于这些算法的特点及优势，本文选用了 YOLOv3-Tiny 算法进行交通锥桶检测，并针对该算法在检测交通锥桶时存在的问题，对其进行了针对性的优化。

然后以 nuScenes 数据集为基础，制作了本文所使用的交通锥桶数据集，并根据数据集的特点，对 YOLOv3-Tiny 算法进行了优化。

优化方法为：

- （1）增加网络输入尺寸；
- （2）更改损失函数；
- （3）更改 YOLOv3-Tiny 主干网络；
- （4）融合浅层特征，构建新的特征金字塔。

最终对改进后的算法进行了实验验证。改进后的算法相比于原始算法 P 提升了 8.7 个百分点，R 提升了 25.7 个百分点。

3 基于先验信息的交通锥桶定位

基于改进优化后的 YOLOv3-Tiny-GB3 算法的检测结果, 本文使用单目相机进行锥桶在相机坐标系下的定位。根据国标规定以及现实中的实际测量, 锥桶的尺寸为高乘宽=35cm×70cm。因此锥桶在现实世界中的尺寸为已知。该尺寸即为先验信息。据此, 提出了以下三种锥桶在相机坐标系下的定位方法。

3.1 基于 PnP 算法的锥桶定位方法

3.1.1 PnP 算法原理

透视 n 点变换 (Perspective- n -Point: PnP) 是一种重要的相机位姿估计方法, 它的原理可简单描述为: 当已知 n 个世界坐标系下的 3D 点坐标以及它们在图像坐标系下对应的投影点坐标, 便可计算出此时相机的位姿。并且由于已知 3D 点在世界坐标系下坐标, 故 PnP 算法不需要对极约束^[60], 且可以在已知较少 3D-2D 点对的情况下, 获得较好的位姿估计。

本章介绍一下常用 PnP 求解方法 P3P。该方法仅使用三对匹配点, 因此计算量低, 便于计算。P3P 方法利用给定的三个点之间的几何关系进行 PnP 算法的求解, 该方法的输入三对 3D-2D 匹配点坐标, 令 3D 点为 A、B、C, 2D 点为 a、b、c, 代表 3D 点在相机坐标系上的投影, 如图 3.1 所示。

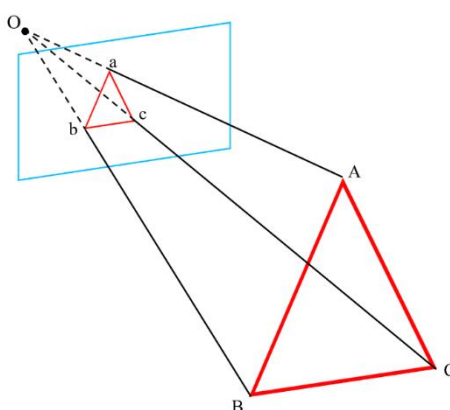


图 3.1 P3P 问题示意图

P3P 求解得到的相机位姿有多个解, 因此需要一对验证点, 来确定正解。记验证点对为 D-d, 相机光心为 O。

由图 3.1 可知, 三角形之间存在对应关系:

$$\Delta Oab - \Delta OAB, \Delta Obc - \Delta OBC, \Delta Oac - \Delta OAC \quad (3.1)$$

考虑到 Oab 与 OAB 的关系, 利用余弦定理, 有:

$$OA^2 + OB^2 - 2OA \cdot OB \cdot \cos(a, b) = AB^2 \quad (3.2)$$

对于其他两个三角形亦有类似性质, 于是有:

$$\begin{aligned} OA^2 + OB^2 - 2OA \cdot OB \cdot \cos(a, b) &= AB^2 \\ OB^2 + OC^2 - 2OB \cdot OC \cdot \cos(b, c) &= BC^2 \\ OA^2 + OC^2 - 2OA \cdot OC \cdot \cos(a, c) &= AC^2 \end{aligned} \quad (3.3)$$

对式(3.3)中的三式全部除以 OC^2 , 并且记 $x = OA/OC, y = OB/OC$, 得:

$$\begin{aligned} x^2 + y^2 - 2xycos(a, b) &= AB^2/OC^2 \\ y^2 + 1^2 - 2ycos(b, c) &= BC^2/OC^2 \\ x^2 + 1^2 - 2xcos(a, c) &= AC^2/OC^2 \end{aligned} \quad (3.4)$$

记 $v = AB^2/OC^2, uv = BC^2/OC^2, wv = AC^2/OC^2$, 有:

$$\begin{aligned} x^2 + y^2 - 2xycos(a, b) - v &= 0 \\ y^2 + 1^2 - 2ycos(b, c) - wv &= 0 \\ x^2 + 1^2 - 2xcos(a, c) - wv &= 0 \end{aligned} \quad (3.5)$$

把式(3.5)中的第一个式子中的 v 放到等式一边, 并代入式(3.5)中的第 2、3 两式, 得:

$$\begin{aligned} (1 - u)y^2 - ux^2 - \cos(b, c)y + 2uxycos(a, b) + 1 &= 0 \\ (1 - w)x^2 - wy^2 - \cos(a, c)x + 2wxycos(a, b) + 1 &= 0 \end{aligned} \quad (3.6)$$

由于 2D 点在图像坐标系中的位置坐标已知, 那么便已知三个余弦角 $\cos(a, b)$ 、 $\cos(b, c)$ 与 $\cos(a, c)$ 。同时, 可以通过 A、B、C 在世界坐标系下的坐标计算得到 $u = BC^2/AB^2, w = AC^2/AB^2$, 比值在变换到相机坐标系下之后并不会改变。因此, 该方程组是关于 x, y 的一个二元二次方程组。计算该方程组的解析解比较复杂, 使用吴消元法可最多得到四个解, 故可以使用验证点对 D-d 来确定正解, 从而得到相机坐标系下 A、B、C 三点的 3D 坐标。然后, 根据 3D-3D 点对, 计算相机的 R、t 矩阵。

P3P 算法原理上首先利用三角形相似性质求得 3D 点在相机坐标系下的投影点的 3D 坐标, 然后将问题转换为 3D-3D 的位姿估计问题。

然而, P3P 也存在着一些问题:

- 1、P3P 仅需 3 个点。无法利用更多的信息以提高精度。
- 2、当点对存在错误匹配或点受到噪声影响时会致使算法失效。

3.1.2 基于锥桶关键点的 PnP 定位方法

在经典的计算机视觉中，主要有三种特征，如图 3.2 所示。信息量最小的是“面特征”，在附近的区域中很难区分。例如，平坦墙壁上的补丁，就是这样一个典型的例子。“线特征”相比于“面特征”更易区分一点，因为它们在特定方向上有一个渐变（穿过边：crossing-over the edge）。然而，如果一个人沿着垂直于这个梯度的方向移动，那么便无法分辨“线特征”，这也被称为孔径问题。到目前为止，最容易区分得特征是“角特征”。它们在两个主要方向上的梯度发生了变化，与附近的区域非常不同，这使得该特征相比于其他两种特征，更容易被找到。

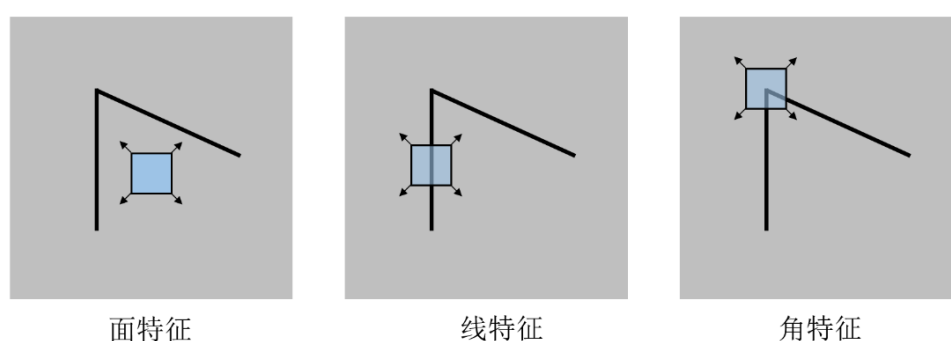


图 3.2 经典计算机视觉三种特征示意图

考虑到这一点，并结合锥桶的几何结构中包含很多“角特征”的特点，本文将检测的关键点位置设置在那些“角特征”处，如图 3.3 所示。在这些地方设置这些关键点有两个主要原因。首先，关键点检测模型定位 6 个非常特定的特征的位置，这些特征在视觉上也是不同的，可以被视为“角特征”，例如不同纹理的合并点和前景与背景界面上的点之间的点。其次，更重要的是，这 6 个点相对容易从固定的世界坐标系 $\mathcal{F}_w$ 中测量。为了方便起见，选择锥桶底部的中心点作为世界坐标系 $\mathcal{F}_w$ 的原点，以便在世界坐标系 $\mathcal{F}_w$ 中轻松测量这 6 个点的 3D 位置。

本文实现了一个卷积神经网络来检测锥桶的关键点，用以从 2D 图像中恢复锥桶的 3D 坐标。锥桶关键点检测模型灵感来自于 Ankit Dhall^[61]，本文将大小为 $120 \times 120 \times 3$ 的图像作为网络的输入。输入图片的大小在选择为 120×120 个像素，因为这是检测到的边界框的平均大小。模型检测的输出向量是 6 个关键点在图像坐标系下的 (x, y) 坐标。

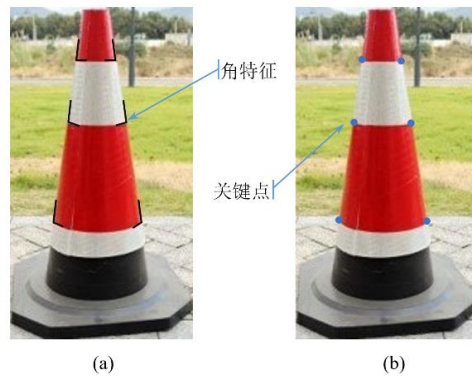


图 3.3 锥桶角特征示意图(a)以及关键点定义图(b)

(1) 关键点检测网络结构

关键点检测模型网络结构有 ResNet^[62]中的基本残差块组成，并使用卷积层来预测每个关键点的坐标，网络结构如图 3.4 所示。

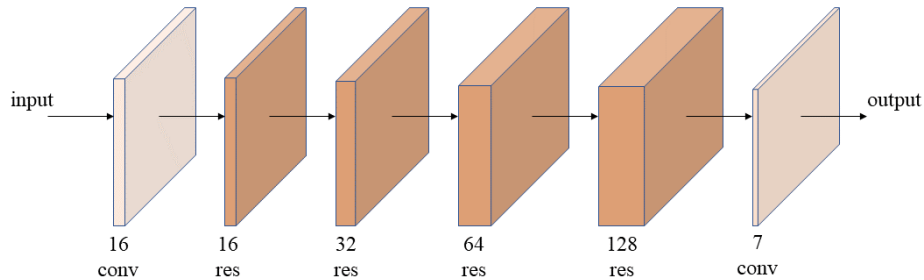


图 3.4 关键点检测网络结构

(2) 关键点数据集

关键点的数据集通过编写 Python 爬虫脚本，从网络上爬取包含锥桶的图片，然后使用之前训练好的锥桶检测模型对图片进行检测，并将检测出的边界框内的图像保存下来，对其进行人工标注。最终得到 380 张数据集，对其进行数据增强，得到 4140 张图片，部分关键点数据集如图 3.5 所示。训练集测试集比例为 7:3。

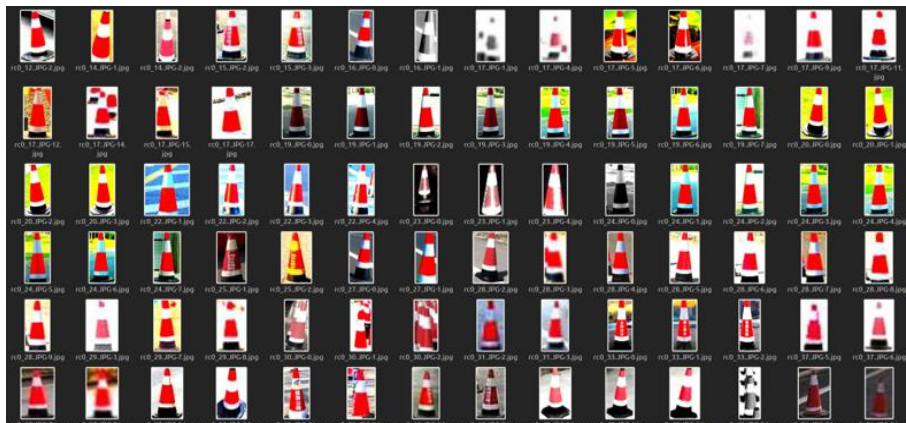


图 3.5 部分关键点数据集

(3) 损失函数

损失函数由两部分构成，分别为关键点损失 Wing Loss^[63]与基于锥桶先验信息的几何损失。损失函数表达式如式(3.7)所示。

$$L_{total} = wing(x) + \gamma_{horz}(2 - V_{12} \cdot V_{34} - V_{34} \cdot V_{56}) + \gamma_{vert}(2 - V_{13} \cdot V_{35} - V_{24} \cdot V_{46}) \quad (3.7)$$

式中 L_{total} 为总的损失函数， $wing(x)$ 为关键点损失 Wing Loss。式(3.7)的后两项即为几何损失，其中 γ_{horz} ， γ_{vert} 分别为横向与纵向两个方向的系数，分别选取为0.055与0.038^[64]，式中其余各项如图3.6所示。

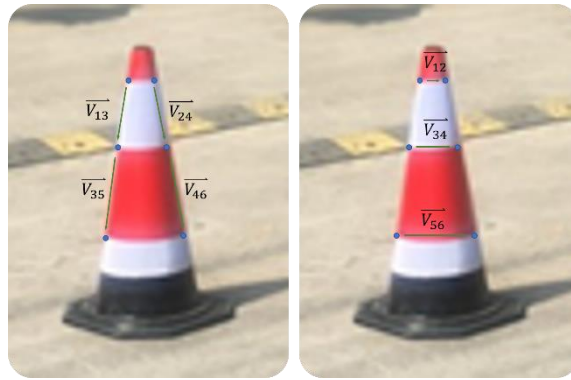


图 3.6 几何向量示意图

(4) 模型训练

在 Ubuntu 系统下，使用 Pytorch 深度学习框架搭建网络模型，硬件环境为英伟达 GTX2080ti 平台。使用随机梯度下降算法（SGD）进行优化，学习率设置为0.0001，动量设置为0.9，batch-size为128，学习率在75和100个epochs后按0.1进行缩放。网络总共训练300个epochs。

(5) 检测效果



图 3.7 关键点检测效果图

训练结果如表 3.1 所示，实际交通环境下的锥桶关键点检测效果如图 3.7 所示。

表 3.1 关键点检测网络训练结果

	Loss
Testing	0.28

（6）定位方法流程

通过关键点检测获得锥桶关键点在图像中坐标后，根据真实世界中锥桶关键点的世界坐标先验信息（世界坐标系原点，选定在锥桶底座中心点），使用 PnP 算法^[65]求得相机的位姿，其中的平移矩阵即为锥桶在相机坐标系下的坐标。定位流程图如图 3.8 所示。



图 3.8 基于关键点检测的 PnP 锥桶定位算法流程

3.1.3 基于锥桶检测框的 PnP 定位方法

锥桶在现实世界中的真实尺寸为已知，故将锥桶抽象为一个圆柱体，那么其圆心横截面矩形的四个顶点的坐标，也可测量得到。那么便可以近似得将目标检测得到的边界框作为圆柱体圆心横截面在图像上的投影，从而获知圆柱体四个顶点在图像坐标系上的投影点的坐标，然后使用 PnP 算法求得相机的位姿，其中的平移矩阵即为锥桶在相机坐标系下的坐标。

2D-3D 的对应匹配示意与相机坐标系与世界坐标系的变换估计如图 3.9 所示。

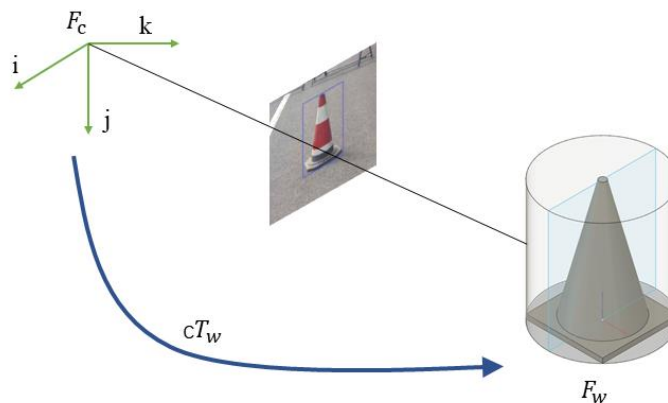


图 3.9 相机坐标系与世界坐标系变换示意图

图 2D-3D 的对应匹配示意图以及相机坐标系与世界坐标系的变换估计算法定位流程如图 3.10 所示。

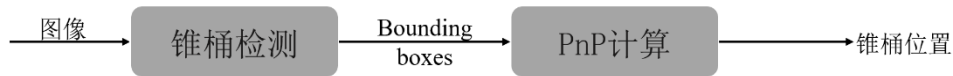


图 3.10 基于锥桶边界框的 PnP 定位方法流程

3.2 基于针孔相机模型的锥桶定位方法

相机成像的过程可以用简单的针孔相机模型来演示，因为成像过程实际上就是将世界坐标系下的三维点映射到成像平面的过程。本节由两部分内容构成，首先介绍针孔成像模型；然后介绍了该方法的定位流程。

3.2.1 针孔相机成像模型

图 3.11 为针孔成像模型，为了使模型得到简化，将成像平面放在了针孔的前面，并且成像也是正立的。

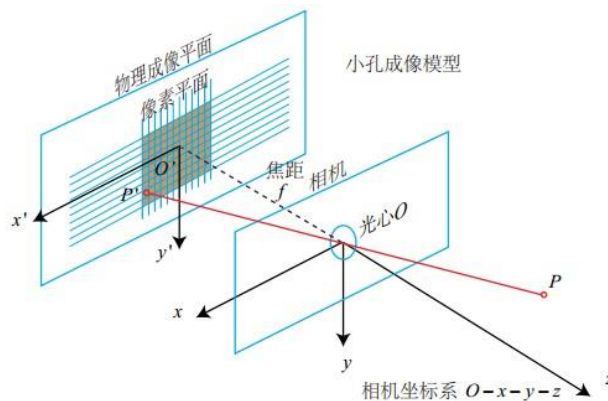


图 3.11 针孔相机成像模型

为了描述针孔的成像过程，需要定义两个坐标系：

- 相机坐标系：相机光心 O_c 为原点， $X_c Y_c Z_c$ 为坐标轴，构成相机坐标系。
- 图像坐标系：成像平面中心 O' 为原点， x', y' 为坐标轴，构成图像坐标系。

假设，三维空间中某一点 P ，在相机坐标系中坐标是 $P_c = [X, Y, Z]^T$ ；其像点 p 在图像坐标系中坐标是 $p = [x, y]^T$ 。依据光轴与成像平面垂直这一相机物理性质，像点 p 在相机坐标系中的坐标是 $p = [x, y, z]^T$ ，其中 $z = f$ （焦距）。由图 3.11 中三角形相似关系：

$$\frac{Z}{f} = \frac{X}{x} = \frac{Y}{y} \quad (3.8)$$

将式(3.8)整理后可得:

$$\begin{cases} x = f \frac{X}{Z} \\ y = f \frac{Y}{Z} \\ z = f \end{cases} \quad (3.9)$$

则式(3.9)则为针孔相机模型的基础公式, 得到该公式便可以推导出针孔模型的参数。

(1) 内参数

相机的内参数由两部分组成:

首先是焦距 f , 相机的光心到成像平面的距离; 然后是图像坐标系到像素坐标系的变换。像素坐标系以像素为基础进行描述, 坐标原点在图像左上角, X 轴沿水平方向向左, Y 轴竖直向下, 如图 3.12 所示。

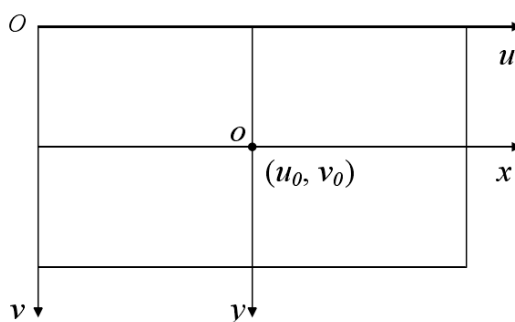


图 3.12 图像像素坐标系与图像物理坐标系

像素为矩形块, 假设其在水平和竖直方向的长度分别为: α 和 β , 故像素坐标和图像坐标之间的变换关系, 为大小的缩放和原点的平移。假设像素坐标的水平轴为 μ , 竖直轴为 ν , 那么将一个图像上的坐标 (x, y) 在水平方向上缩放 α 倍, 在竖直方向上缩放 β 倍, 同时平移 (c_x, c_y) , 就可以得到像素坐标 (μ, ν) , 其公式如下:

$$\begin{aligned} u &= \alpha \cdot x + c_x \\ v &= \beta \cdot y + c_y \end{aligned} \quad (3.10)$$

将上面求得的 (x, y) 代入式(3.10)可得:

$$\begin{cases} u = \alpha \cdot f \frac{X}{Z} + c_x \\ v = \beta \cdot f \frac{Y}{Z} + c_y \end{cases} \Rightarrow \begin{cases} u = f_x \frac{X}{Z} + c_x \\ v = f_y \frac{Y}{Z} + c_y \end{cases} \quad (3.11)$$

式中, $f_x = \alpha \cdot f$, $f_y = \beta \cdot f$ 。

将式(3.11)写成齐次坐标的形式并推导可得:

$$\begin{bmatrix} \mu \\ \nu \\ 1 \end{bmatrix} = Z \begin{bmatrix} \mu \\ \nu \\ 1 \end{bmatrix} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \end{bmatrix} \quad (3.12)$$

则相机的内参矩阵（Camera Intrinsic） K 为：

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \quad (3.13)$$

式中， f_x 、 f_y 和相机焦距、像素大小有关， c_x 、 c_y 是平移的距离，和相机成像平面大小有关。求解相机内参数的过程被称为标定。

（2）外参数

依据上述的推导过程，可知针孔相机成像的过程为：

$$p = KP \quad (3.14)$$

式(3.14)中， p 是像素坐标， K 为相机内参矩阵， P 为相机坐标系下三维点坐标。

由于随着相机的移动，相机坐标系的原点位置与坐标轴方向也会随之发生变化，这导致在该坐标系下推导得到的三维点坐标也会发生变化。故引进一个稳定坐标系：世界坐标系。

设 P 在相机坐标系下坐标为 P_c ，世界坐标系下坐标为 P_w ，便可以使用一个 3×3 的旋转矩阵 R 和一个 3×1 的平移向量 t ，将 P_c 变换为 P_w ，表达式为：

$$\begin{bmatrix} X_c \\ Y_c \\ Z_c \\ 1 \end{bmatrix} = \begin{bmatrix} R & t \\ 0^T & 1 \end{bmatrix} \begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix} \quad (3.15)$$

则相机的外参数（Camera Extrinsic） T 为：

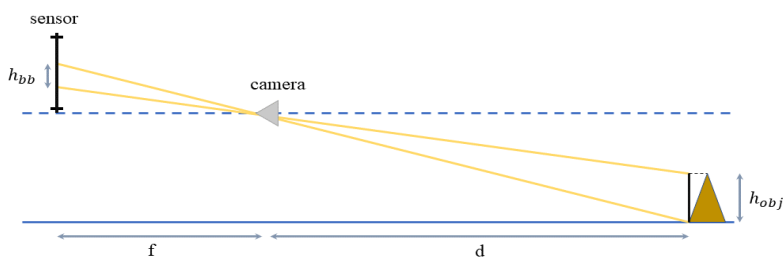
$$T = \begin{bmatrix} R & t \\ 0^T & 1 \end{bmatrix} \quad (3.16)$$

综上，图像中一像点 $p = (\mu, \nu)$ 与真实世界中的三维点坐标 $P_c = (X_c, Y_c, Z_c)$ 之间的转换关系就为：

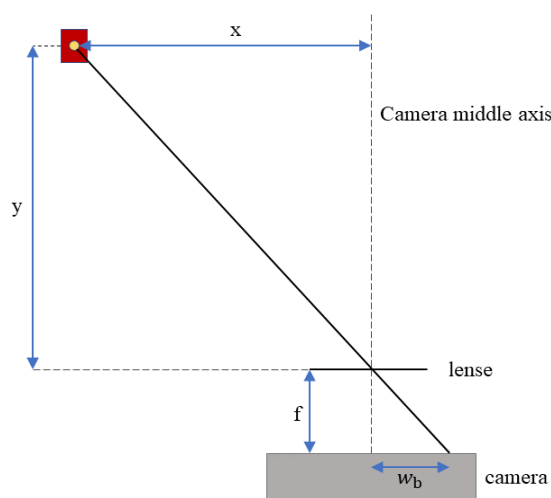
$$\begin{bmatrix} \mu \\ \nu \\ 1 \end{bmatrix} = \begin{bmatrix} f_x & 0 & c_x & 0 \\ 0 & f_y & c_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} R & t \\ 0^T & 1 \end{bmatrix} \begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix} \quad (3.17)$$

3.2.2 定位方法流程

本文在检测到锥桶并得到其边界框后，采用针孔相机成像模型来估算锥桶在相机坐标系下的坐标位置。



a) 侧面视角下的针孔相机模型



b) 鸟瞰视角下的针孔相机模型

图 3.13 针孔相机成像模型估算锥桶坐标示意图

图 3.13 为针孔相机成像模型估算锥桶坐标示意图，根据图中几何关系可得计算锥桶在相机坐标系下的坐标 (x, y) 的公式为：

$$\begin{aligned} d &= f \frac{h_{obj}}{h_{bb}} \\ x &= w_b \frac{f}{y}, \quad y = d \end{aligned} \quad (3.18)$$

式中， x 、 y 分别为锥桶相对于相机坐标系横向与纵向距离， f 为摄像头的焦距， h_{bb} 为边界框的高度， h_{obj} 为锥桶在现实世界中的高度， w_b 为边界框中心到相机光轴的距离。

3.3 锥桶定位实验

基于锥桶关键点的 PNP 定位方法，关键在于锥桶关键点的定位。由于相机所采集的图像距离越远，锥桶所占像素点越少，越不利于检测关键点，因此该定位方法适用于近处的锥桶定位。基于边界框的 PNP 定位方法不受限于采集图像的远近，但由于近处的锥桶与相机光轴不平行，因此存在一定的误差。基于针孔相机模型的定位方法简单，鲁棒性较高。

因此本文分别使用两个单目相机来负责不同距离内的锥桶定位。其中基于锥桶关键点的 PNP 定位方法用于近距离定位，由于关键点检测基于神经网络，存在误检的情况，因此当定位误差超过一定阈值，则使用基于相机针孔模型的定位方法，提高鲁棒性；远距离使用基于边界框的 PNP 的定位方法。

锥桶出现的场景多数为交通施工养护、停车场引导车辆以及标示危险工况。这类场景根据《道路交通安全法》规定，应根据具体的限速标志降低车速，城市道路中一般限速为 30km/h ，故本文设定场景车辆的最大速度为 30km/h ，在该速度下，根据 GB 12676 和 GB/T 13594 规定^[66]，刹车距离应不超过 9m，故设置如图 3.14 所示的相机可视距离。

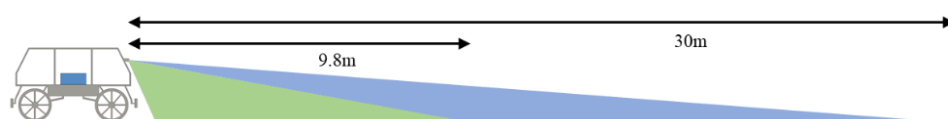


图 3.14 相机可视距离示意图

锥桶定位算法的实验平台如图 3.15 所示。试验平台为线控底盘搭载两颗工业焦距分别为 6mm 与 12mm 的 Leopard Imaging 工业摄像头和 Nuvo-8108GC 工控机，该工控机配备 Ubuntu 18.04 系统、Pytorch 1.7、CUDA 10.1、英特尔酷睿 i9-9900K CPU @3.60GHz×16、GeForce RTX 2070 SUPER GPU 卡和 32G RAM。



图 3.15 实车实验平台

3.3.1 相机标定

使用单目相机对交通锥桶进行定位之前，需要获知相机的内外参数。所以本文使用 MATLAB/Camera Calibrator 工具箱对实验平台所搭载的两颗焦距分别为 6mm 与 12mm 的 Leopard Imaging 工业摄像头进行标定，如图 3.16、3.17 所示。

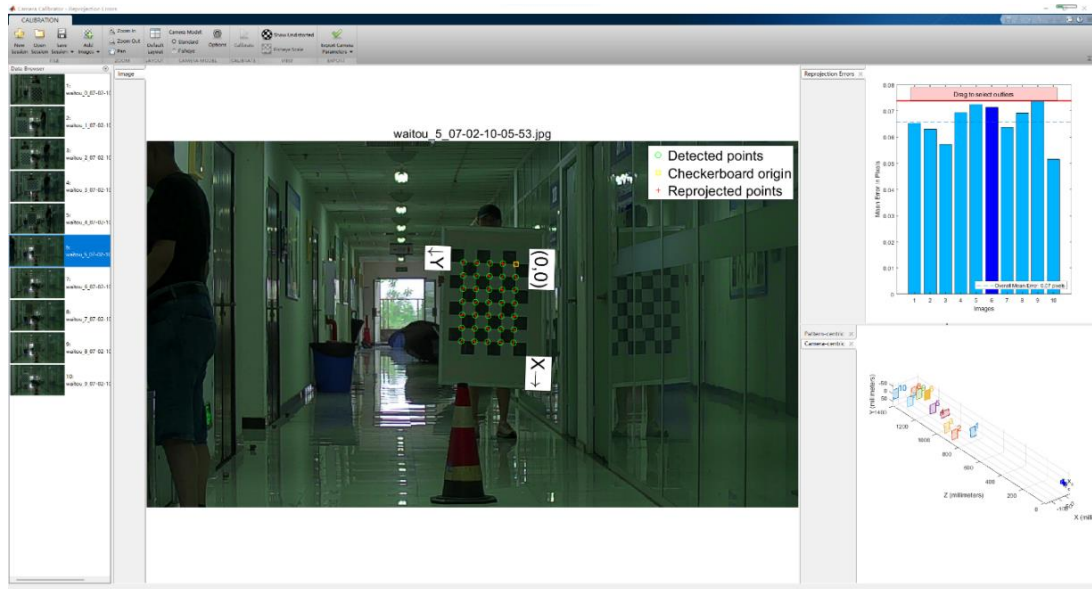


图 3.16 6mm 相机 MATLAB/Camera Calibrator 标定工具运行结果

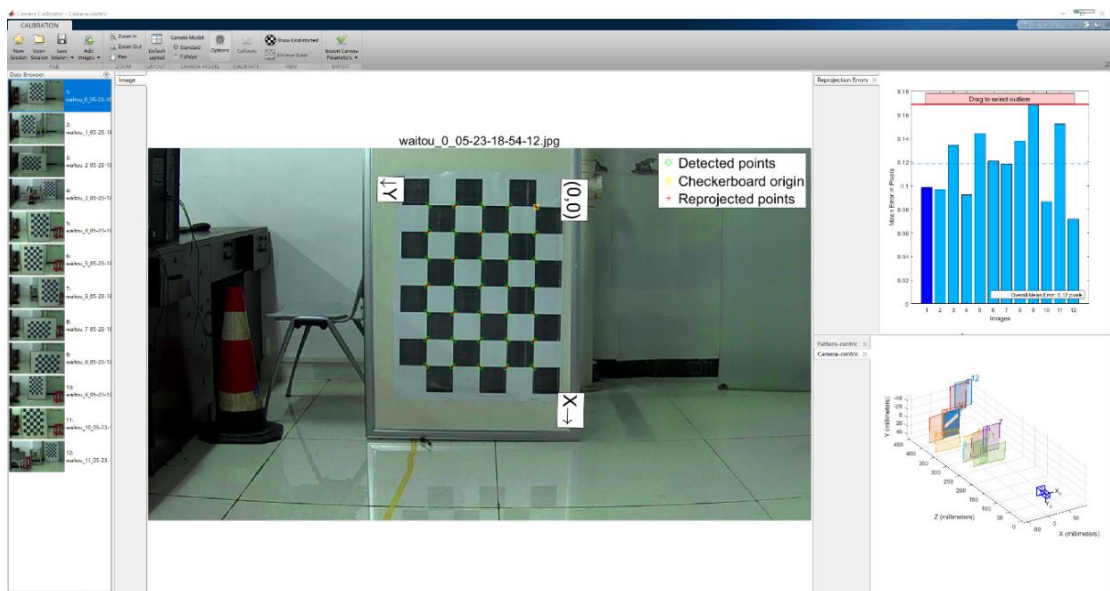


图 3.17 12mm 相机 MATLAB/Camera Calibrator 标定工具运行结果

标定得到的相机参数如表 3.2 所示。

表 3.2 相机参数

相机	Intrinsic matrix	Distortion matrix
6mm	$\begin{bmatrix} 1.9473e+03 & 0.03075 & 641.682 \\ 0 & 1.9481e+03 & 289.0594 \\ 0 & 0 & 1 \end{bmatrix}$	$\begin{bmatrix} -0.5788 & -0.2707 & 0 & 0 \end{bmatrix}$
12mm	$\begin{bmatrix} 3.3078e+03 & 0 & 636.5581 \\ 0 & 3.3096e+03 & 300.5593 \\ 0 & 0 & 1 \end{bmatrix}$	$\begin{bmatrix} -0.6080 & 0.6029 & 0 & 0 \end{bmatrix}$

3.3.2 定位方法试验对比分析

为了评估算法对锥桶纵向和横向距离测量的准确性，分别在纵向 3-30m，横向在相机光轴左右 1m 的位置随机放置锥桶，采集的图像尺寸为 1280×720，试验结果如表 3.3、3.4、3.5 所示。

表 3.3 关键点+PNP 测距试验结果

算法	实际距离(x,y)/m	测量距离(x,y)/m	误差 (%)
关键点+PNP	(-0.4,3.5)	(-0.43,3.6)	(7.5,2.8)
	(0.8,8.7)	(0.14,8.8)	(30,1.14)
	(0.25,11.0)	(0.32,11.3)	(28,2.7)
	平均误差		(21.83,2.21)

表 3.4 边界框+PNP 测距试验结果

算法	实际距离(x,y)/m	测量距离(x,y)/m	误差 (%)
边界框+PNP	(-0.3,11.7)	(-0.275,12.3)	(8.3,5.13)
	(0.4,19.5)	(0.41,19.63)	(2.5,0.67)
	(0.9,29.1)	(0.96,28.44)	(6.6,2.26)
	平均误差		(5.8,2.68)

表 3.5 针孔相机模型测距试验结果

算法	实际距离(x,y)/m	测量距离(x,y)/m	误差 (%)
针孔相机	(-0.35,12.3)	(-0.32,12.8)	(8.5,0.4)
	(0.4,16.5)	(0.50,17.19)	(25,4.18)
	(0.3,27.3)	(0.32,26.05)	(6.6,4.57)
	平均误差		(13.36,3.05)

锥桶检测与定位场景为室内走廊，锥桶的真实距离由皮尺测量得到，图 3.18 为锥桶检测与定位实验结果。

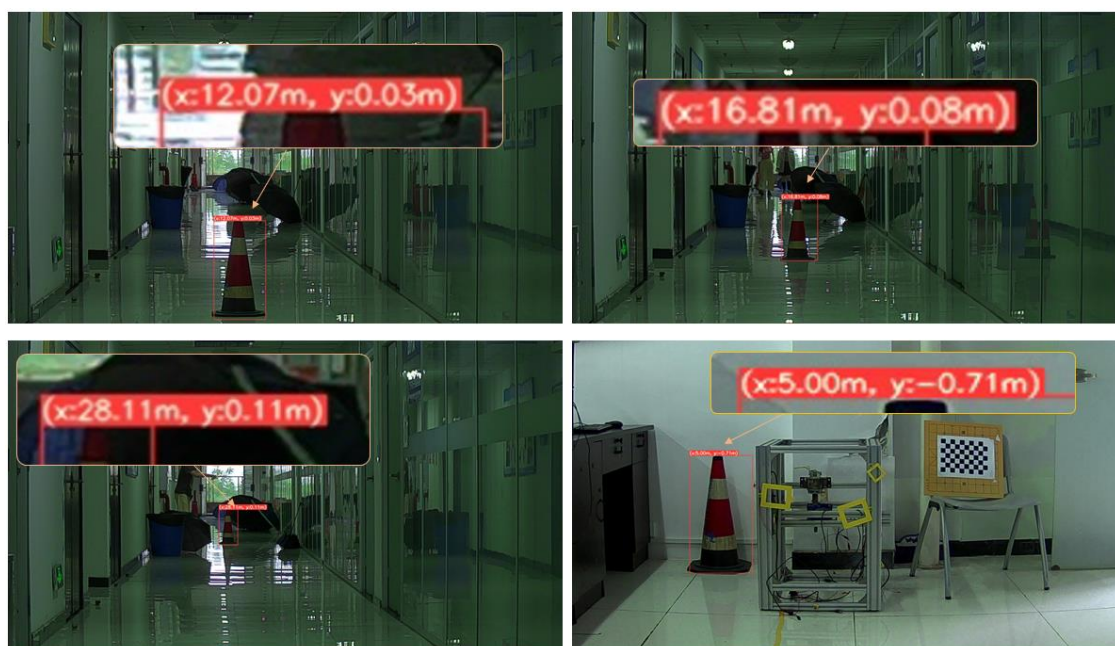


图 3.18 锥桶定位部分实验结果

由以上数据结果可以得知，在通过改进后的 YOLOv3-Tiny-GB3 交通锥桶检测模型得到检测框后，这三种定位方法的纵向误差都控制在 4%以内，而横向误差因为距离近，所以误差略大。因此，通过本文这三种方法可以在一定误差范围内实现锥桶的定位。

3.4 本章小结

本章依据锥桶在现实世界中的尺寸为已知这一先验信息，实现了三种锥桶在相机坐标系下的定位方法。通过分析锥桶的几何特征，设计了锥桶关键点检测网络，并制作了数据集，通过模型训练得到了锥桶关键点检测模型，然后利用 PnP 算法实现了第一种基于锥桶关键点的 PnP 定位方法；通过将锥桶抽象为圆柱体，将其圆心横截面矩形在图像坐标系上的投影近似为锥桶的检测框，利用 PnP 算法实现了第二种基于检测框的 PnP 定位方法；通过分析相机成像模型，利用针孔相机成像模型实现了第三种基于针孔相机成像模型的锥桶定位方法。最后在实车实验平台上，利用两颗工业摄像头进行了定位实验，验证了其定位精度。锥桶定位结果将作为决策规划模块的输入。

4 基于场景决策的路径规划

在真实道路环境中，不同的环境信息对行驶车辆而言，也具有不同的作用，具体可分为引导性的作用与约束性的作用。



图 4.1 环境信息示意图

如图 4.1 所示，约束性环境信息就例如在道路上摆放交通锥桶；而引导性环境信息就例如车道线。对于车道线这种起到引导性的环境信息，在遇到紧急情况时车辆可以跨越；而对于锥桶这类起到约束性的环境信息，在遇到紧急情况时则不能跨越。但是锥桶在道路交通环境中的使用非常的频繁，且使用起来灵活多变，因此针对锥桶环境下的智能车辆决策与规划便显得极为重要。

本文设计了基于场景决策的路径规划算法，通过环境信息决策出当前车辆所处场景工况，然后执行相应的规划算法来规划路径。

4.1 场景工况选取与简化

由于锥桶在道路交通环境中的使用灵活多变，因此锥桶环境下的道路工况便显得非常复杂多变。因此本文经过研究锥桶的使用场景，以及一些真实的锥桶使用案例，提出了本文所研究的四种典型工况，如图 4.2 所示。

(1) 换道工况：该工况大量出现在某一段道路正在进行维修保养，使用锥桶来约束道路引导车辆驶离该车道-换道；

(2) 标示危险区域：该工况大量出现在前方道路塌方、下水井盖丢失以及出现车祸等场景下；

(3) 隔离道路保养施工区域：因为道路施工保养路段一般都是长距离进行，因此，需要大量锥桶来约束道路，隔离车辆与施工作业现场，保证车辆与施工人员安全；

(4) 构成临时车道：该工况经常出现在无车道线的环境下，如公园、场馆以及停车场等地点，因此使用交通锥桶来搭建临时车道，约束车辆的行驶。

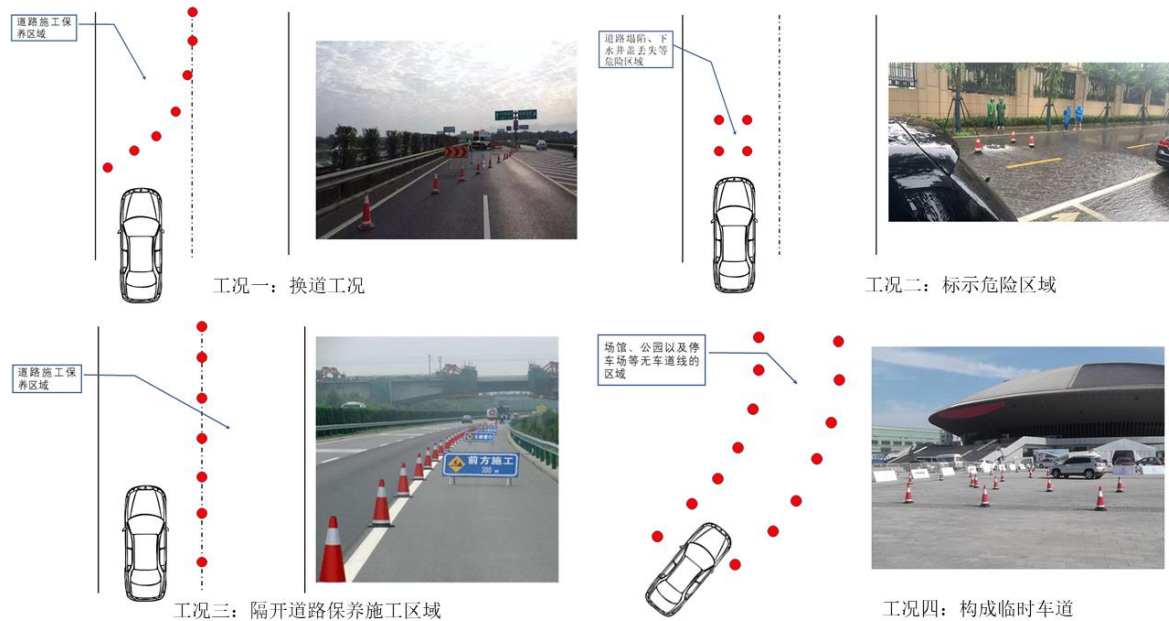


图 4.2 本文选取的四种典型工况与简化后的场景示意图

虽然车辆在行驶过程中，周围的道路交通环境随时可能发生变化，但本文主要研究车辆在锥桶环境约束下的智能车辆决策规划，为了聚焦关键问题，因此对四种典型工况进行了如下简化：

- (1) 道路环境只考虑锥桶与车道线，无其他障碍物；
- (2) 交通参与车辆只考虑自车；
- (3) 自车在行驶过程中，速度为匀速，保持不变。

然后得到本文所研究的简化后的四种场景：

- (1) 纯车道场景：车辆在行驶过程中的默认场景；
- (2) 换道场景：工况一与工况二，当前车道出现锥桶，车辆需要进行换道；
- (3) 混合车道场景：工况三，道路一侧为车道线，另一侧为锥桶构成的临时车道；
- (4) 纯锥桶场景：工况四，完全由锥桶构成临时车道。

4.2 场景决策算法设计

在车辆正常行车过程中，可能随机遇到本文选定的四种典型场景工况，而对于不同的场景工况，车辆的可行驶区域也会受到不同的约束。因此需要对车辆当前所处场景工况进行决策。

根据本文所简化的四种场景的特点，设计了如图 4.3 所示的场景决策算法。算法的输入分别为：

(1) 前两章得到的锥桶在相机坐标系下的坐标，因为相机在车辆上的安装位置可以测量得到，因此可以得到锥桶在车辆坐标系下的坐标；

(2) 车道线，由于目前的车道线检测已经有大量学者展开了研究，且取到了丰富的研究成果，故本文假定车道线已由其他感知模块检测得到。

然后根据输入的锥桶与车道线之间的几何关系，决策当前场景。

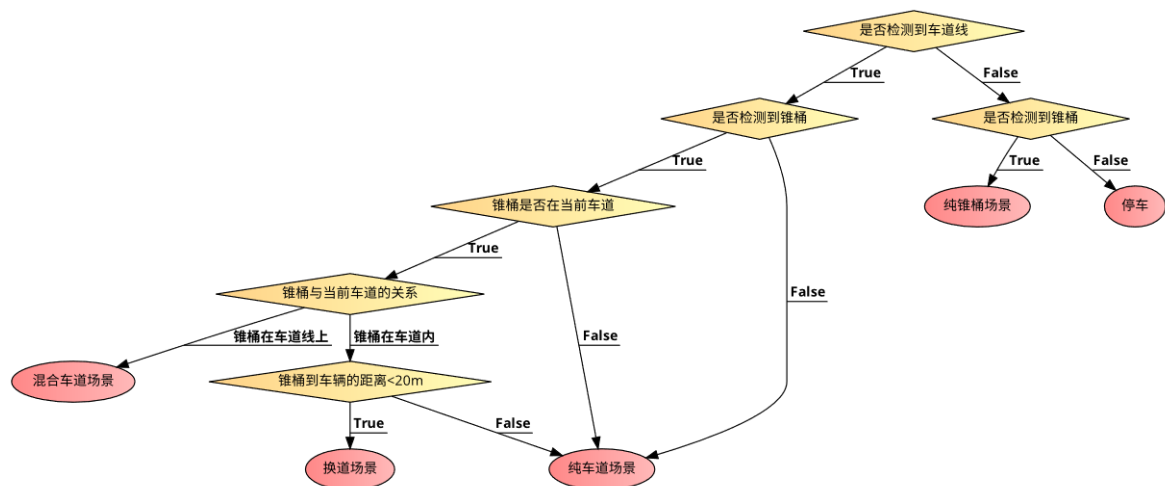


图 4.3 场景决策逻辑图

当车辆启动后，首先判断当前环境内是否检测到车道线，若没有检测到，便判断是否检测到锥桶，若此时检测到锥桶，则当前场景为纯锥桶场景，若没有检测到，则当前场景需要车辆停车；若检测到车道线，便判断是否检测到锥桶，若检测到锥桶，便判断锥桶是否在当前车道，若锥桶在当前车道，便判断锥桶与当前车辆所在车道的车道线的关系，如果在车道内，则当前场景为换道场景，若在车道线上，则为混合车道场景；若没有检测到锥桶或者检测到锥桶，但锥桶不在当前车道，那么车辆当前场景就为纯车道场景。

4.3 不同场景下的路径规划算法

通过上一节得到的场景决策算法，根据车辆周围环境信息，决策出当前车辆所处场景，继而选择相应的规划算法进行车辆的路径规划。不同场景所对应的路径规划方法类别如表 4.1 所示。

表 4.1 不同场景对应的路径规划方法类别

场景	路径规划法类别
换道场景	换道路径规划
纯车道场景	车道保持路径规划
混合车道场景	车道保持路径规划
纯锥桶场景	基于搜索的路径规划

由于道路交通环境复杂多变，存在许多随机不确定因素，且规划得到的路径须满足车辆自身的约束条件。故规划出的路径应满足：

- (1) 规划出的路径应为连续曲线；
- (2) 路径曲线具有连续有界的一阶、二阶导数，且车辆在跟踪路径时状态量不会发生突变；
- (3) 易于在车载端实现且易于车辆执行结构执行。

4.3.1 换道场景下的路径规划

换道场景中的环境信息语义主要体现为当前车道内存在锥桶，车辆无法在该车道继续行驶，因此需要进行换道路径的规划。

下面比较了几种常见的换道路径规划方法。

(1) 在基于正弦函数的换道路径规划方法中，车辆的横向加速度在开始和结束时都不为零，这很难满足乘客的乘坐舒适性；

(2) 基于梯形加速度的换道路径规划方法，其横向加速度变化率为正反梯形，难以调整。

(3) 基于圆弧直线拼接的换道路径规划方法在换道路径拼接处，车辆横向加速度变化率趋于无穷大，导致乘员的乘坐舒适性较差；

(4) 基于多项式的换道路径规划方法，以多项式函数曲线作为换道路径，由于其一阶、二阶导数光滑连续，故乘坐舒适性好^[67]。

综上所述，本文选用基于多项式的换道路径规划。

满足约束的多项式换道路径表达式如式(4.1)所示:

$$y(x) = a_0 + a_1x + a_2x^2 + \cdots + a_{n-1}x^{n-1} + a_nx^n \quad (4.1)$$

使用多项式来进行换道路径规划具有以下两点优势:

- (1) 多项式曲线及其一阶、二阶导数光滑连续, 跟踪过程中不会发生突变;
- (2) 多项式所生成的路径曲线高频分量更小, 系统容易通过反馈系统对其进行补偿控制^[68]。

考虑到多项式换道路径规划的计算成本和精确程度, 本文选用五次多项式进行换道路径规划, 表达式为:

$$y(x) = a_0 + a_1x + a_2x^2 + a_3x^3 + a_4x^4 + a_5x^5 \quad (4.2)$$

其中 x 为换道过程中的纵向位移, $y(x)$ 为换道过程中的横向位移。

将式(4.2)改写为 x 与 y 分别关于时间 t 的函数, 则有:

$$\begin{cases} x(t) = \sum_{i=0}^5 a_i t^i \\ y(t) = \sum_{i=0}^5 b_i t^i \end{cases} \quad (4.3)$$

通过对式(4.3)中的 $x(t)$ 与 $y(t)$ 分别求其一阶导数与二阶导数, 得到了基于五次多项式的换道路径横向速度 $\dot{x}$ 、横向加速度 $\ddot{x}$ 、纵向速度 $\dot{y}$ 、纵向加速度 $\ddot{y}$, 这些变量可写成如下表达式:

$$\begin{cases} x(t_0) = a_0 + a_1t_0 + a_2t_0^2 + a_3t_0^3 + a_4t_0^4 + a_5t_0^5 \\ \dot{x}(t_0) = a_1 + 2a_2t_0 + 3a_3t_0^2 + 4a_4t_0^3 + 5a_5t_0^4 \\ \ddot{x}(t_0) = 2a_2 + 6a_3t_0 + 12a_4t_0^2 + 20a_5t_0^3 \\ x(t_e) = a_0 + a_1t_e + a_2t_e^2 + a_3t_e^3 + a_4t_e^4 + a_5t_e^5 \\ \dot{x}(t_e) = a_1 + 2a_2t_e + 3a_3t_e^2 + 4a_4t_e^3 + 5a_5t_e^4 \\ \ddot{x}(t_e) = 2a_2 + 6a_3t_e + 12a_4t_e^2 + 20a_5t_e^3 \end{cases} \quad (4.4)$$

$$\begin{cases} y(t_0) = b_0 + b_1t_0 + b_2t_0^2 + b_3t_0^3 + b_4t_0^4 + b_5t_0^5 \\ \dot{y}(t_0) = b_1 + 2b_2t_0 + 3b_3t_0^2 + 4b_4t_0^3 + 5b_5t_0^4 \\ \ddot{y}(t_0) = 2b_2 + 6b_3t_0 + 12b_4t_0^2 + 20b_5t_0^3 \\ y(t_e) = b_0 + b_1t_e + b_2t_e^2 + b_3t_e^3 + b_4t_e^4 + b_5t_e^5 \\ \dot{y}(t_e) = b_1 + 2b_2t_e + 3b_3t_e^2 + 4b_4t_e^3 + 5b_5t_e^4 \\ \ddot{y}(t_e) = 2b_2 + 6b_3t_e + 12b_4t_e^2 + 20b_5t_e^3 \end{cases} \quad (4.5)$$

则式(4.4)、式(4.5)中的系数矩阵为:

$$A = [a_0, a_1, a_2, a_3, a_4, a_5]^T \quad (4.6)$$

$$B = [b_0, b_1, b_2, b_3, b_4, b_5]^T \quad (4.7)$$

$$T = \begin{bmatrix} 1 & t_0 & t_0^2 & t_0^3 & t_0^4 & t_0^5 \\ 0 & 1 & t_0 & t_0^2 & t_0^3 & t_0^4 \\ 0 & 0 & 2 & t_0 & t_0^2 & t_0^3 \\ 1 & t_e & t_e^2 & t_e^3 & t_e^4 & t_e^5 \\ 0 & 1 & t_e & t_e^2 & t_e^3 & t_e^4 \\ 0 & 0 & 2 & t_e & t_e^2 & t_e^3 \end{bmatrix} \quad (4.8)$$

在换道过程中，车辆的纵向、横向状态量分别为：

$$\begin{aligned} q_x &= [x(t_0), \dot{x}(t_0), \ddot{x}(t_0), x(t_e), \dot{x}(t_e), \ddot{x}(t_e)] \\ q_y &= [y(t_0), \dot{y}(t_0), \ddot{y}(t_0), y(t_e), \dot{y}(t_e), \ddot{y}(t_e)] \end{aligned} \quad (4.9)$$

综上，五次多项式系数矩阵为：

$$\begin{aligned} A &= T^{-1} \cdot q_x \\ B &= T^{-1} \cdot q_y \end{aligned} \quad (4.10)$$

设车辆换道横向位移 $Y=3.75\text{m}$ ，纵向位移 $X=50\text{m}$ ，纵向速度 $v_x = 10\text{m/s}$ ，则车辆的横纵向状态量分别为：

$$q_x = [0, 10, 0, 50, 10, 0], \quad q_y = [0, 0, 0, 3.75, 0, 0]$$

故通过上述推导得到公式可以得到五次多项式换道路径规划的仿真结果如图 4.4 所示。

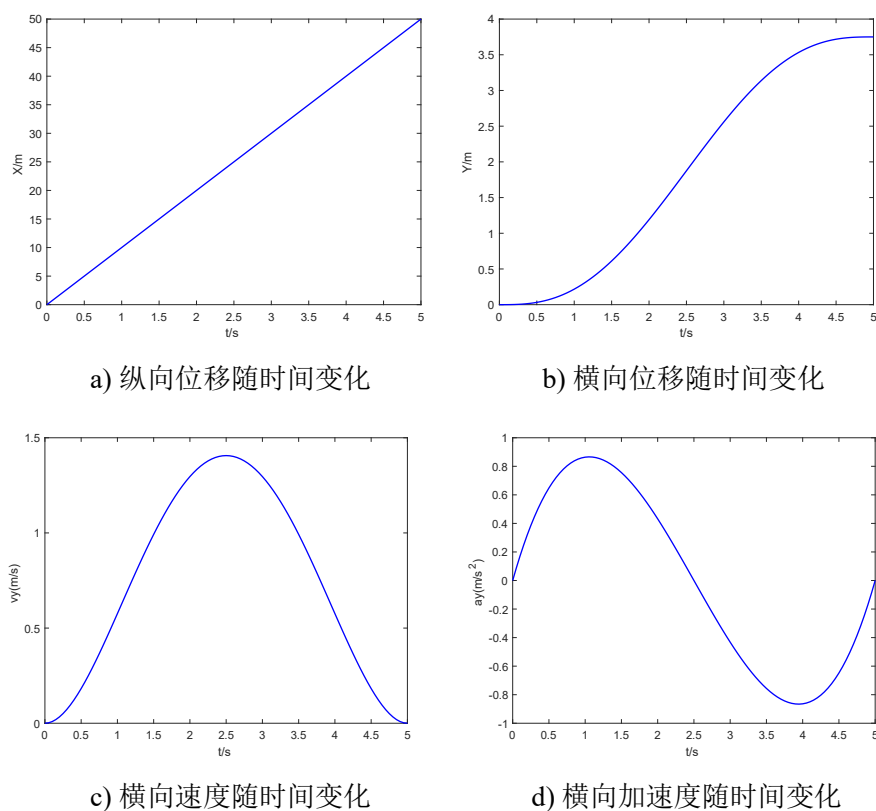


图 4.4 基于五次多项式的换道路径状态量随时间变化

图 3-11 为基于多项式的换道路径状态量随时间变化，换道总时间为 5s，横向速

度最大值为 1.42 m/s ，横向加速度最大值为 0.88 m/s^2 。

4.3.2 纯车道场景下的路径规划

纯车道场景下的环境信息语义体现为当前车道内没有锥桶，因此车辆需按照车道线行驶，即为车道保持。

由于本文假定车道线由其他感知模块检测，因此在车道线为已知。在车道线的位置信息为已知后，需要提取两条车道线的中线作为车辆进行车道保持所跟踪的路径。因此分别对两条车道线以 0.01 m 的采样间隔进行采样，得到两条车道线的采样点，然后求取两条车道线采样点中对应的一对采样点的中点，即为提取出的中线路径。因为采样点足够密集，因此路径无需平滑处理。

4.3.3 混合车道场景下的路径规划

混合车道场景下的环境信息语义体现为当前车道内的锥桶都分布在其中一条车道线上，起到约束车辆的作用，因此此时车辆需按照由车道线与另一侧锥桶构成的临时车道线所构成的车辆可行驶区域行驶的中线行驶，即为车道保持。

由于车道线与锥桶的位置信息已知，因此以锥桶的纵向坐标为采样点，离散车道线，得到车道线的离散点，然后根据锥桶坐标对应的车道线离散点坐标，求出对应这两点之间的中点，然后对中点离散点进行曲线拟合。下面介绍两种常用的曲线拟合方法。

(1) 折线法

即简单的将离散点进行依次连接，得到折线段的集合。该方法计算量小，但并不连续，曲率不连续。

(2) 最小二乘法

最小二乘法是一种数学优化技术是，使用该方法可以很容易地得到未知的数据，并且可以使得到的数据和实际数据之间的误差平方之和最小化。该方法用于曲线拟合，其基本思路为：

$$f(x) = a_1\varphi_1(x) + a_2\varphi_2(x) + \cdots + a_m\varphi_m(x) \quad (4.11)$$

其中， $\varphi_k(x)$ 为事先选定的一组线性无关的函数， a_k 是待定系数（ $k = 1, 2, \cdots, m, m < n$ ），拟合准则是使 $y_i (i = 1, 2, \cdots, n)$ 与 $f(x_i)$ 的距离平方和 δ_i 的平方和最小。

该方法计算复杂，计算量大，而车道在真实交通环境中一般呈直线或者小曲率曲线，因此考虑到计算成本，本文选择折线法进行车道线中线的拟合。由于混合车道场景下，锥桶的位置摆放为人工摆放，因此锥桶的位置可能出现不规律的情况，体现在规划出的路径上面，即此时路径可能存在一些突变，所以需要对其进行路径

平滑。

4.3.4 纯锥桶场景下的路径规划

纯锥桶场景下的环境信息语义体现为当前车辆所处环境中没有车道线来引导车辆行驶，只有锥桶构成的临时车道线。因此，此时车辆需要在锥桶构成的临时车道中搜索出一条可行驶的路径。

由于锥桶的位置信息都已知，故据此提出了本文在纯锥桶场景下基于搜索的路径规划算法。该算法包括四个步骤：

（1）首先对搜索空间进行离散化

因为路径规划的目标是使车辆在在锥桶所构成的临时车道内行驶，那么锥桶自然形成一个离散的空间。因此，本文通过执行 Delaunay 三角剖分（Delaunay^[69], 1934）（Lee 和 Schachter^[70], 1980）来离散 X-Y 空间，该方法将 X-Y 空间细分为连接的三角形。三角剖分的顶点为锥桶在车辆坐标系的定位值，三角形的选择使每个三角形的最小内角最大化。Delaunay 三角剖分是使用 CGAL 库进行的（CGAL 项目^[71], 2018 年）。

（2）提取路径备选点

通过上一步得到的离散空间内的 Delaunay 三角，计算每个 Delaunay 三角形的每个边的中点，存进一个数组中，作为路径的备选点。

（3）计算路径备选点的代价（Cost），搜索路径点

通过上一步得到的路径备选点，首先根据车辆的位置信息，计算备选路径点集合中距离车辆 5m 以内的路径备选点到车辆的距离，其中距离最小的点，为路径的起点。然后，借鉴 A*^[72]路径规划算法中的计算每个路径备选点的代价，选代价最小的路径备选点加入路径的思想，设计了本文路径搜索的算法。

从路径起点开始，搜索下一个路径点，然后在搜索到的路径点为当前路径点，搜索下一个路径点，依次往复，最终搜索得到整条路径。为了避免浪费内存与计算资源，每次都只搜索一定范围内的路径备选点，搜索范围设置为当前路径点之后 5m。

本文设计的路径点的代价，如表 4.2 所示。

表 4.2 搜索路径点的 cost

Cost	reason
最大转向角	车辆的转向角度有限制
路径点之间距离	该场景下锥桶摆放间隔一般为 2m 左右
该段路径与前两段路径的导数的方差	路径不会发生突变

(4) 连接搜索得到的路径点

最终，将搜索出的各个离散路径点连接起来，便为纯锥桶场景下的路径。此时路径曲率不连续，因此对其进行平滑处理。

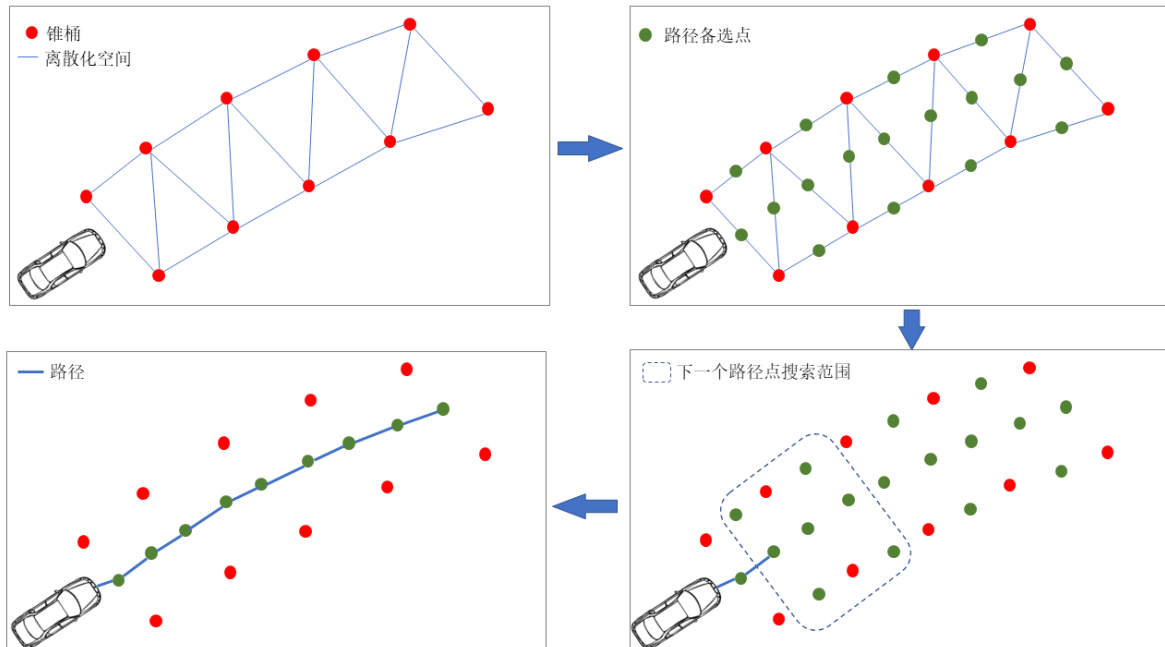


图 4.5 纯锥桶场景基于搜索的路径规划方法示意图

图 4.5 为纯锥桶场景下的路径规划算法流程图，图中按箭头顺序分别对应纯锥桶场景下基于搜索的路径规划算法的四个步骤，其中红色圆点代表鸟瞰视角下锥桶的在车辆坐标系下的位置坐标；绿色圆点代表空间离散化后得到的路径备选点；虚线框代表下一个路径点的搜索范围；深蓝色粗线段代表搜索得到的路径。

4.4 基于贝塞尔曲线的路径平滑处理

由于纯锥桶场景以及混合场景下规划所得到的路径为离散点连接得到的折线，故路径存在曲率不连续的问题，而这会导致车辆在对路径进行跟踪时，出现状态突变，因此需要对路径进行平滑以满足曲率连续。路径平滑算法大多可分为基于多项式的路径平滑算法以及基于贝塞尔曲线的路径平滑算法。其中贝塞尔曲线（Bézier curve）由法国工程师皮埃尔·贝塞尔（Pierre Bézier）^[73]于 1962 年发表提出后，后来经过不断发展，被广泛应用于自动驾驶系统的路径规划领域。由给定点 P_0 、 P_1 、 $\dots$ 、 P_n 的 n 阶贝塞尔曲线可表示为如式(4.12)所示的通式形式^[74]。

$$\begin{aligned}
B(t) &= \sum_{i=0}^n \binom{n}{i} P_i (1-t)^{n-i} t^i \\
&= \binom{n}{0} P_0 (1-t)^n t^0 + \binom{n}{1} P_1 (1-t)^{n-1} t^1 + \cdots t \in [0,1] \\
&\quad + \binom{n}{n-1} P_{n-1} (1-t)^1 t^{n-1} + \binom{n}{n} P_n (1-t)^0 t^n
\end{aligned} \tag{4.12}$$

在二维 XY 平面坐标系中，将纵向位移 x 和横向位移 y 表示成参数 t 的函数，则三阶和五阶贝塞尔曲线可以分别由式(4.13)、式(4.14)表示：

$$\begin{cases} x = x_1(1-t)^3 + 3x_2t(1-t)^2 + 3x_3t^2(1-t) + x_4t^3 \\ y = y_1(1-t)^3 + 3y_2t(1-t)^2 + 3y_3t^2(1-t) + y_4t^3 \end{cases} \tag{4.13}$$

$$\begin{cases} x = x_1(1-t)^5 + 5x_2t(1-t)^4 + 10x_3t^2(1-t)^3 + \\ \quad 10x_4t^3(1-t)^2 + 5x_5t^4(1-t) + x_6t^5 \\ y = y_1(1-t)^5 + 5y_2t(1-t)^4 + 10y_3t^2(1-t)^3 + \\ \quad 10y_4t^3(1-t)^2 + 5y_5t^4(1-t) + y_6t^5 \end{cases} \tag{4.14}$$

式中， (x_i, y_i) ， $i = 1, 2, 3, 4, \dots$ 为选取的各坐标点的坐标信息； y 为路径的横向位移， x 为路径的纵向位移； t 为曲线参数， $t \in [0, 1]$ 。

贝塞尔曲线的曲率可由下式求得：

$$k(t) = \frac{x'y'' - y'x''}{(x'^2 - y'^2)^{\frac{3}{2}}} \tag{4.15}$$

式中， $x' = \frac{dx}{dt}$ ， $x'' = \frac{dx'}{dt}$ ， $y' = \frac{dy}{dt}$ ， $y'' = \frac{dy'}{dt}$ 。

分别对三阶、五阶贝塞尔曲线进行了分析，如下：

三阶贝塞尔曲线所需控制点个数为四个，其坐标分别为(0,0)、(10,0)、(10,10)、(20,10)，得到如图 4.6、4.7 所示的三阶贝塞尔曲线及其曲率。

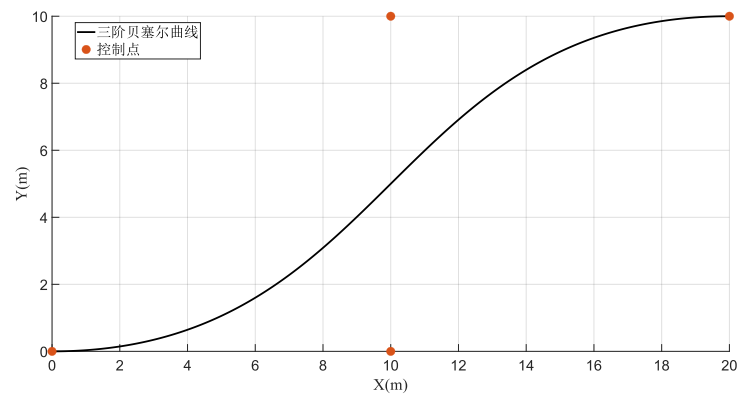


图 4.6 三阶贝塞尔曲线图

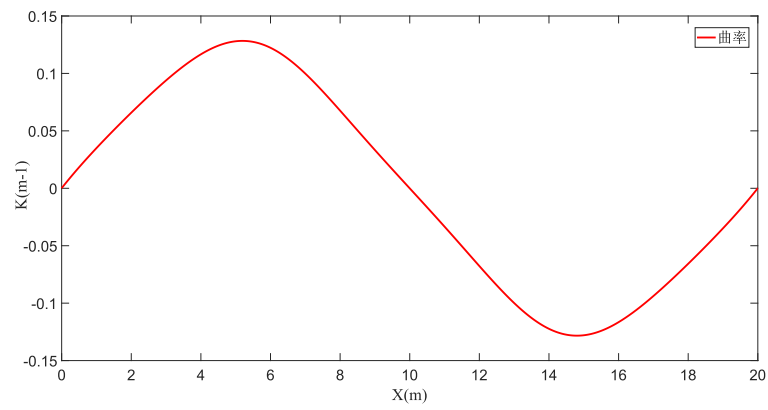


图 4.7 三阶贝塞尔曲线曲率图

五阶贝塞尔曲线所需控制点个数为六个，其坐标分别为(0,0)、(5,0)、(10,0)、(10,10)、(15,10)、(20,10)，得到如图 4.8 所示五阶贝塞尔曲线。

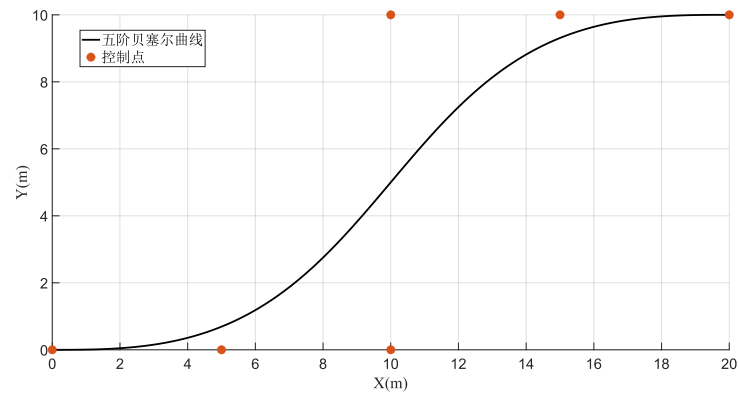


图 4.8 五阶贝塞尔曲线

其曲率如图 4.9 所示。

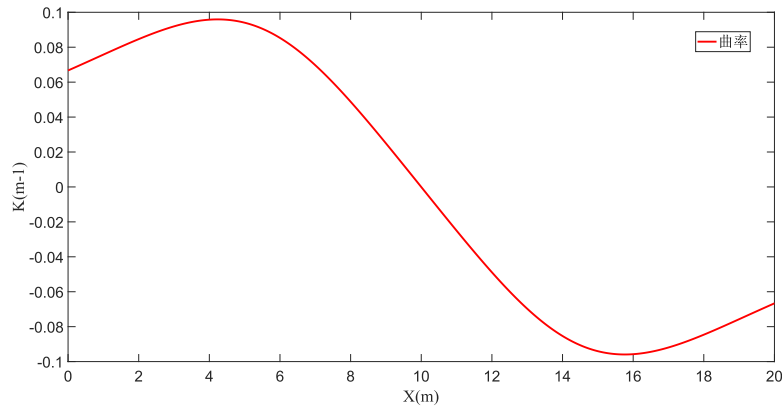


图 4.9 五阶贝塞尔曲线曲率图

由上图中三阶贝塞尔曲线以及五阶贝塞尔曲线的曲率可知，贝塞尔曲线的曲率平滑且连续，适用于本文进行路径平滑处理。

由于在混合车道以及纯锥桶车道场景下，路径规划所得到的离散路径点多于五阶贝塞尔曲线所需的 6 个控制点，而贝塞尔曲线只会通过最后一个和最后一个控制点，且生成的贝塞尔曲线可以通过操作控制点进行改变，这也致使贝塞尔曲线在大量控制点情况下会出现数值不稳定性。

因此，本文提出使用多段贝塞尔曲线进行拼接来实现整条路径的平滑。假定前段贝塞尔曲线的控制点为 P_0 、 P_1 、 $\dots$ 、 P_n ，后段贝塞尔曲线的控制点为 Q_0 、 Q_1 、 $\dots$ 、 Q_n ，相邻两段贝塞尔曲线在拼接处，应该满足连续性约束条件：

- (1) G0 连续：要求前一段曲线的末点 P_n 与后段曲线的起始点 Q_0 重合。
- (2) G1 连续：前后两段贝塞尔曲线在连接点处的一阶导数应该相同，即 $\eta P'(1) = Q'(0)$ ，其中 η 为大于 0 的常数，即 P_{n-1} 、 $P_n(Q_0)$ 以及 Q_1 共线。
- (3) G2 连续：即曲率连续， $k_1(1) = k_2(0)$ 。

然而，由于控制点的限制，难以实现每两段相邻贝塞尔曲线的控制点同时满足 P_{n-1} 、 $P_n(Q_0)$ 以及 Q_1 三点共线、G2 连续。因此，本文在曲线拼接处，均补充一个辅助控制点 Q_1 。

本文根据路径点数量的不同，考虑了五阶贝塞尔曲线拼接五阶贝塞尔曲线以及五阶贝塞尔曲线拼接三阶贝塞尔曲线的两种情况。

4.4.1 五阶贝塞尔曲线拼接五阶贝塞尔曲线

假定前段贝塞尔曲线的控制点为 $P_0(x_0, y_0)$ 、 $P_1(x_1, y_1)$ 、 $\dots$ 、 $P_5(x_5, y_5)$ ，后段贝塞尔曲线的控制点为 $Q_0(x'_0, y'_0)$ 、 $Q_1(x'_1, y'_1)$ 、 $\dots$ 、 $Q_5(x'_5, y'_5)$ 。那么其连续性约束条件则为：

(1) G0 连续: $P_5(x_5, y_5) = Q_0(x'_0, y'_0)$ 。

(2) G1 连续: $\eta P'(1) = Q'(0)$ 。

(3) G2 连续: 由于前段曲线的控制点为已知, 则令前段曲线在曲线末点处即 ($t=1$ 时) 的曲率为 $k_1(1)$ 。

由 G1 连续可得, $\eta(x_5 - x_4) = (x'_1 - x'_0)$, $\eta(y_5 - y_4) = (y'_1 - y'_0)$ 。令, $g = x_5 - x_4$, $h = y_5 - y_4$ 。后段的贝塞尔曲线的起始点 ($t=0$ 时) 的曲率 $k_2(0)$ 为:

$$k_2(0) = \frac{4[(x'_1 - x'_0)(y'_2 - 2y'_1 + y'_0) - (y'_1 - y'_0)((x'_2 - 2x'_1 + x'_0))]}{5((x'_1 - x'_0)^2 - (y'_1 - y'_0)^2)^{\frac{3}{2}}} \quad (4.16)$$

化简得到:

$$k_2(0) = \frac{4[g(y'_2 - y'_1 - \eta h) - h(x'_2 - x'_1 - \eta g)]}{5\eta^3(g^2 - h^2)^{\frac{3}{2}}} \quad (4.17)$$

则由 G2 连续可得:

$$\frac{4[g(y'_2 - y'_0) - h(x'_2 - x'_0)]}{5\eta^3(g^2 - h^2)^{\frac{3}{2}}} = k_1(1) \quad (4.18)$$

令, $(g^2 - h^2)^{\frac{3}{2}} = r$ 。由于 $Q_2(x'_2, y'_3)$ 、 $\dots$ 、 $Q_5(x'_5, y'_5)$ 均为已知点, 那么由式 (4.18) 计算可得:

$$\eta = \sqrt{\frac{[g(y'_2 - y'_0) - h(x'_2 - x'_0)]}{1.25k_1(1)r}} \quad (4.19)$$

当 $k_1(1) = 0$ 时, 表明曲线在拼接点处是一条直线。则辅助控制点 Q_1 为:

$$\begin{aligned} x'_1 &= \eta g + x'_0 \\ y'_1 &= \eta h + y'_0 \end{aligned} \quad (4.20)$$

此时, 便得到了在连接点处满足 G0、G1、G2 连续的两段贝塞尔曲线的控制点坐标, 即前后两段曲线拼接在了一起。

4.4.2 五阶贝塞尔曲线拼接三阶贝塞尔曲线

假定前段贝塞尔曲线的控制点为 $P_0(x_0, y_0)$ 、 $P_1(x_1, y_1)$ 、 $\dots$ 、 $P_5(x_5, y_5)$, 后段贝塞尔曲线的控制点为 $Q_0(x'_0, y'_0)$ 、 $Q_1(x'_1, y'_1)$ 、 $\dots$ 、 $Q_3(x'_3, y'_3)$ 。那么其连续性约束条件则为:

由于三阶贝塞尔曲线与五阶贝塞尔曲线仅存在系数的不同, 因此计算三阶辅助控制点的公式推导过程在此省略, 直接给出公式为:

$$\eta = \sqrt{\frac{[g(y'_2 - y'_0) - h(x'_2 - x'_0)]}{1.5k_1(1)r}} \quad (4.21)$$

当 $k_1(1) = 0$ 时，表明曲线在拼接点处是一条直线。则辅助控制点 Q_1 为：

$$\begin{aligned} x'_1 &= \eta g + x'_0 \\ y'_1 &= \eta h + y'_0 \end{aligned} \quad (4.22)$$

此时，便得到了在连接点处满足 G0、G1、G2 连续的两段贝塞尔曲线的控制点坐标，即前后两段曲线拼接在了一起。

4.5 仿真实验分析

4.5.1 场景决策仿真实验

本文基于 C++ 开发环境搭建了简易场景对本文提出的决策算法进行了仿真验证，仿真环境与实验结果如图 4.10 所示。

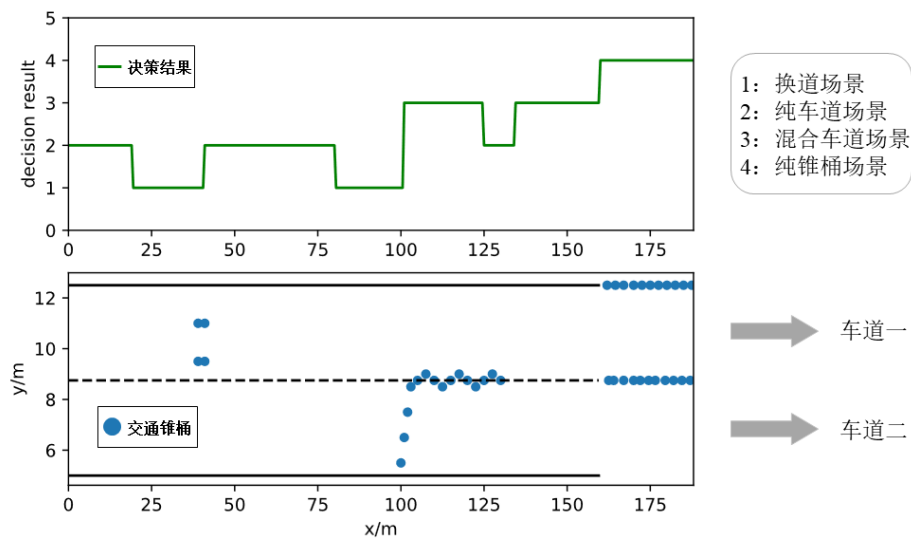


图 4.10 决策结果图

本文搭建的简易场景为图中的下图，场景为简易的直线双车道场景，其中 X 轴为车道的纵向长度，单位为 m；Y 轴为车道的纵向宽度，单条车道宽度为 3.75m；蓝色圆点代表锥桶。该简易场景从 0m 到 178m，所经历的场景工况为：首先在车道一处于纯车道场景，然后为换道场景，换道至车道二处于纯车道保持场景，然后为换道场景，换道至车道一处于混合车道场景，然后处于纯车道场景，然后处于混合车道场景，最后处于纯锥桶场景。

通过本文决策模块对该简易场景进行决策，输入为搭建的简易场景中的车道线

以及锥桶坐标，最终决策结果如图中上图所示，其中数字 1 代表换道场景；数字 2 代表纯车道场景；数字 3 代表混合车道场景；数字 4 代表纯锥桶场景。

由图可以看出本文的决策算法可以正确地决策出当前车辆所处的场景。

4.5.2 基于贝塞尔曲线的路径平滑拼接实验

为验证本文基于贝塞尔曲线的路径平滑拼接方法，基于 C++ 开发环境，对拼接方法进行了验证。

(1) 五阶贝塞尔曲线拼接五阶贝塞尔曲线

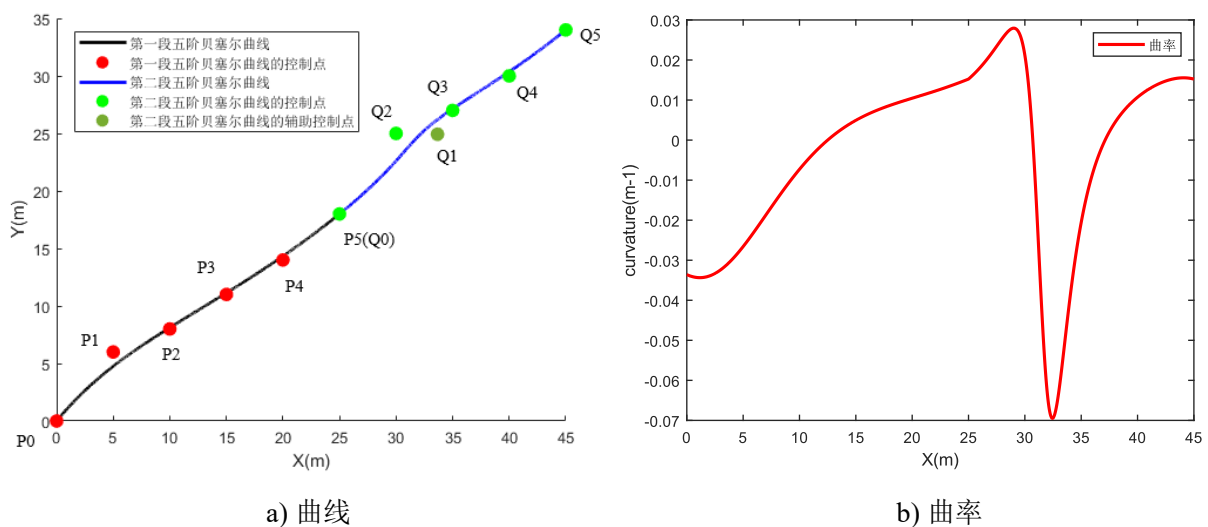


图 4.11 五阶贝塞尔曲线拼接五阶贝塞尔曲线的曲线及曲率图

前段五阶贝塞尔曲线的控制点为：(0,0)、(5,6)、(10,8)、(15,11)、(20,14)、(25,18)。

由于后段五阶贝塞尔曲线的起始点与前段曲线的末点重合，且 Q_1 辅助控制点为计算得到，则后段五阶贝塞尔曲线所需已知的控制点只需 4 个，为：(30,25)、(35,27)、(40,30)、(45,34)。

故，通过之前推导得到公式计算出 Q_1 辅助点为：(33.6603, 24.9282)

则前段五阶贝塞尔曲线与后段五阶贝塞尔曲线拼接后的曲线及其曲率如图 4.11 所示。可以看出拼接得到的曲线满足曲率在拼接处的连续，整体曲线曲率连续。

(2) 五阶贝塞尔曲线拼接三阶贝塞尔曲线

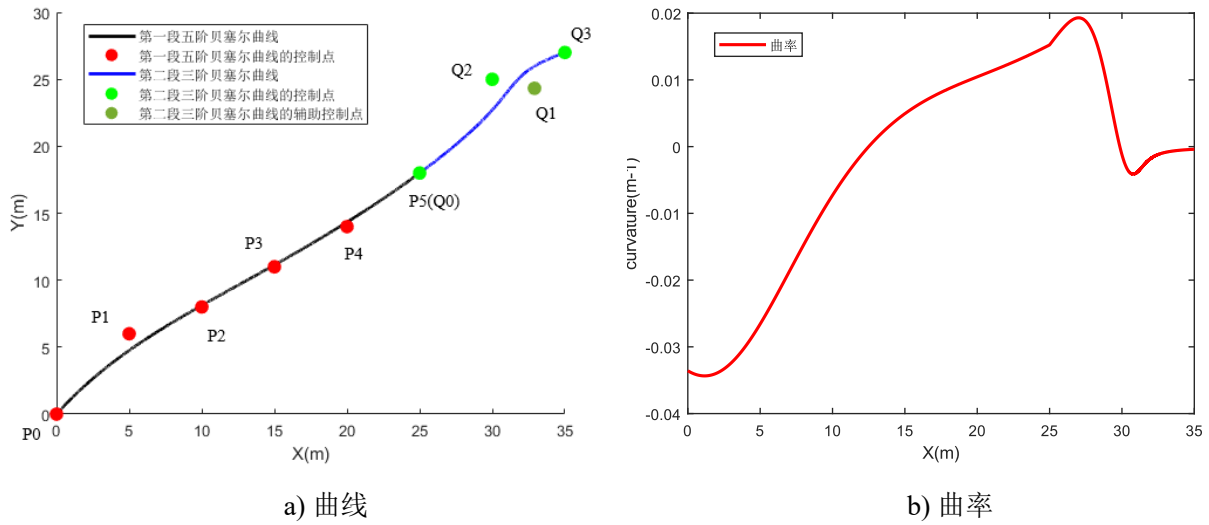


图 4.12 五阶贝塞尔曲线拼接三阶贝塞尔曲线的曲线及曲率图

前段五阶贝塞尔曲线的控制点为：(0,0)、(5,6)、(10,8)、(15,11)、(20,14)、(25,18)。

由于后段五阶贝塞尔曲线的起始点与前段曲线的末点重合，且 Q_1 辅助控制点为计算得到，则后段五阶贝塞尔曲线所需已知的控制点只需 2 个，为：(30,25)、(35,27)。

故，通过之前推导得到公式计算出 Q_1 辅助点为：(32.9057,24.3246)

则前段五阶贝塞尔曲线与后段五阶贝塞尔曲线拼接后的曲线及其曲率如图 4.12 所示。可以看出拼接得到的曲线满足曲率在拼接处的连续，整体曲线曲率连续。

4.5.3 换道场景下的路径规划仿真实验

在真实交通环境中，由于车道宽度的限制，横向位移一般不会发生变化，但交通场景的随机性较大。因此为了验证本文的五次多项式换道规划方法的适应性，基于 C++ 开发环境，分别在不同纵向位移和纵向车速下，分析该方法鲁棒性。

(1) 保持纵向车速 $v_x = 10\text{m/s}$ ，换道横移距离为 3.75m，改变纵向位移，将纵向位移分别设置为 50m、40m、30m，则对应状态量为：

$$\text{a: } q_x = [0, 10, 0, 50, 10, 0], q_y = [0, 0, 0, 3.75, 0, 0]$$

$$\text{b: } q_x = [0, 10, 0, 40, 10, 0], q_y = [0, 0, 0, 3.75, 0, 0]$$

$$\text{c: } q_x = [0, 10, 0, 30, 10, 0], q_y = [0, 0, 0, 3.75, 0, 0]$$

对以上三种工况进行仿真，得到仿真结果如图 4.13 所示。

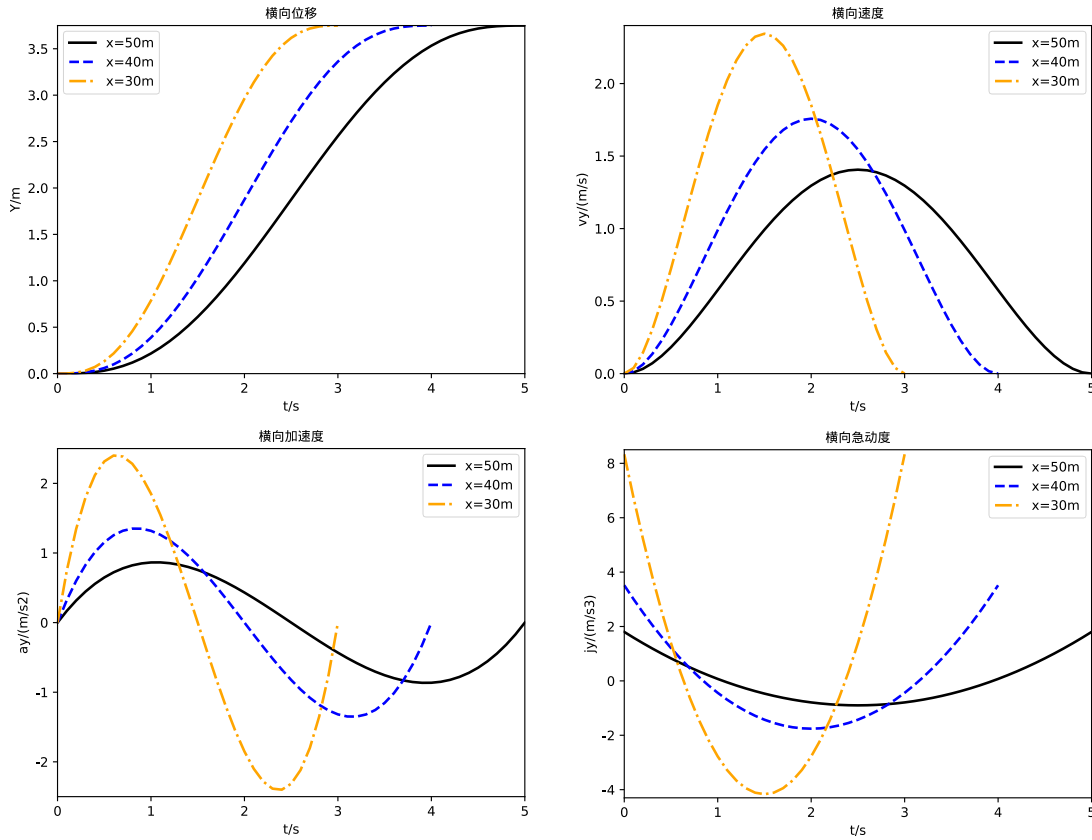


图 4.13 不同纵向位移下的换道路径规划仿真结果

由图 3-12 可知，纵向速度 $v_x = 10\text{m/s}$ 时，在不同纵向位移下，五次多项式换道轨迹在纵向位移 x 等于 30m 时，最大横向速度为 2.33m/s ，最大横向速度加速度为 2.43m/s^2 ，最大横向急动度为 -4.12m/s^3 ；在纵向位移 x 等于 40m 时，最大横向速度为 1.76m/s ，最大横向加速度为 1.37m/s^2 ，最大横向急动度为 -1.73m/s^3 ；在纵向位移 x 等于 50m 时，最大横向速度为 1.4m/s ，最大横向加速度为 0.86m/s^2 ，最大横向急动度 -0.83m/s^3 ；

(2) 保持纵向位移 $x = 60\text{m}$ 不变，改变纵向速度，将纵向速度分别设置为 5m/s^2 、 10m/s^2 、 15m/s^2 ，对应状态量为：

$$\text{a: } q_x = [0, 5, 0, 60, 5, 0], q_y = [0, 0, 0, 3.75, 0, 0]$$

$$\text{b: } q_x = [0, 10, 0, 60, 10, 0], q_y = [0, 0, 0, 3.75, 0, 0]$$

$$\text{c: } q_x = [0, 15, 0, 60, 15, 0], q_y = [0, 0, 0, 3.75, 0, 0]$$

对以上三种工况进行仿真，得到仿真结果如图 4.14 所示。

由图 3-12 可知, 在纵向位移保持 60m 不变时, 纵向速度 $v_x = 15\text{m/s}$ 时, 最大横向速度为 1.754m/s , 最大横向速度加速度为 1.352m/s^2 , 最大横向急动度为 -1.77m/s^3 ; 在纵向速度 $v_x = 10\text{m/s}$ 时, 最大横向速度为 1.31m/s , 最大横向加速度为 0.602m/s^2 , 最大横向急动度为 -0.53m/s^3 ; 在纵向速度 $v_x = 5\text{m/s}$ 时, 最大横向速度为 0.585m/s , 最大横向加速度为 0.147m/s^2 , 最大横向急动度为 -0.1m/s^3 ;

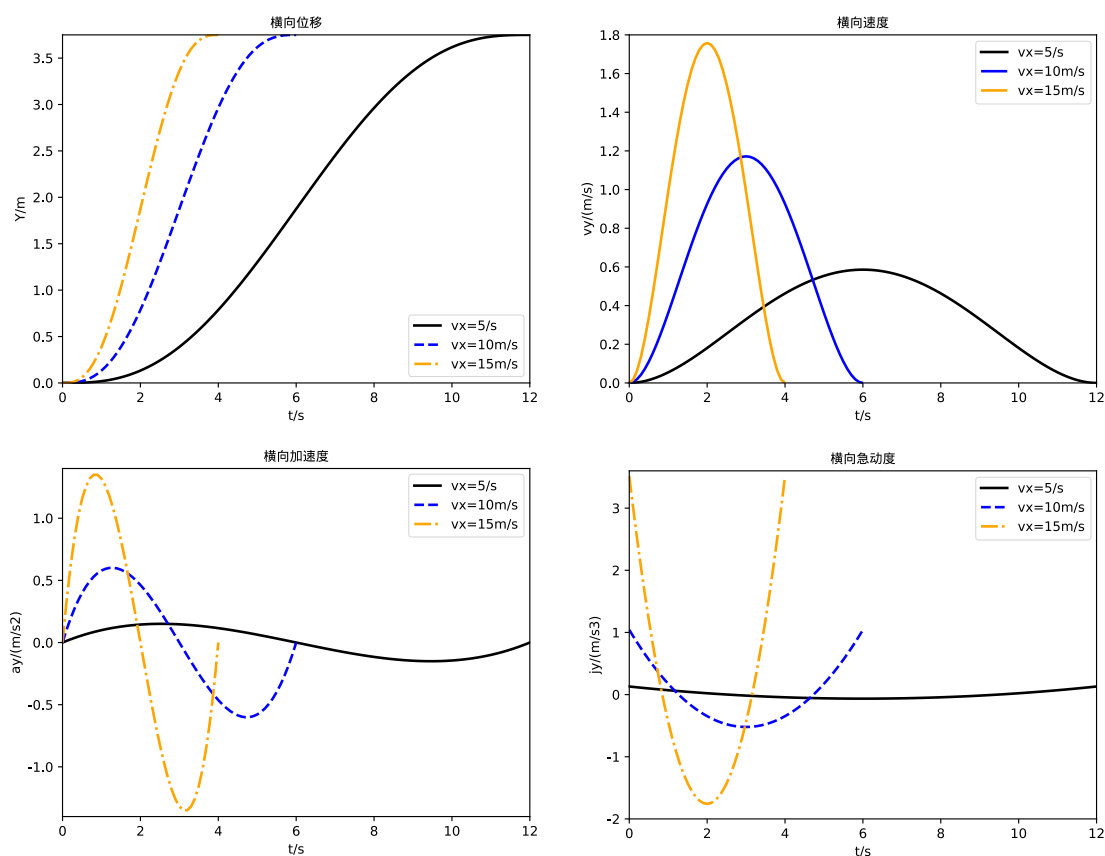


图 4.14 不同纵向速度下的换道路径规划仿真结果

根据以上两种情况的分析结果, 可以得到五次多项式规划得到的路径平滑且连续, 满足后续路径跟踪控制器的要求。

4.5.4 纯车道场景下的路径规划仿真实验

为验证本文纯车道场景下的路径规划算法, 本文基于 C++ 开发环境, 搭建简易实验场景, 对算法进行了仿真验证。

搭建的场景与规划出的路径如图 4.15 所示。

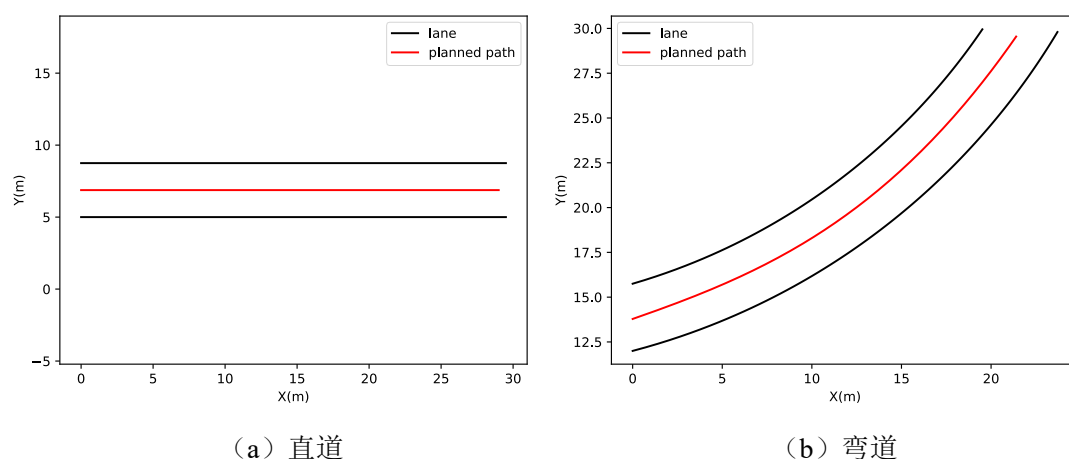


图 4.15 纯车道场景下路径规划示意图

图中，黑色实线为车道线，红色实线为纯车道场景下的路径规划算法规划出的路径。其曲率如图 4.16 所示。

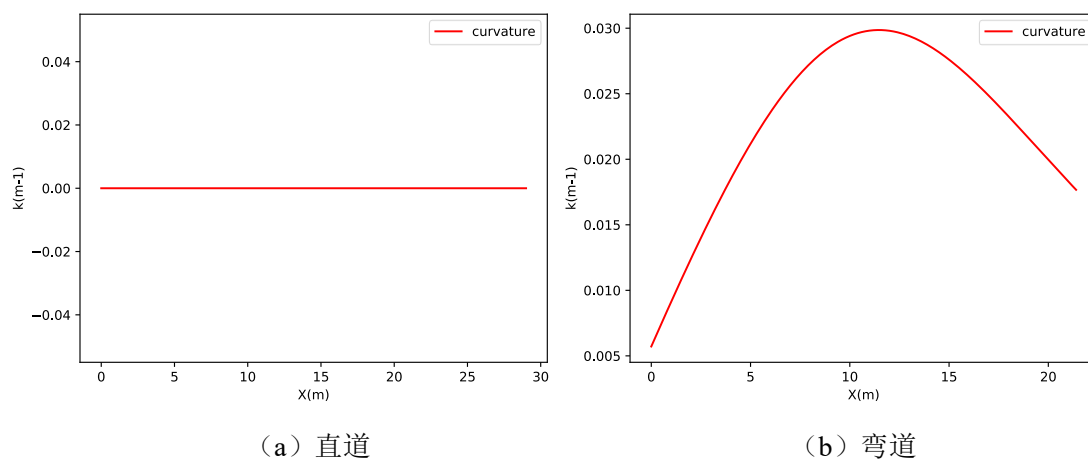


图 4.16 纯车道场景下路径规划算法规划出的路径的曲率

曲率连续且平滑，因此本文纯车道场景下的路径规划算法规划出的路径可行。

4.5.5 混合车道场景下的路径规划仿真实验

为验证本文纯车道场景下的路径规划算法，基于 C++ 开发环境，搭建简易实验场景，对算法进行了仿真验证。

本文分别搭建了直线道路工况与弯道工况来验证本文的算法。搭建的场景与规划出的路径如图 4.17 所示。

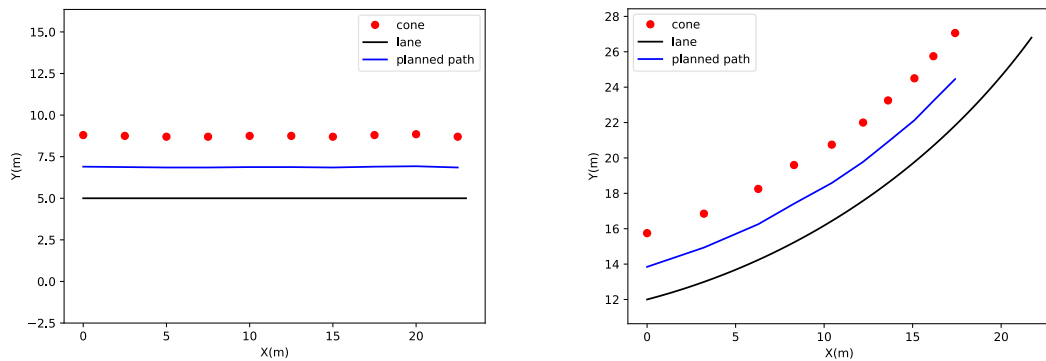
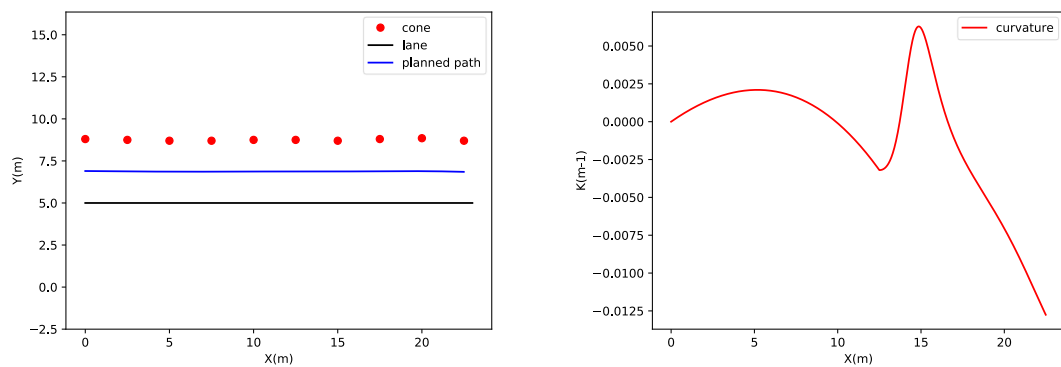
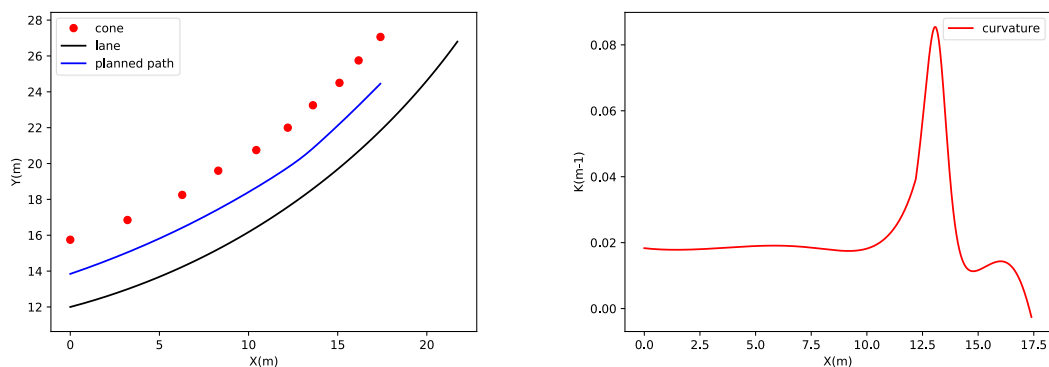


图 4.17 混合车道场景下路径规划示意图

图中，黑色实线为车道线；红色圆点为锥桶；蓝色实线为混合车道场景下路径规划算法规划出的路径。由图可以看出路径不平滑，因此需要对其进行多段贝塞尔曲线拼接平滑。平滑后的路径与其曲率如图 4.18 所示。



(a) 直线道路工况



(b) 弯道工况

图 4.18 平滑后的路径以及路径曲率

由图可以看出，贝塞尔曲线平滑后的路径，路径平滑且曲率连续。因此本文在

混合车道场景下的路径规划算法规划出的路径可行。

4.5.6 纯锥桶场景下的路径规划仿真实验

为验证本文纯车道场景下的路径规划算法，基于 C++ 开发环境，搭建简易实验场景，对算法进行了仿真验证。

本文分别搭建了直线道路工况与弯道工况来验证本文的算法。搭建的场景与规划出的路径如图 4.19 所示。

图中绿色菱形块为锥桶；绿色虚线为搜索空间离散化后的 Delaunay 三角形的边；红色圆点为 Delaunay 三角形的边的中点，即路径备选点；蓝色实线为纯锥桶场景下的基于搜索的路径规划算法规划出的路径。

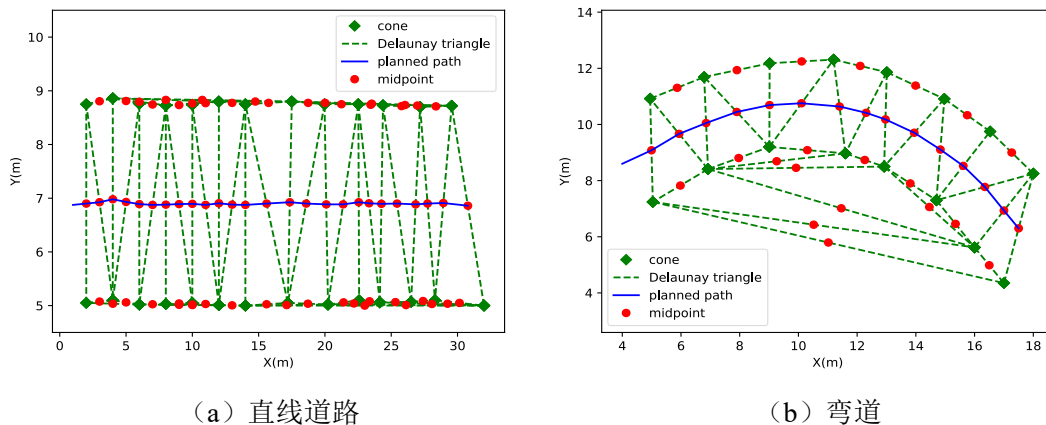
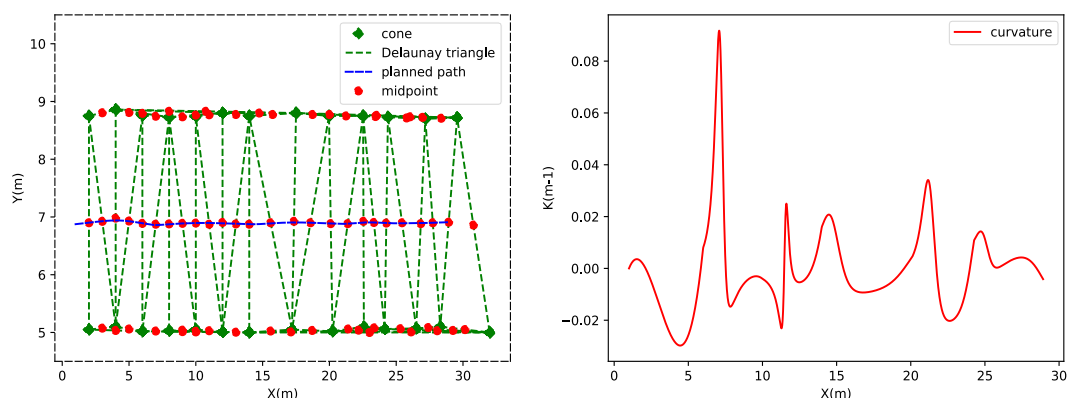
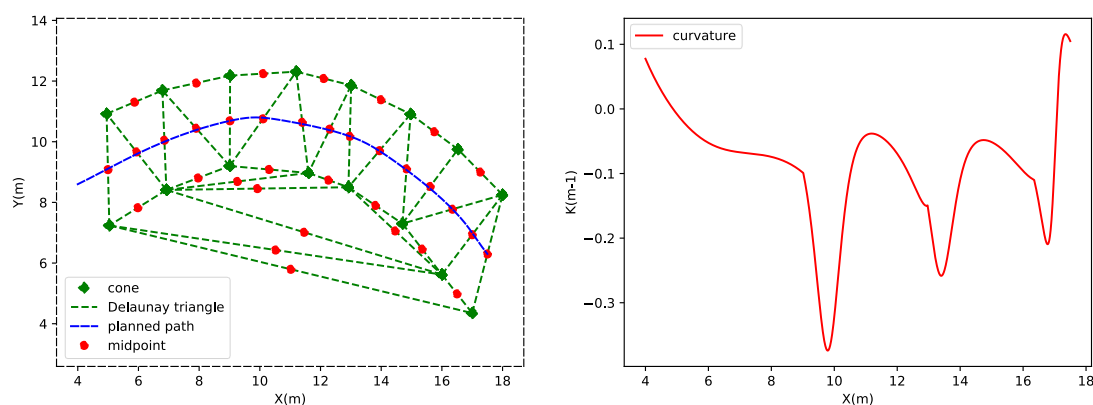


图 4.19 纯锥桶场景下的路径规划算法路径示意图

由图中路径可以看出，由该方法直接规划出的路径为点对点的连接，因此路径不平滑，曲率不连续，故使用多段贝塞尔曲线拼接对其进行平滑处理。平滑后的路径及其曲率如图 4.20 所示。



(a) 直线道路工况



(b) 弯道工况

图 4.20 平滑后的路径及其曲率图

由图可以看出，多段贝塞尔曲线拼接平滑后的路径，整体路径平滑且曲率连续，且路径正确，因此本文在纯锥桶场景下的路径规划算法规划出的路径可行。

4.6 本章小结

本章依据锥桶在真实道路交通场景中的应用，选取了本文所研究的四种工况，并对其进行了简化，继而确定了本文需要决策的场景类型，并设计了决策算法；然后设计了本文所决策的四种场景下的路径规划算法；其次针对得到的路径曲率不连续的问题，设计了多段贝塞尔曲线进行拼接平滑路径；最后在 C++ 开发环境下验证了本文的决策算法以及四种场景下的规划算法的鲁棒性以及路径平滑方法对路径平滑后，整体路径是否平滑且曲率是否连续。所设计的决策算法与四种场景下的规划算法规划出的路径将作为路径跟踪控制模块输入期望参考路径。

5 基于 Pure Pursuit 算法的路径跟踪控制

本章将对 Pure Pursuit 算法（纯跟踪算法）的基本原理进行介绍，给出算法运用到实车路径跟踪控制的过程。首先依托 2021 i-VISTA^[75]自动驾驶汽车挑战赛-低速无人驾驶挑战赛，使用参赛车辆对比赛路线进行了参考路径提取。然后，建立了车辆运动学模型，使用 Pure Pursuit 算法确定车辆前轮转角，进行路径跟踪，然后基于 ROS（机器人操作系统）^[76]，对算法跟踪控制及跟踪误差进行了仿真分析。最后，通过 2021 i-VISTA 自动驾驶汽车挑战赛-低速无人驾驶挑战赛的参赛车辆，对本文所实现的控制方法进行了实车实验验证。

5.1 参考路径提取

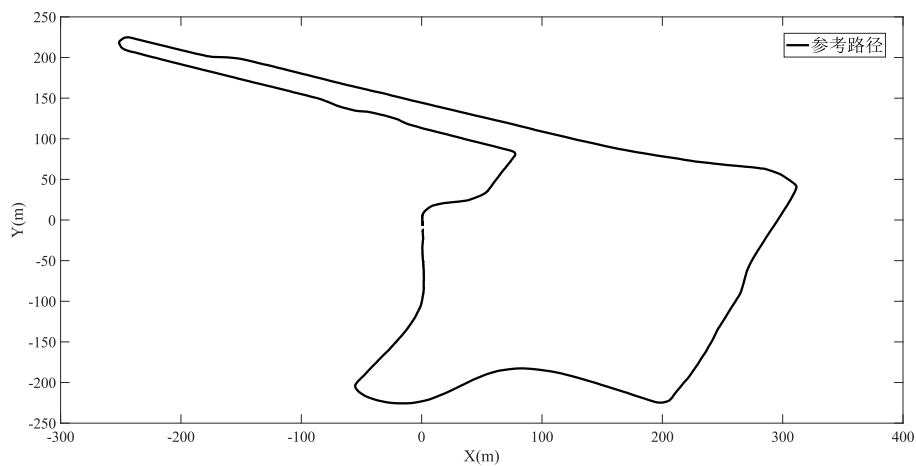
依托参赛车辆所搭载的 velodyne16 线激光雷达，如图 5.1 所示，使用 SLAM（Simultaneous Localization And Mapping）技术进行比赛场景的建图以及参考路径的提取，步骤如下：



图 5.1 比赛参赛及后续实车实验车辆

首先遥控参赛车辆沿比赛路线行驶，使用激光雷达 SLAM 算法对比赛场景进行建图，同时定位，当遥控参赛车辆按比赛路线再次行驶到起点时，结束算法。然后依据 SLAM 算法最终得到的地图，提取其中遥控车辆行驶的 Pose 点，即为比赛路线的参考路径。

最终提取出的参考路径如图 5.2 所示。



(a) 参考路径



(b) 比赛路线示意图

图 5.2 参考路径及比赛路线示意图

5.2 基于 Pure Pursuit 算法的路径跟踪

5.2.1 车辆运动学模型

车辆低速行驶时，转向轮基本无侧向力，可忽略轮胎滚动时的轮胎侧偏角。此时车辆转向模型如图 5.3 所示，外侧轮胎转弯半径大于内侧轮胎^[77]，即 Ackermann 转向模型。

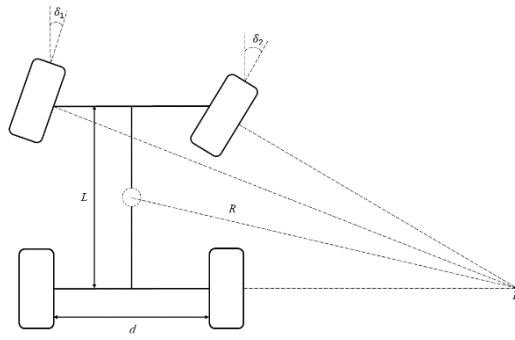


图 5.3 阿克曼转向模型

由图 5.3 可知，小转向角时，据车辆几何关系，可得转向角：

$$\begin{cases} \delta_1 = \frac{L}{R + \frac{d}{2}} \\ \delta_2 = \frac{L}{R - \frac{d}{2}} \\ \delta_1 \approx \delta_2 \end{cases} \quad (5.1)$$

式中， d 为轮距， L 为轴距， R 为转弯半径， δ_1 为左前轮转角， δ_2 为右前轮转角， P 为转向中心。为简化计算，将车辆四轮模型转换为车辆自行车模型，并假设车辆在平面行驶^[78]。简化后的模型如下图。

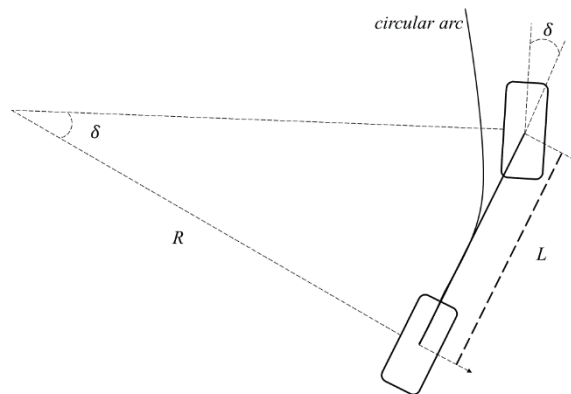


图 5.4 车辆自行车模型

车辆单轨模型是由四轮模型简化得到的，只考虑车轮在平路上行驶。使用该模型的好处是可以预测车辆在低速场景下的运动，前轮、后轴转向角遵循曲率关系，由图 2-3 及式(5.1)可知：

$$\tan \delta = \frac{L}{R} \quad (5.2)$$

式中， δ 表示前轮转向角， R 表示在给定转向角下的后轴所遵循的圆半径。

5.2.2 Pure Pursuit 算法

Pure Pursuit 算法作为移动机器人领域的路径跟踪算法，自提出以来受到广泛引用，因为其具有简单以及跟踪精准的特点，随着智能汽车的飞速发展，该算法在智能汽车的路径跟踪控制上也得以应用^[79]。Pure Pursuit 算法的研究对象为单车模型的后轴中心点，通过控制车辆前轮转角，使车辆沿着参考路径上的目标点行驶^[80]，跟踪示意图，如图 5.5 所示。

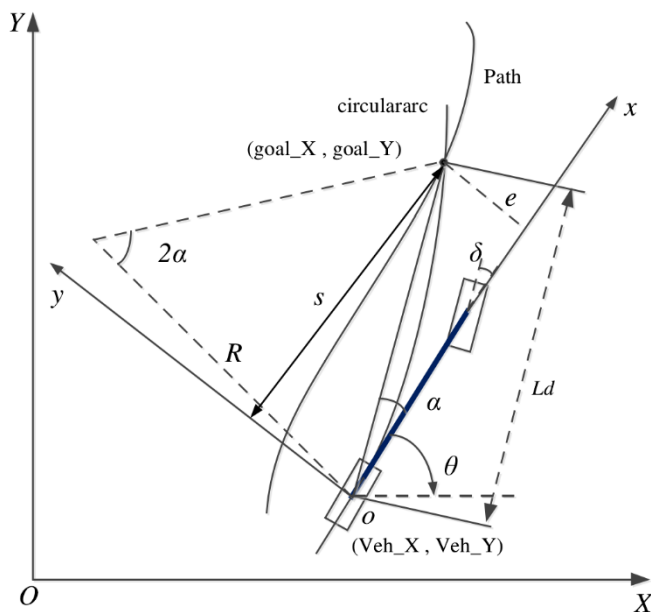


图 5.5 路径跟踪示意图

由图上可知， XOY ：大地坐标系； xoy ：车辆坐标系； $(goal_X, goal_Y)$ ：预瞄点坐标； (Veh_X, Veh_Y) ：车辆坐标。结合正弦定理可知：

$$\begin{cases} \frac{Ld}{\sin(2\alpha)} = \frac{R}{\sin(\frac{\pi}{2} - \alpha)} \\ \frac{Ld}{2\sin\alpha\cos\alpha} = \frac{R}{\cos\alpha} \\ \frac{Ld}{\sin\alpha} = 2R \end{cases} \quad (5.3)$$

式中， Ld 表示车辆后轴中心位置到预瞄点的距离， α 表示目前车身姿态和预瞄点之间的夹角，道路曲率为 k ，如式(5.4)所示：

$$k = \frac{1}{R} \quad (5.4)$$

由式(5.2)、式(5.3)和式(5.4)可求出前轮转角，如式(5.5)所示：

$$\delta = \arg \tan \left(\frac{2L\sin(\alpha)}{Ld} \right) \quad (5.5)$$

预瞄点在车辆坐标系下的位置坐标设为 $(goal_x, goal_y)$ ，由图 2-4 中可知，与

点($goal_X, goal_Y$)重合, 通过如式(5.6)所示的大地坐标系与车辆坐标系之间转化关系^[81]:

$$\begin{bmatrix} goal_x \\ goal_y \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \cdot \begin{bmatrix} goal_X - Veh_X \\ goal_Y - Veh_Y \end{bmatrix} \quad (5.6)$$

式中, θ 表示车辆航向角, 则由式(5.7)可知:

$$\sin\alpha = \frac{goal_y}{\sqrt{(goal_x)^2 + (goal_y)^2}} \quad (5.7)$$

式中, $goal_x$ 表示车辆当前位置坐标距离参考路径点的横向偏差, 定义一个新的变量 e 来表示; $goal_y$ 表示车辆预瞄距离, 设为 s 。则前轮转角为:

$$\delta = \arctan \frac{2Le}{e^2 + s^2} \quad (5.8)$$

预瞄距离 s 的选取与车速、跟踪误差有关^[82], 过大的预瞄距离会致使路径跟踪存在迟滞, 过小的预瞄距离会致使路径跟踪发生震荡。在实际运用过程中需要考虑到道路参考路径曲率^[83]。本章路径跟踪预瞄距离选取考虑车速的影响, 预瞄距离有两部分构成, 基础预瞄距离设置为 2m, 考虑到车速的预瞄距离增益, 增益系数设置为 0.1。横向偏差 $e = goal_x$ 成立条件^[83]:

$$[(goal_x)_{x(i)} - s] \cdot [(goal_x)_{x(i+1)} - s] \leq 0 \quad (5.9)$$

5.3 控制器效果分析

5.3.1 仿真分析

为了验证路径跟踪算法的有效性, 本文基于 ROS 平台来进行实验环境搭建以及仿真实验。

(1) ROS 机器人操作系统

ROS (Robot Operating System) 是与 2010 年发布的主要应用于机器人开发的操作系统, 经过长时间的发展, 已经成为了机器人开发领域的主要平台。具有模块化、分布式的特点。由于智能驾驶汽车本质上也可看做一个四轮机器人, 因此可以基于 ROS 对感知、决策以及规划控制算法进行模块化开发, 降低了相关研究人员的集成设计和程序编写的难度。同时 ROS 也提供了十分丰富的功能, 来满足开发者的需要^[84]。

(2) GAZABO 物理模型平台

在仿真实验中需要给机器人提供与现实相近的虚拟物理环境, 如道路环境、物理距离等。GAZABO 是 ROS 操作平台中的一款机器人仿真工具, 具有强大的物理

引擎，与 ROS 兼容性很高。GAZEBO 具备多种环境库，能够轻松构建出实验所需的静态环境。搭建的模型^[85]如图 5.6 所示。

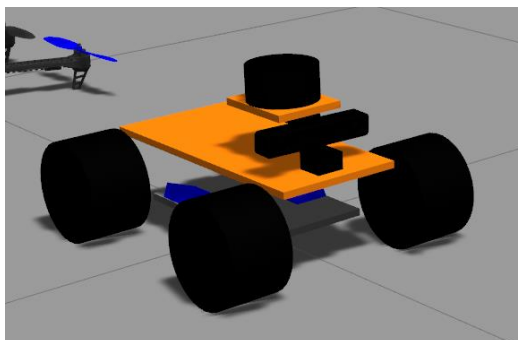


图 5.6 阿克曼转向小车模型

(3) RVIZ 可视化工具

为了使开发者可以直观的观察算法运行的效果，需要对算法执行过程中产生的数据进行可视化处理。

ROS 中有一款三维可视化平台—RVIZ。在 RVIZ 中，提供了多种数据输入接口，通过订阅 ROS 话题或者服务的形式，将产生的数据进行实时传输，并对数据进行图形或展示，以此来显示周围环境^[86]。同时 RVIZ 还提供了地图数据接口，通过加载制作好的地图文件，可以显示车辆所在的真实环境。通过模型、感知数据以及地图就可以实时观察车辆的运行状态。RVIZ 可视化平台界面如图 5.7 所示。

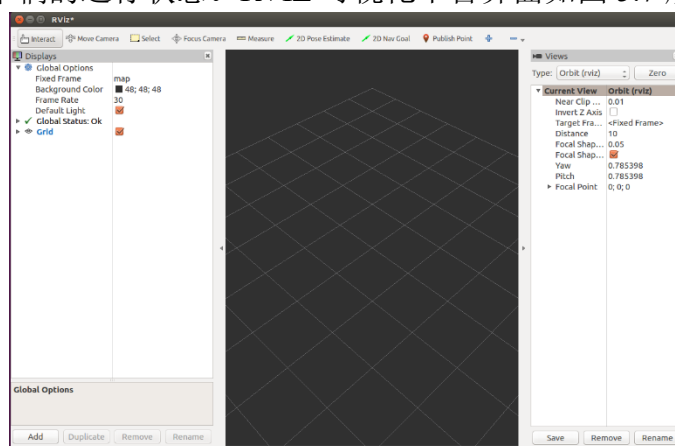


图 5.7 RVIZ 界面

(3) 仿真实验及结果分析

由于比赛限定最高车速，因此本文将车速固定为 12km/h，车辆的轴距为 1m。在仿真环境中，基于 ROS 系统运行算法，向 GAZEBO 中的模型车辆发送控制信息，同时获取车辆的信息发送给算法进行计算，得到当前的前轮转向角，实现数据闭环，从而进行算法的仿真验证^[87]。

仿真结果如图 5.8 所示。

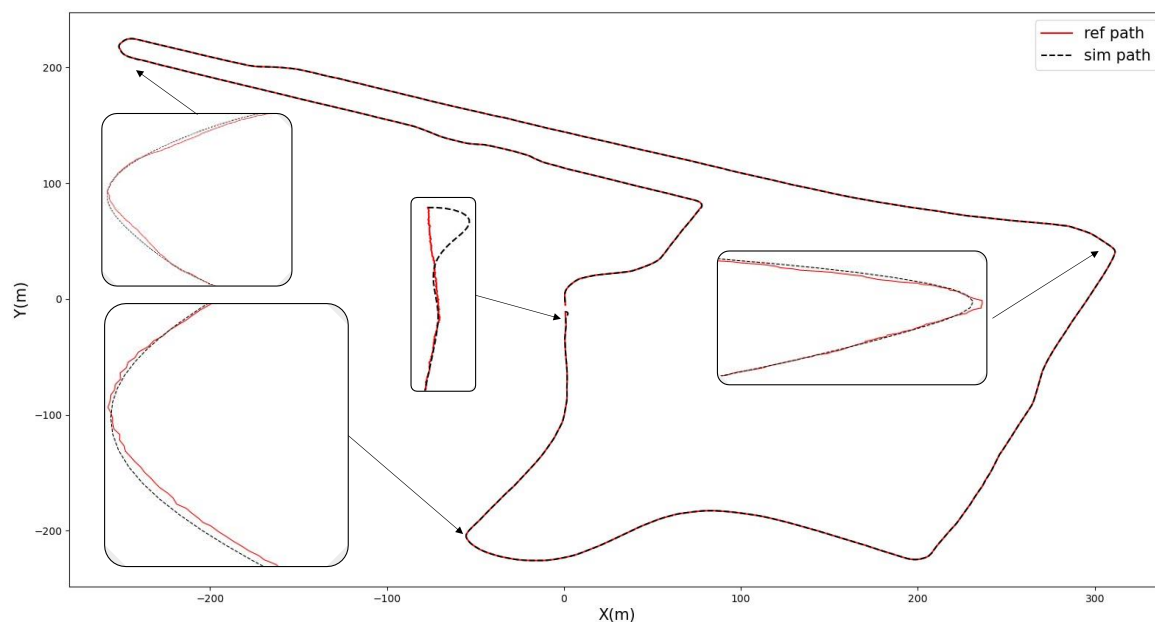


图 5.8 Pure Pursuit 控制算法仿真结果

由图 5.8 可知，车速设置为 12km/h 时，算法仿真得到的轨迹于参考轨迹在初始阶段存在振荡现象，但只持续了很短的时间，之后仿真轨迹与参考轨迹在直线度几乎重合，而在弯道处则产生了较大误差，可能是由于预瞄点的设置所导致的，整体仿真结果与参考轨迹的误差不超过 0.2m。

故通过仿真可以得知，在车辆低速行驶时，Pure Pursuit 算法可以很好的跟踪上参考轨迹。

5.3.2 实车分析

在 5.3.1 中，使用 ROS 机器人操作系统来进行路径跟踪算法仿真试验，验证了本文 Pure Pursuit 路径跟踪算法的效果。但是仿真实验不足以验证该的有效性，因为真实环境中包含诸多不可控因素，因此需要通过实车实验来验证路径跟踪算法的效果。

(1) 实车实验平台

本文使用的实车实验平台为 2021 i-vista 自动驾驶汽车挑战赛-低速无人驾驶挑战赛参赛车辆，如图 5.9 所示。

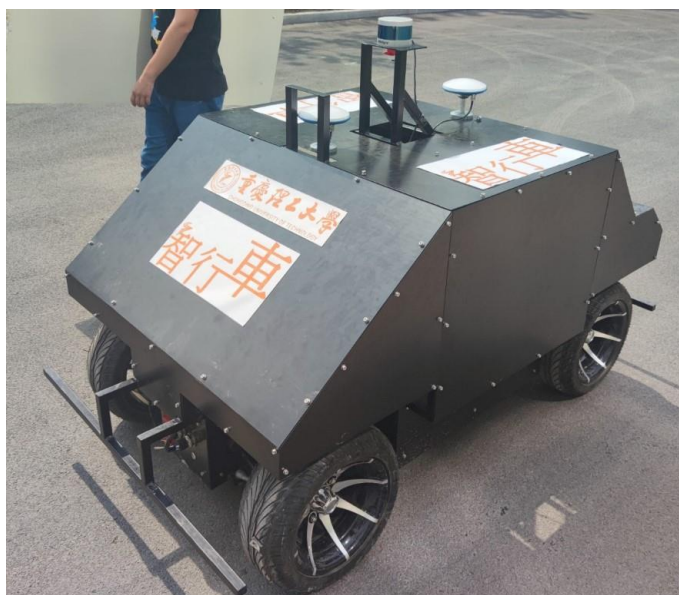


图 5.9 实车实验平台

实车实验平台基于中云智车线控底盘，搭载了 velodyne16 线激光雷达传感器，工控机为个人笔记本电脑，软件系统与硬件系统之间的通信通过 CAN 协议进行通信。车辆的软硬件系统架构图如图 5.10 所示。

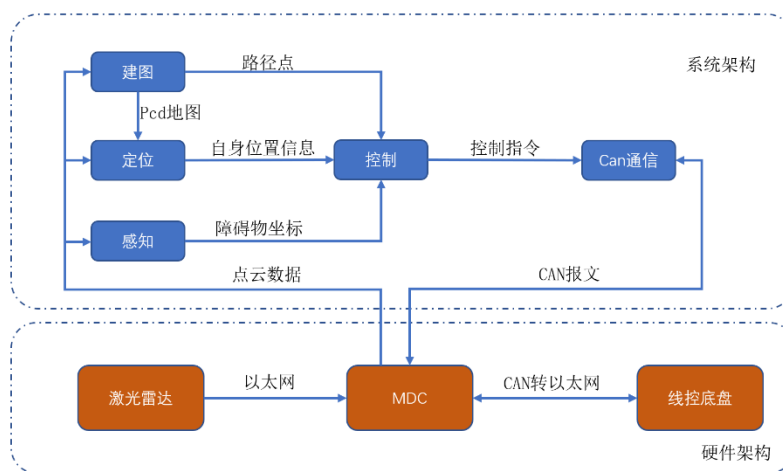


图 5.10 软硬件系统架构

(2) 实车实验及结果分析

首先通过车顶的 16 线激光雷达感知周围环境，得到环境的激光雷达点云信息，与之前提取参考路径时建立的环境地图进行 NDT 点云匹配，确定车辆当前的位置，然后使用 Pure Pursuit 算法对参考路径进行跟踪，计算出前轮转角，通过底盘 CAN 通信，使用 CAN 转以太网模块向线控底盘发送控制指令，纵向车速固定为与仿真相同

的值，为 12km/h，实验结果如图 5.11 所示。

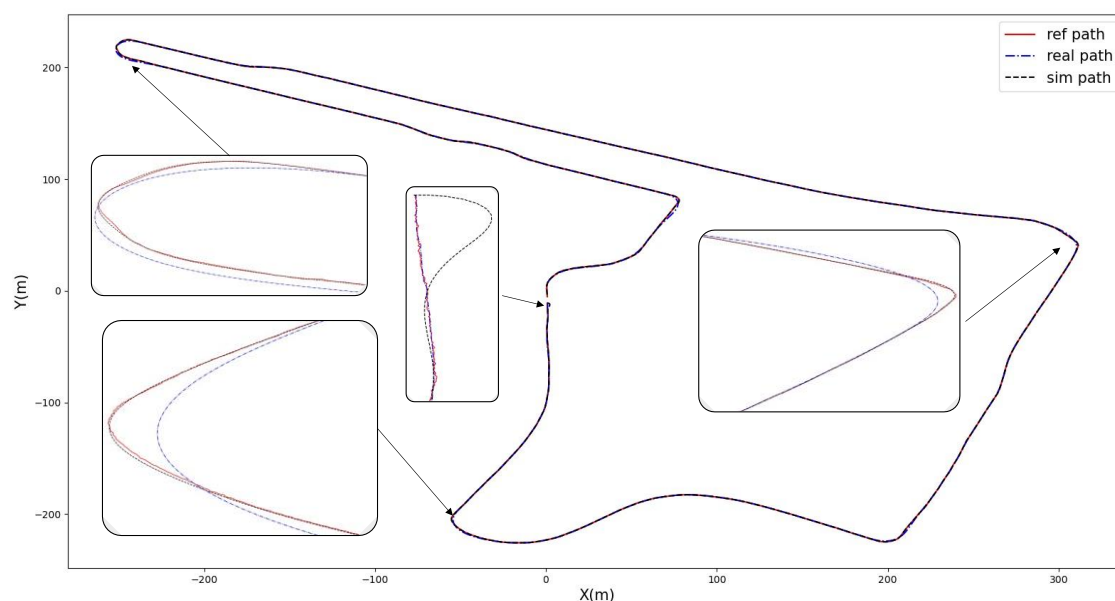


图 5.11 仿真实车实验轨迹曲线

由图可知，车速为 12km/h 时，实车运行得到的轨迹与参考轨迹的变化趋势接近一致。实车轨迹的初始阶段相比于仿真轨迹与参考轨迹相比，没有发生振荡现象。而整个实车实验过程中，在直线段实车轨迹与仿真轨迹几乎重合，误差不超过 0.3m，而在弯道段，出现了较大的误差，最大误差不超过 1m。

故通过实车实验可以得知，在车辆低速行驶时，Pure Pursuit 算法可以很好的跟踪上参考轨迹，并且依据本章实现的 Pure Pursuit 算法，获得了 2021 i-VISTA 自动驾驶汽车挑战赛-低速无人驾驶挑战赛的卓越奖（第三名）。

5.4 本章小结

本章依托 2021 i-vista 自动驾驶汽车挑战赛-低速无人驾驶挑战赛，提取了比赛场景的参考路径，基于车辆运动学模型，搭建了 Pure Pursuit 路径跟踪控制器，并基于 ROS 系统进行了路径控制算法的仿真，并借助参赛车辆进行了实车路径跟踪实验，仿真与实验结果表明，Pure Pursuit 算法在低速工况下，可以满足无人驾驶车辆路径跟踪的要求。

6 总结与展望

6.1 总结

本文对真实道路环境中的交通锥桶的检测识别与定位，以及锥桶场景工况下的路径规划与路径跟踪控制进行了相关研究。交通锥桶在道路环境中，使用非常灵活，起到约束、引导车辆的作用，因此对其展开检测识别与定位十分重要，本文使用 YOLOv3-tiny 对交通锥桶进行检测识别，并针对其不足进行了改进。与此同时，本文针对在车辆在锥桶环境下的车辆决策与路径规划控制展开了相关研究，针对锥桶出现的不同场景，设计了决策算法对其进行决策，然后针对不同的场景设计了不同的路径规划算法，并设计了路径跟踪控制器，并通过软件仿真与实车实验验证了控制算法的有效性。本文的主要研究如下：

(1) 基于深度学习算法对交通锥桶进行检测识别，对自动驾驶数据集 nu-scene 进行数据清理，只保留锥桶类别的图片，并与人工采集的生活中常见的锥桶图像进行合并，得到本文的锥桶数据集；然后针对 YOLOv3-Tiny 算法在检测锥桶时存在的不足，对其进行了改进，首先增加网络输入尺寸为 640×640 ，其次更改损失函数为 Focal loss，然后对 YOLOv3-Tiny 网络的主干网络进行了改进，替换最大池化层为 Ghost-Bottleneck 模块，最后融合浅层特征，构建新的特征金字塔，以提高对锥桶小目标的检测效果。最后对本文改进后的算法进行了实验，改进后相较于改进前对锥桶检测的精确率与召回率分别提升了 8.7 与 25.7 个百分点，检测效果得到了显著的提升。

(2) 依据锥桶在现实世界中的尺寸为已知这一先验信息，实现了三种锥桶在相机坐标系下的定位方法。通过分析锥桶的几何特征，设计了锥桶关键点检测网络，并制作了数据集，通过模型训练得到了锥桶关键点检测模型，然后利用 PnP 算法实现了第一种基于锥桶关键点的 PnP 定位方法；通过将锥桶抽象为圆柱体，将其圆心横截面矩形在图像坐标系上的投影近似为锥桶的检测框，利用 PnP 算法实现了第二种基于检测框的 PnP 定位方法；通过分析相机成像模型，利用针孔相机成像模型实现了第三种基于针孔相机成像模型的锥桶定位方法。最后在实车实验平台上，利用两颗工业摄像头进行了定位实验，验证了其定位精度，可以满足车辆在行驶中的锥桶定位要求。

(3) 依据锥桶在真实道路交通场景中的应用，选取了本文所研究的四种锥桶环境下的工况，并对其进行了简化。然后根据简化后的工况，确定了本文需要决策的

场景类型，并设计了决策算法；然后分别设计了本文所决策的四种场景下的路径规划算法；其次针对混合车道场景与纯锥桶场景下规划得到的路径曲率不连续的问题，设计了多段贝塞尔曲线进行拼接来平滑路径，包括五阶贝塞尔曲线拼接五阶贝塞尔前、五阶贝塞尔曲线拼接三阶贝塞尔曲线。然后基于 C++ 开发平台对算法进行了仿真验证。

（4）依托 2021 i-vista 自动驾驶汽车挑战赛-低速无人驾驶挑战赛，提取了比赛场景的参考路径，基于车辆运动学模型，搭建了 Pure Pursuit 路径跟踪控制器，并基于 ROS 系统进行了路径跟踪控制算法的仿真，并借助参赛车辆进行了实车路径跟踪实验，对算法的路径跟踪效果进行了验证。

6.2 展望

本文虽然对车辆在交通锥桶环境下的感知决策规划控制有所研究，但是仍有几个关键问题需要进一步解决，以及一些问题需要进一步完善：

（1）本文所设计的交通锥桶检测算法，虽然对日间的锥桶检测效果较好，但由于数据集的限制，算法对夜间、大雾天气等极端场景下的锥桶检测仍存在问题，需要进一步研究。

（2）本文所研究的锥桶定位，是建立在锥桶的尺寸已知的前提下，但由于锥桶的尺寸有大中小等几类，因此今后的工作中，可以对不同锥桶的真实尺寸进行进一步研究。

（3）本文所设计的路径拼接平滑算法，虽然满足了在连接点出的曲率连续，但可能存在曲率的变化率突变，因此今后的研究中，可以将辅助控制点选取的条件中加入曲率变化率的连续。

（4）本文只对锥桶的检测、识别以及路径跟踪控制进行了实车验证，而对于场景决策与规划只进行了仿真，因此希望在今后的工作中可以将其作为一个系统进行实车验证。

致谢

回顾研究生三年时光，如白驹过隙，三年里，两年半的时间与疫情相伴，也第一次在家待了那么长时间，也时隔很久再次感受到了曾经以为以治理好了得家里的风沙，还是那么大。

在这三年里，要感谢课题组里的每位导师，胡远志教授、刘西老师、任立海老师、蒋成约老师对我的关心与照顾。感谢课题组提供的丰富的学习资源。

感谢李星月、段锦程、张嘉成、宋佳、陈羿先、王荡荡、耿庄程、祝恩朋、王中铭、张志阳等师兄在生活和学习上的帮助。

感谢张隆、曾宪员、李东峻、雷发林、李盛、蒋涛、李世杰等同门的支持和帮助。

感谢王德朝、董泰宏、石得君等师弟的支持和帮助。

最后我要感谢我的父母、我的女朋友、我远在其他学校的朋友们这一路上的关心与照顾，让我可以无拘无束，肆意表达。

参考文献

- [1] 李章维, 胡安顺, 王晓飞. 基于视觉的目标检测方法综述 [J]. 计算机工程与应用, 2020, 8: 1-9.
- [2] HOSANG J, BENENSON R, PIOTR D, et al. What Makes for Effective Detection Proposals? [J]. IEEE Transactions on Pattern Analysis & Machine Intelligence, 2016, 38(4): 814.
- [3] 杜晨健. 基于图像处理的交通锥识别与定位算法研究 [D]; 郑州大学, 2017.
- [4] 鄢磊澎, 龚家元, 毕栋, 等. 基于 OpenCV 的锥桶识别技术研究 with 实现 [J]. 湖北汽车工业学院学报, 2019, 33(4): 4.
- [5] 卢远志. 智能车测试环境中虚拟车道线的快速构建方法 [J]. 中南大学学报(自然科学版), 2013, 03: 124-30.
- [6] 李松斌, 杨洁, 陈敬虎, 等. 一种基于图像颜色模式匹配的桶形道路路障检测方法 [P]. 2018.
- [7] 史林波. 自动驾驶中路锥车道的识别方法及其路径规划研究 [D]; 电子科技大学, 2020.
- [8] HUANG Y, XUE J. Real-time traffic cone detection for autonomous vehicle [J]. Chinese Control Conference Ccc, 2015.
- [9] DHALLA A, DAID, VAN GOOL L. Real-time 3D Traffic Cone Detection for Autonomous Driving; proceedings of the 2019 IEEE Intelligent Vehicles Symposium (IV), F, 2019 [C].
- [10] 赵轩. 基于单目视觉的前向车辆检测、跟踪与测距 [D]; 北京工业大学, 2017.
- [11] 刘军, 后士浩, 张凯, 等. 基于单目视觉车辆姿态角估计和逆透视变换的车距测量 [J]. 农业工程学报, 2018, 34(13): 70-6.
- [12] 吴强, 刘勇. 基于计算机视觉的停车场泊车定位研究 [J]. 2019.
- [13] 王振, 王化明. 基于单目视觉的障碍物定位和测量 [J]. 机械制造与自动化, 2015, 44(1): 4.
- [14] 韩峰. 基于 SURF 算法的单目转双目视觉定位 [J]. 计算机系统应用, 2015, 24(11): 5.
- [15] 毛先胤, 吕黔苏, 马晓红, 等. 基于双目视觉的电力线巡检机器人障碍物定位测距 [J]. 自动化与仪器仪表, 2021, 9: 5.
- [16] 王立勇, 王浩东, 苏清华. 基于深度学习的交通锥桶检测定位方法, 系统及存储介质 [P]. 2021.
- [17] GLASNER D, GALUN M, ALPERT S, et al. Viewpoint-aware object detection and pose estimation; proceedings of the 2011 International Conference on Computer Vision, F, 2011 [C].
- [18] TULSIANI S, MALIK J. Viewpoints and Keypoints [J]. 2015.
- [19] MORADI H, KARAMI M, SHAMAGHDARI S. DeepSDP: A Real-Time Deep Stereo Detection and Positioning Method for 3D Object Detection [C], proceedings of the 2020 28th Iranian Conference on Electrical Engineering (ICEE), F 4-6 Aug. 2020, 2020.
- [20] 王富奎. 高动态环境下智能车局部路径规划研究 [D]; 电子科技大学, 2018.
- [21] 韩月起, 张凯, 宾洋, 等. 基于凸近似的避障原理及无人驾驶车辆路径规划模型预测算法 [J]. 自动化学报, 2020, 46(1): 15.
- [22] 杜明博. 基于人类驾驶行为的无人驾驶车辆行为决策与运动规划方法研究 [D]; 中国科学技术大学, 2016.
- [23] 李麟, 裴玉龙, 尹亮, 等. 基于稳定性和宏观交通流模型的车辆路径规划 [J]. 中国公路学报,

- 2020, 33(8): 10.
- [24] 赵杰,胡浩然,孙启智,等. 改进果蝇算法的运输车辆路径规划 [J]. 黑龙江科技学院学报, 2020.
- [25] 李刚,金鸿耀,冀同涛,等. 一种无人驾驶赛车路径规划算法研究 [J]. 辽宁工业大学学报:自然科学版, 2021, 41(4): 4.
- [26] 王炳琪. 狭窄区域无人车路径规划与车辆控制研究 [D];上海交通大学,2018.
- [27] LIU J, ANAVATTI S, GARRATT M, et al. Modified continuous Ant Colony Optimisation for multiple Unmanned Ground Vehicle path planning [J]. Expert Systems with Applications, 2022, 196.
- [28] FERNANDES P B, OLIVEIRA R C L D, NETO J V F. A Modified QPSO for Robotic Vehicle Path Planning; proceedings of the 2018 IEEE Congress on Evolutionary Computation (CEC),F 8-13 July 2018,[C].
- [29] IBRAHIM M M S, ATIA M R, FAKHR M W. Autonomous Vehicle Path Planning using Q-Learning [J]. Journal of Physics: Conference Series, 2021, 2128(1): 012018.
- [30] MAW A A, TYAN M, LEE J-W. iADA*: Improved Anytime Path Planning and Replanning Algorithm for Autonomous Vehicle [J]. Journal of Intelligent & Robotic Systems,2020,100(3): 1005-13.
- [31] 白国星, 孟宇, 刘立, 等. 基于可变预测时域及速度的车辆路径跟踪控制 [J]. 中国机械工程, 2020, 31(11): 8.
- [32] 任玥, 冀杰, 赵颖, 等. 基于最小模型误差估计的智能汽车路径跟踪控制 [J]. 汽车工程, 2021, 43(4): 8.
- [33] 负海涛, 曾欣, 林晋召, 等. 车辆紧急避让路径跟踪神经网络自抗扰控制 [J]. 机械设计与制造, 2020, 9: 5.
- [34] 金立生, 谢宪毅, 司法, 等. 考虑驾驶人特性的智能驾驶路径跟踪算法 [J]. 汽车工程, 2021, 43(4): 9.
- [35] LEE K, JEON S, KIM H, et al. Optimal Path Tracking Control of Autonomous Vehicle: Adaptive Full-State Linear Quadratic Gaussian (LQG) Control [J]. IEEE Access, 2019, 7(109120-33).
- [36] HEWING L, KABZAN J, ZEILINGER M N. Cautious Model Predictive Control using Gaussian Process Regression [J]. 2017.
- [37] MOHAMMADZADEH A, TAGHAVIFAR H. A robust fuzzy control approach for path-following control of autonomous vehicles [J]. Soft Computing, 2019, 24(8).
- [38] 魏存超. 基于卷积神经网络的医学图像分类的研究 [D]; 哈尔滨工业大学,2017.
- [39] LECUN Y, KAVUKCUOGLU K, FARABET C. Convolutional Networks and Applications in Vision [C]; proceedings of the Proceedings of 2010 IEEE International Symposium on Circuits and Systems, F, 2010.
- [40] UNIVERSITY S."Supervised Learning:Pooling"[EB/OL].(<http://ufldl.stanford.edu/tutorial/supervised/Pooling/>).
- [41] STANFORD."CS231n:Convolutional Neural Networks for Visual Recognition"[EB/OL].(<http://cs231n.stanford.edu/>).

- [42] HE K, ZHANG X, REN S, et al. Spatial Pyramid Pooling in Deep Convolutional Networks for Visual Recognition [J]. Pattern Analysis & Machine Intelligence IEEE Transactions on, 2015, 37(9): 1904-16.
- [43] GIRSHICK R, DONAHUE J, DARRELL T, et al. rich feature hierarchies for accurate object detection and semantic segmentation tech report (v5) [J]. 2017.
- [44] REDMON J, DIVVALA S, GIRSHICK R, et al. You Only Look Once: Unified, Real-Time Object Detection [J]. IEEE, 2016.
- [45] LIU W, ANGUELOV D, ERHAN D, et al. SSD: Single Shot MultiBox Detector [J]. 2015.
- [46] WENG L. Object Detection Part 4: Fast Detection Models [J].
- [47] CAESAR H, BANKITI V, LANG A H, et al. nuScenes: A Multimodal Dataset for Autonomous Driving[C]; proceedings of the 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), F, 2020.
- [48] LIN T Y, MAIRE M, BELONGIE S, et al. Microsoft COCO: Common Objects in Context [J]. Springer International Publishing, 2014.
- [49] 周飞燕, 金林鹏, 董军. 卷积神经网络研究综述 [J]. 计算机学报, 2017, 6.
- [50] 吕铄, 蔡烜, 冯瑞. 基于改进损失函数的 YOLOv3 网络 [J]. 计算机系统应用, 2019, 28(02): 3-9.
- [51] 江金洪、鲍胜利、史文旭、韦振坤. 基于 YOLO v3 算法改进的交通标志识别算法 [J]. 计算机应用, 2020, v.40;No.360(08): 308-14.
- [52] LIN T Y, GOYAL P, GIRSHICK R, et al. Focal Loss for Dense Object Detection [J]. IEEE Transactions on Pattern Analysis & Machine Intelligence, 2017, PP(99): 2999-3007.
- [53] 岳晓新、贾君霞、陈喜东、李广安. 改进 YOLO V3 的道路小目标检测 [J]. 计算机工程与应用, 2020, v.56;No.964(21): 224-9.
- [54] WANG X, WANG S, CAO J, et al. Data-driven based tiny-YOLOv3 method for front vehicle detection inducing SPP-net [J]. IEEE Access, 2020, PP(99).
- [55] 姜义成, 李凡. 基于深度可分离卷积和多级特征金字塔网络的行人检测 [J]. 汽车安全与节能学报, 2020, 11(1): 8.
- [56] 张陶宁, 陈恩庆, 肖文福. 一种改进 MobileNet_YOLOv3 网络的快速目标检测方法 [J]. 小型微型计算机系统, 2021, 42(5): 7.
- [57] RITA N D, AIMAR A, DELBRUCK T. CNN-based Object Detection on Low Precision Hardware: Racing Car Case Study; proceedings of the 2019 IEEE Intelligent Vehicles Symposium (IV), F, 2019 [C].
- [58] HAN K, WANG Y, TIAN Q, et al. GhostNet: More Features From Cheap Operations; proceedings of the 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), F, 2020 [C].
- [59] 凌艳, 陈莹. 多尺度上下文信息增强的显著目标检测全卷积网络 [J]. 计算机辅助设计与图形学学报, 2019, 031(011): 2007-16.
- [60] 高翔、张涛. 视觉 SLAM 十四讲 [M]. 2017.3.

- [61] DHALL A. Real-time 3D Pose Estimation with a Monocular Camera Using Deep Learning and Object Priors On an Autonomous Racecar [J],2018.
- [62] HE K, ZHANG X, REN S, et al. Deep Residual Learning for Image Recognition [J]. IEEE, 2016.
- [63] FENG Z H, KITTLER J, AWAIS M, et al. Wing Loss for Robust Facial Landmark Localisation with Convolutional Neural Networks [J]. International Journal of Computer Vision, 2018.
- [64] STROBEL K, ZHU S, CHANG R, et al. Accurate, Low-Latency Visual Perception for Autonomous Racing:Challenges, Mechanisms, and Practical Solutions [J]. 2020.
- [65] 徐文立. 计算机视觉的 PNP 问题的最优解 [J]. 自动化学报, 1992.
- [66] 李罡. 广汽自主品牌乘用车制动系统(制动器,摩擦片)的设计与匹配 [J]. 华南理工大学.
- [67] 杨杰. 基于视觉的智能车辆换道过程横向运动控制研究 [D]; 华南理工大学,2019.
- [68] 谷广. 考虑驾驶特性的智能车辆协同换道轨迹规划 [D]; 华南理工大学,2018.
- [69] DELAUNAY B N. Sur la Sphère Vide [J]. Izvakadnauk Sssr, 1934.
- [70] LEE D T, SCHACHTER B J. two algorithms for constructing a delaunay triangulation 1 [J], 2017.
- [71] BERBERICH E. CGAL – Reliable Geometric Computing for Academia and Industry; proceedings of the International Congress on Mathematical Software, F, 2014 [C].
- [72] GURUJI A K, AGARWAL H, PARSEDIYA D K. Time-efficient A* Algorithm for Robot Path Planning [J]. Procedia Technology, 2016, 23(144-9).
- [73] FARIN G. Algorithms for rational Bézier curves [J]. Computer-aided design, 1983, 15(2): 73-7.
- [74] SHENG P, MA J, WANG D, et al. Intelligent trajectory planning model for electric vehicle in unknown environment [J]. Journal of Intelligent & Fuzzy Systems, 2019, 37(1): 397-407.
- [75] i-VISTA 自动驾驶汽车挑战赛[EB/OL].(<http://car.i-vista.org/>).
- [76] QUIGLEY M, FAUST J, FOOTE T, et al. ROS: an open-source Robot Operating System, F.
- [77] 马雪洁, 高蒙, 王新房. 全球无人驾驶汽车现状综述 [J]. 电脑知识与技术: 学术版, 2019, 15(7): 2.
- [78] 刘文涛. 基于曲率前馈的智能车辆路径跟踪算法 [J]. 汽车实用技术, 2019, 24: 3.
- [79] 明朗. 面向智能车室内测试工况的构建方法 [D]; 重庆理工大学,2021.
- [80] LI T, HU J, GAO L, et al. Agricultural machine path tracking method based on fuzzy adaptive pure pursuit model [J]. Nongye Jixie Xuebao/Transactions of the Chinese Society of Agricultural Machinery, 2013, 44(1): 205-10.
- [81] 赵治国, 周良杰, 朱强. 无人驾驶车辆路径跟踪控制预瞄距离自适应优化 [J]. 机械工程学报, 2018, 24: 8.
- [82] 唐小涛, 陶建峰, 李志腾, 等. 自动导航插秧机路径跟踪系统稳定性模糊控制优化方法 [J]. 农业机械学报, 2018, 49(1): 6.
- [83] 陈无畏, 谈东奎, 汪洪波, 等. 一类基于轨迹预测的驾驶员方向控制模型 [J]. 机械工程学报, 2016, 52(14): 10.
- [84] FRANKLIN G. Feedback Control of Dynamic Systems [M]. Feedback Control of Dynamic Systems, 2007.
- [85] 古月居[EB/OL].(<https://www.guyuehome.com/>).

- [86] 胡春旭. 基于 ROS 的云机器人平台设计与实现 [D]; 华中科技大学, 2015.
- [87] 曹正万, 平雪良, 陈盛龙, 等. 基于 ROS 的机器人模型构建方法研究 [J]. 2022, 8.

